

Informatik 2

Übungen in der virtuellen Labor-Umgebung

Prof. Dr. Nils Martini

Hochschule für Angewandte Wissenschaften Hamburg
Fakultät Informatik und Digitale Gesellschaft
Studiengang Medieninformatik

Inhaltsverzeichnis

0 Einführung.....	4
0.1 Die virtuelle Labor-Umgebung.....	4
0.2 Virtueller Server des EuC-Labors.....	4
0.2.1 eucv1.mt.haw-hamburg.de.....	4
0.2.2 Einrichten der Umgebungen.....	5
0.2.3 Zugang per RDP.....	5
0.2.4 Umgang mit den VMs.....	6
0.3 VirtualBox auf dem eigenen Rechner.....	6
0.3.1 Installation.....	6
0.3.2 Konfiguration.....	7
0.3.3 Virtuelle Netzwerkumgebung.....	8
1 Linux-Installation.....	9
1.1 Vorbereitungen.....	9
1.2 Installation des Betriebssystems Linux OpenSuse Leap-15.6.....	10
1.3 Software-Installation.....	11
1.4 Apache-Installation und Grundkonfiguration.....	13
2 Linux-Einführung und Shell-Skripte.....	15
2.1 Linux.....	15
2.2 Shell-Skripte.....	17
2.3 Lösungen zu Kapitel 2.2.....	19
3 Reguläre Ausdrücke.....	22
3.1 Übungsaufgaben zu sed.....	22
3.2 Übungsaufgaben zu awk.....	22
3.3 Reguläre Ausdrücke: „gieriges Verhalten von * “.....	23
3.4 Lösungen.....	23
3.4.1 sed.....	23
3.4.2 awk.....	23
4 Dateisysteme.....	25
4.1 MBR-Partitionierung.....	25
4.1.1 Partitionen anlegen.....	25
4.1.2 MBR-Block auslesen.....	26
4.1.3 Partitionstabelle der logischen Partition auslesen.....	27
4.2 GPT-Partitionierung.....	28
4.3 Formatierung mit einem Filesystem.....	29
4.4 Verschwendung von Plattenplatz bei kleinen Dateien.....	30
4.5 Superblock und Inodes.....	30
4.6 Synchrones oder asynchrones Schreiben.....	32
4.7 Journaling-Filesysteme ext3 und ext4.....	33
4.8 Filesystem BtrFS.....	34
4.9 Hinweise zur Interpretation der Kommando-Ausgaben.....	36
4.9.1 Aufgabenteil 2.1 (MBR-Partition):.....	36
4.9.2 Aufgabenteil 2.2 (GPT-Partition).....	37
5 IP-Adressierung.....	38
5.1 IPv4 und Netzwerk-Masken.....	38
5.1.1 Zwei LANs mit je einem Rechner.....	38
5.1.2 LAN um einen Rechner erweitern.....	39
5.1.3 Ein weiteres Subnetz hinzufügen.....	40
5.2 Address Resolution Protocol ARP.....	40
5.3 IPv6-Adressen.....	41
5.3.1 Grundkonfiguration.....	41
5.3.2 Neighbor-Solicitation.....	41
5.3.3 Router aktivieren.....	42
5.3.4 Unique Local/Global Address im Linux-Rechner.....	43
5.4 Lösungen.....	44
6 Routing-Verfahren.....	45
6.1 Grundaufbau.....	45
6.2 RIP-Routing.....	46

6.3 Distance-Vector-Routing.....	47
6.3.1 Routing Information Protocol RIP	47
6.3.2 Split Horizon.....	49
6.4 Statisches Routing.....	49
6.5 IPv6-Routing.....	50
7 OSPF-Routing.....	51
7.1 Grundaufbau.....	51
7.2 Stub-Netzwerke.....	52
7.2.1 Konfiguration des OSPF-Routing.....	52
7.2.2 Routing-Tabellen.....	53
7.2.3 Paket-Analyse.....	54
7.3 Transit-Netzwerke.....	57
7.4 Area Border Router ABR.....	58

0 Einführung

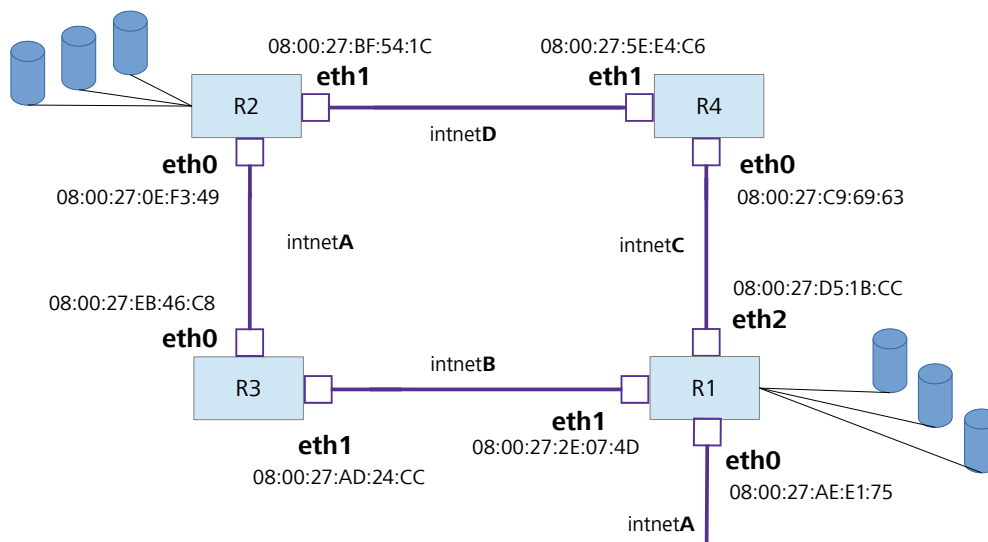
0.1 Die virtuelle Labor-Umgebung

Die Lehre im Modul „Informatik 2“ basiert im Wesentlichen auf praxisorientierten Laborübungen, deren Ziel es ist, ein vertieftes Verständnis der Vorgänge bei der Übertragung von Daten im Internet zu vermitteln. Für diese Übungen wird nicht nur eine realistische Netzwerkkumgebung benötigt sondern auch spezielle, in diesem Themenbereich in der heutigen Praxis eingesetzte Hardware.

Um trotzdem die Durchführung der Laborübungen außerhalb der Hochschule im Selbststudium zu ermöglichen, steht eine Umgebung aus virtuellen Maschinen zur Verfügung. Das hier vorliegende Aufgabenblatt fasst alle diejenigen Laborübungen zusammen, die ohne reale Netzwerke und Hardware in dieser virtuellen Umgebung durchführbar sind. Der theoretische Hintergrund zu den Übungsaufgaben ist im Vorlesungsskript dargestellt.

Vorausgesetzt werden erste Basiskenntnisse im Umgang mit dem Betriebssystem Linux.

Für das Labor stehen vier virtuellen Maschinen zur Verfügung. Diese virtuellen Rechner verfügen über mehrere Netzwerkschnittstellen, womit unterschiedliche Netzwerk-Topologien in den Übungen 5 und 6 aufgebaut werden können. Für die Übung 4 werden die Rechner R1 oder R2 verwendet, da hier zusätzliche Festplatten erforderlich sind.



Diese virtuellen Maschinen können auf dem virtuellen Server des EuC-Labors (siehe Kapitel 0.2) genutzt werden. Als Betriebssystem wird – wie auch im realen Labor – Linux OpenSuse-15.5 eingesetzt. Alternativ kann diese VM-Umgebung – bei ausreichender Hardware – auch auf dem eigenen Rechner installiert werden (Links zu den vorgefertigten VMs in Kapitel 0.3.3).

0.2 Virtueller Server des EuC-Labors

0.2.1 eucv1.mt.haw-hamburg.de

Die virtuelle Umgebung, mit der die Laborübungen von zuhause aus durchgeführt werden können, läuft auf einem Server des EuC-Labors. Die Umgebung wird mithilfe einer Web-Oberfläche eingerichtet. Die einzelnen virtuellen Maschinen werden auf diesem Server gestartet und können von zuhause aus mithilfe des Remote Desktop Protokolls (RDP) gesteuert werden. Auf dem heimischen Rechner sind hierzu *keine* zusätzlichen Software-Installationen oder Anpassungen erforderlich und es laufen dort auch *keine* virtuellen Maschinen.

Das EuC-Labor stellt diese Umgebung zur Verfügung, in der virtuelle Maschinen (mit denen die Labor-übungen teilweise durchgeführt werden) **jederzeit** auf dem Server des Labors gestartet werden können, auch außerhalb der Vorlesungszeiten. Unter dem URL <https://eucv1.mt.haw-hamburg.de> wird der Virtualisierungs-Server des EuC-Labors erreicht. Dort kann die gewünschte VM-Umgebung gestartet werden. Unter Windows 11 wird die (meist) standardmäßig installierte „Remotedesktopverbindung“ verwendet (Startleiste „Windows-Tools“) und eine Verbindung hergestellt zum genannten URL zzgl. der RDP-Port-Nummer, die mit der die jeweilige individuelle virtuelle Maschine verknüpft ist. Das Passwort für alle Benutzer auf den VMs lautet: `geheim`

0.2.2 Einrichten der Umgebungen

Der Login auf dem Server eucv1.mt.haw-hamburg.de erfolgt per Browser mit der individuellen HAW-Kennung und dem eigenen Passwort. Auf der Startseite steht unter „VM Environment“ eine Auswahl verschiedener virtueller Umgebungen (u.a. Informatik 2) zur Verfügung. Wichtiger Hinweis: Es kann **nur jeweils eine** der Umgebungen gleichzeitig aktiviert werden. Sollte bereits eine Umgebung eingerichtet worden sein, so überschreibt die Aktivierung einer anderen Umgebung alle vorherigen Einstellungen. Nach der Aktivierung der Umgebung stehen die jeweiligen virtuellen Maschinen sofort lauffähig zur Verfügung. Alle Maschinen laufen auf dem Server des EuC-Labors, der auch alle erforderlichen Ressourcen zur Verfügung stellt. Daher ist auf dem heimischen Rechner *keine* Installation einer Virtualisierungs-Software oder virtueller Maschinen erforderlich, lediglich ein RDP-Client muss vorhanden sein (siehe Kapitel 0.2.3). Im Bereich „VM Images“ werden die Maschinen der gewählten Umgebung dargestellt. Sollte eine Maschine nicht aktiv laufen, so ist dies an dem rot gefärbten Einschalt-Button zu erkennen; ein Klick auf den roten Button startet die Maschine. Ist dieser Button grün, so läuft die Maschine und kann mittels RDP-Client erreicht werden (das Herunterfahren der Maschine kann über die entsprechende Funktion in der laufenden VM erfolgen oder durch Anklicken des grünen Buttons).

Video in der Mediathek (Channel „Informatik 2“):

[Virtuelles Labor \(Zugang zur virtuellen Lernumgebung\), Kurz- und Langfassung](#)

0.2.3 Zugang per RDP

Die Verbindung zu den einzelnen virtuellen Maschinen erfolgt mittels RDP (Remote Desktop Protocol). In Windows 10 steht die Anwendung unter „Startmenü, Windows-Zubehör, Remotedesktopverbindung“ zur Verfügung, in Windows 11 unter „Startmenü/alle Apps, Windows-Tools, Remotedesktopverbindung“. Für Linux und MacOS stehen ebenfalls freie RDP-Clients zur Verfügung, die i.d.R. gesondert installiert werden müssen. Hinweise zu RDP-Clients und deren Bedienung finden sich unter „Documentation“ und dem Symbol für das jeweilige Betriebssystem (externe Links).

Es muss für jede der vier virtuellen Maschinen eine *eigene* RDP-Verbindung mit der jeweils individuellen Port-Nummer hergestellt werden. Der Zugang zu den eigenen virtuellen Maschinen erfolgt mit dem URL

`eucv1.mt.haw-hamburg.de:RDP-Port`

der in der jeweiligen RDP-Client-Software einzugeben ist. Der für (jede einzelne Maschine eindeutige) RDP-Port ist unter der jeweiligen Maschine im Bereich „VM-Images“ angegeben. Zur Vereinfachung der Eingabe ist dort neben dem URL ein Copy-Symbol zu finden. Zusätzlich erfolgt eine Authentifizierung: es muss der eigene HAW-Benutzername (Format: wxy123) angegeben und (MS Windows) das Feld „Speichern von Anmeldeinformationen zulassen“ aktiviert werden. Danach erfolgt eine Abfrage desjenigen Passwortes, das im Bereich „VM Images“ unter dem URL zu finden ist (es existiert ebenfalls ein Copy-Symbol).

Nach dem ersten Start muss ggf. in der Windows-Firewall der Zugriff des RDP-Clients auf das Netzwerk erlaubt werden.

Der Login auf den VMs selbst erfolgt für alle Benutzer (`root` oder `student`) mit dem Passwort `geheim`

Für eine stabile Verbindung zu den jeweiligen virtuellen Maschinen ist (insbesondere bei mehreren gleichzeitig genutzten Verbindungen) ein Internet-Anschluss mit ausreichender Übertragungsrate erforderlich. Bei geringen Übertragungsraten oder anderweitig stark ausgelasteten Leitungen, kann die grafische Darstellung „hakelig“ werden, die RDP-Verbindung instabil werden und abbrechen. Dies hat jedoch *keine* Auswirkungen auf die virtuellen Maschinen selbst, die auf dem Server weiterlaufen und nach einem erneuten RDP-Verbindungsaufbau sofort wieder im Zugriff sind. In diesen Fällen sollten RDP-Verbindungen mit geringerer Auflösung gewählt werden (Windows-RDP-Client: „Optionen einblenden“, Anzeige, Farben (*nicht* die höchste Qualität auswählen) oder unter „Leistung“ die Verbindungsqualität manuell einstellen).

Die Bildschirmauflösung der VMs ist für die HD-Auflösung optimiert (1920*1080). Bei (deutlich) abweichenden Auflösungen am RDP-Rechner können folgende Einstellungen verändert werden (für Windows10/11): Rechte Maustaste am oberen Fensterrand der Remotedesktop-Verbindung, „intelligente Größenänderung“ abwählen, dann erneut rechte Maustaste, unter „Zoomen“ gewünschten Vergrößerungsfaktor einstellen. Falls lediglich die Schrift z.B. im Terminalfenster, im Editor oder in Wireshark zu klein sein sollte: Mit Strg-+ lässt die die Schrift vergrößern. Die Skalierung der gesamten grafischen Ausgabe der VM lässt sich einstellen unter: Start-Button (unten links) – Settings – System Settings, dann „Display and Monitor“ und dort „Global Scale“ anpassen (anschließend aus- und wieder einloggen).

0.2.4 Umgang mit den VMs

Alle virtuellen Maschinen der gewählten Umgebung stehen für eine *individuelle* Nutzung zur Verfügung, d.h. jede Nutzerin und jeder Nutzer kann die Maschinen individuell konfigurieren. Alle Einstellungen, die an den eigenen virtuellen Maschinen vorgenommen werden, bleiben solange erhalten, bis die Umgebung gelöscht wird.

Es ist empfehlenswert, sich bei vorübergehender Abwesenheit auszuloggen und die Maschinen nach der Nutzung herunterzufahren (dies geschieht automatisch nach einer bestimmten Zeit der Nicht-Nutzung). Die virtuellen Maschinen verfügen über keine externe Netzwerkverbindung. Daher sind weder Updates oder zusätzliche Software-Installationen möglich.

Hinweis: Das EuC-Labor kann keine 100%ige Verfügbarkeit des Systems garantieren, da weder die personellen noch technischen Ressourcen einen dafür notwendigen hochverfügbaren Rechenzentrumsbetrieb ermöglichen.

0.3 VirtualBox auf dem eigenen Rechner

Dieser Abschnitt beschreibt die Installation und Konfiguration von „VirtualBox“ auf dem eigenen Rechner. Dies ist für die Durchführung der Laborübungen grundsätzlich *nicht* zwingend erforderlich. Daher braucht dieses Kapitel nur dann beachtet zu werden, wenn die Installation der virtuellen Umgebung auf der eigenen Hardware erwünscht ist!

0.3.1 Installation

Als *Alternative* zum virtuellen Server des EuC-Labors steht die vorgefertigte virtuelle Umgebung auch für die Installation auf dem heimischen Rechner zur Verfügung. Die VMs können in diesem Fall auch ohne aktive Internet-Verbindung auf der eigenen Hardware verwendet werden.

Hierzu müssen die Software **Oracle VirtualBox** und das **Extension Package** für das verwendete Host-Betriebssystem installiert werden. Download unter:

<https://www.virtualbox.org>

Nach dem Start des VirtualBox Managers kann unter „Datei – Einstellungen – Zusatzpakete“ das Extension Package eingebunden werden (Hinweis zur Lizenz: Das ExtensionPack ist nur bei nicht-kommerziellem Einsatz frei verwendbar). Ohne Extension Pack funktioniert der Anschluss eines USB-Sticks oder einer USB-Festplatte an die virtuellen Maschinen nicht (die Durchführung der Laborübungen wird dadurch jedoch nicht beeinträchtigt).

Der Prozessor des Host-Rechners muss virtuelle Maschinen (VM) unterstützen. Oftmals ist diese Funktion nicht aktiviert und muss daher im BIOS eingeschaltet werden. Die genaue Vorgehensweise und die jeweiligen Bezeichnungen im BIOS sind von der verwendeten Hardware abhängig (bspw. „CPU Configuration“ – „Intel Virtualization“ – „Enabled“ oder „Security – Virtualization – AMD V“).

Für das **Labor informatik 2** steht eine vorgefertigte virtuelle Netzwerkumgebung mit vier virtuellen Maschinen zur Verfügung. Als Betriebssystem wird – wie auch im realen Labor – OpenSuse-15.6 eingesetzt. Die aktuellen Links zu den vorgefertigten virtuellen Maschinen (ZIP-Ordner) finden sich in Kapitel 0.3.3.

Die VMs können entweder direkt von einer externen USB-Festplatte (SSD, NTFS-formatiert) gestartet werden oder zunächst auf die lokale Festplatte in das Verzeichnis „VirtualBox VMs“ (ggf. neu anlegen) kopiert werden; es werden ca. 30 GB freier Speicher auf der Festplatte (möglichst SSD) benötigt. Werden

die VMs von einer externen USB-SSD gestartet, muss unbedingt ein USB-3.0 oder USB-3.1-Anschluss verwendet werden und es ist empfehlenswert, einen festen Laufwerksbuchstaben für die USB-SSD zu vergeben.

Die VMs müssen in VirtualBox eingebunden werden; hierzu den VirtualBox Manager starten:

- Menüleiste: Maschine (oder Gruppe) – Maschine Hinzufügen
- in das VirtualBox-Verzeichnis wechseln
- dort die gewünschte VBOX-Datei auswählen (z.B. Unterverzeichnis „INFB_R1“ dort die Datei „R1.vbox“ auswählen)
- dies muss für alle vier VMs einzeln durchgeführt werden

Die Voreinstellungen der virtuellen Hardware der VMs sind so gewählt, dass alle vier VMs auf einem Host-Rechner mit 16 GB RAM, Bildschirm bzw. Grafikkarte mit HD-Auflösung und einem 2-Kern-Prozessor (mit je zwei logischen Prozessoren) laufen können. Bei geringerer Leistungsfähigkeit des Host-Rechners können evtl. Anpassungen der VMs an die reale Hardware-Umgebung erforderlich sein (s.u.).

Start der VMs: Entweder in der linken Spalte die gewünschte Maschine anklicken und „Start“ (oder Doppelklick) oder die Gruppenüberschrift „Labor_R1“ anklicken und „Start“, dann werden alle vier VMs gestartet (Empfehlung bei einfacher Hardware: Maschinen einzeln nacheinander starten). Nach dem ersten Start muss ggf. in der Windows-Firewall der Zugriff der VMs auf das Netzwerk erlaubt werden.

Der Login erfolgt mit dem Benutzer `root` (Passwort: `geheim`). In vielen Laborübungen werden die meisten Kommandos nur auf einem der Rechner eingegeben. Für eine bessere Bedienbarkeit kann dieser Rechner mit der Tastenkombination `Strg(rechts) – F` in den Vollbildmodus umgeschaltet werden (Vollbildmodus verlassen mit der gleichen Tastenkombination oder mit der verdeckten Menüleiste am unteren oder oberen Bildschirmrand).

Herunterfahren der VMs: Die virtuellen Rechner können wie reale Rechner heruntergefahren werden: Start-Button unten links, Herunterfahren oder Reboot. Die VMs können auch unter dem Menü „Maschine – ACPI-Shutdown“ gestoppt werden (vorher ausloggen). Wird das VM-Fenster direkt geschlossen, gehen i.d.R. alle Änderungen, die während einer Sitzung entstanden sind, verloren. Oftmals ist es aber ohnehin sinnvoll, die Maschine wieder in der Urzustand zu versetzen, indem das vorgefertigte Snapshot zurückgespielt wird: „Maschine – Werkzeuge – Sicherungspunkte“.

0.3.2 Konfiguration

Die vorgefertigten virtuellen Maschinen sind für eine übliche Rechner- und Betriebssystemumgebung vorbereitet, sodass i.d.R. keine Änderungen an der Konfiguration erforderlich sein sollten. Falls ggf. Anpassungen der VMs an die reale Hardware-Umgebung vorgenommen werden sollen, finden sich alle Einstellungen unter dem Menü „Maschine bzw. Gruppe – Werkzeuge – Konfiguration“ (alternativ auf die drei Punkte rechts neben dem Maschinennamen klicken, dort „Details“ auswählen). Dies ist aber nur dann erforderlich, wenn die vorhandene Host-Hardware deutlich von den oben genannten Vorgaben abweicht oder spezielle Anpassungen vorgenommen werden sollen.

- Die vorgefertigten VMs verwenden jeweils 2 GB RAM: Der Host-Rechner sollte daher über 16 GB RAM verfügen (bei größerem Hauptspeicher des Host-Rechners kann dieser Wert auch erhöht werden). Konfigurationsbereich „System“
- Es werden pro VM zwei Prozessoren verwendet (bei leistungsfähiger Host-Hardware, kann dieser Wert auch erhöht werden). Konfigurationsbereich „System“
- Als Grafikspeicher sind 32 MB pro VM voreingestellt: Dieser Wert kann verringert werden, allerdings könnte dann bei hochauflösender Grafik der Vollbildmodus (Tastenkombination `Strg(rechts) – F`) nicht mehr funktionieren. Größere Werte sollten jedoch bei einfacherer Hardware nicht verwendet werden. Konfigurationsbereich „Anzeige“
- USB-3.0 ist aktiviert (falls das Extension Pack nicht installiert ist, muss auf USB-1.1 umgestellt werden). Konfigurationsbereich „USB“
- Die Bildschirmauflösung der VMs ist für die HD-Auflösung optimiert (1920*1080). Bei (deutlich) abweichenden Auflösungen am Host-Rechner können folgende Einstellungen verändert werden:
 - Im Konfigurationsbereich „Anzeige“ den Skalierungsfaktor einstellen.

- Oder *nach* dem Start der VM im Menü „Anzeige – Virtueller Monitor“ die gewünschte Skalierung anwählen und ggf. die grafische Oberfläche der Linux-VM neu starten (ausloggen und wieder einloggen)
- Falls nach dem Einloggen die Schriften zu klein sein sollten: Start-Button (unten links) – Settings – Configure Desktop, dann „Display and Monitor“ und dort „Scale-Display“ anpassen.
- Die Schriftart und -größe der Konsole (Terminalfenster) kann in der Werkzeugleiste unter „Settings – Edit current Profile – Appearance“ angepasst werden.
- Falls lediglich die Schrift im Terminalfenster zu klein sein sollte, kann sie mit Strg-+ schrittweise vergrößert werden.
- Hinweis: Die automatische Anpassung der VM-Anzeige beim Vergrößern/Verkleinern des VM-Fensters funktioniert nur bei aktivierter Gasterweiterung (Guest Additions)

0.3.3 Virtuelle Netzwerkkumgebung

Die vier VMs aus der vorgefertigten virtuellen Netzwerkkumgebung für die Installation **auf dem eigenen Rechner** können unter den folgenden Links heruntergeladen und in VirtualBox eingebunden werden (vier Zip-Archive à ca. 4 GB, entpackt insgesamt ca. 40 GB, Passwort für Benutzer „root“ bei allen Maschinen: geheim):

- INF2_R1: <https://cloudy.mt.haw-hamburg.de/s/bBnZkCGgRrjc7yM>
- INF2_R2: <https://cloudy.mt.haw-hamburg.de/s/34TPHzANwXZ7KEa>
- INF2_R3: <https://cloudy.mt.haw-hamburg.de/s/ca2EeqmdqtKzk7B>
- INF2_R4: <https://cloudy.mt.haw-hamburg.de/s/kcjgdKTsk8FtFyd>

Stand Wintersemester 2025/2026: Die vorgefertigten VMs (OpenSuse-15.6) sind für die VirtualBox-Version 7.2.2 auf einem Windows11-Host optimiert; die entsprechende GuestAddition ist installiert. Die VMs laufen i.d.R. aber auch auf einem Linux- oder MacOS-Host.

1 Linux-Installation

Diese Übung kann nur in einer lokalen VirtualBox-Umgebung durchgeführt werden (zu Hause oder im Labor). Die Durchführung in der virtuellen Laborumgebung ist **nicht** möglich.

1.1 Vorbereitungen

In dieser Laborübung soll das Betriebssystem OpenSuse-Leap-15.6 als virtuelle Maschine in der Virtualisierungssoftware Oracle VirtualBox-7.1 installiert werden. Wird diese Übung auf dem eigenen Rechner ausgeführt, so muss zunächst VirtualBox installiert werden (auf den Rechnern in der Hochschule ist VirtualBox bereits vorhanden). Einzelheiten zur Installation von VirtualBox finden sich in Kapitel 0.3.

Für die Installation von OpenSuse-Leap-15.6 sind folgende Vorbereitungen erforderlich (*nur* auf dem eigenen Rechner; die Rechner in der Hochschule sind bereits entsprechend vorbereitet, hier weiter mit Kapitel 1.2):

- Download der Installationsquelle von www.opensuse.org, „Leap installieren“, „Download“, i.d.R. Auswahl des DVD-Image für Intel/AMD 64 Bit („Offline Image“, ca. 4,1 GB)
- Speichern dieser ISO-Datei
- Eine entsprechend vorgefertigte virtuelle Maschine, mit der die Installation sofort gestartet werden kann, ist in Moodle bereitgestellt („OpenSuse-15.6_leer.zip“) und muss in VirtualBox eingebunden werden (siehe „Virtualbox_Installation_INFB.pdf“)
- In dieser VM muss zunächst die ISO-Datei als DVD eingebunden werden:
 - In der linken Spalte des VirtualBox-Managers das Listensymbol (3 Punkte) neben dem Namen der neuen Maschine „OpenSuse_15.6_leer“ einmal anklicken (*nicht* Doppelklick), dann „Details“ auswählen und im rechten Bereich „Massenspeicher“ anklicken.
 - Im sich öffnenden Fenster unter der Zeile „Controller: IDE“ das DVD-Symbol (mit der Bezeichnung „leer“) anklicken und im rechten Bereich „Optisches Laufwerk“ die neue ISO-Datei mittels des anderen DVD-Symbols **oben, ganz rechts** einbinden (DVD-Symbol – „Abbild auswählen“, ISO-Datei „openSUSE-Leap-15.6...iso“ auswählen und mit OK abschließen). Falls eine Opensuse-15.6-ISO-Datei bereits vorhanden sein sollte, so muss diese zuvor entfernt werden.
- Danach kann die neue Maschine gestartet werden; der weitere Ablauf der Laborübung ist in Kapitel 1.2 erläutert.
- Falls bei der Installation unerwünschte oder fehlerhafte Eingaben gemacht werden sollten, so kann die Maschine jederzeit ausgeschaltet werden (Fenster schließen) und der vorgefertigte Sicherungspunkt (Snapshot) eingespielt werden. Hierzu die drei Punkte neben dem Maschinennamen anklicken, „Sicherungspunkte“ auswählen, dann „Sicherungspunkt leer“ mit der rechten Maustaste anklicken und „Wiederherstellen“ auswählen. Alle Änderungen werden damit zurückgesetzt.

Alternativ (statt der vorgefertigten Maschine) kann in VirtualBox eine neue virtuelle Maschine selbst angelegt werden:

- VirtualBox-Manager: Maschine – Neu, beliebiger Name, Ordner: (Standardordner für VirtualBox auswählen), ISO-Abbild auswählen („OpenSuse-Leap-15.6...iso“), Typ „Linux“, Version „Opensuse (64 Bit)“
- Im Folgenden weitere Einstellungen vornehmen: Hauptspeicher (RAM) empfohlen 2048 MB (in Abhängigkeit von der Host-Hardware ggf. auch weniger), Prozessoren mindestens 2, EFI anwählen, dann „jetzt eine virtuelle Festplatte erstellen“, Größe der Platte 15 GB, dann weiter und „Fertigstellen“
- Die neu angelegte Maschine benötigt eine zweite virtuelle Festplatte:
 - In der linken Spalte des VirtualBox-Managers das Listensymbol (3 Punkte) neben dem Namen der neuen Maschine einmal anklicken (*nicht* Doppelklick), dann „Details“ auswählen und im rechten Bereich „Massenspeicher“ anklicken.
 - Im sich öffnenden Fenster die Zeile „Controller SATA“ anklicken, „Festplatte hinzufügen“ (Plus-Symbol ganz rechts neben SATA), im sich öffnenden Fenster: „Erzeugen“, „VDI“, Art der Speicherung „dynamisch“ (hier also nichts anwählen), Größe 15 GB, dann „Fertigstellen“.
 - Die neu erstellte Festplatte anschließend in der Liste markieren und „auswählen“ und das Einstellungsfenster mit „OK“ schließen.
- Danach kann die neue Maschine gestartet werden; der weitere Ablauf der Laborübung ist in Kapitel 1.2 erläutert.

1.2 Installation des Betriebssystems Linux OpenSuse Leap-15.6

Die Installation des Betriebssystems erfolgt in einer virtuellen Maschine in der Virtualisierungssoftware Oracle VirtualBox. Vor Beginn der Übung am heimischen Rechner muss entweder die vorgefertigte virtuelle Maschine „OpenSuse_15.6_leer“ in VirtualBox eingebunden werden (siehe oben) oder selbst eine neue virtuelle Maschine erstellt werden. Das Installationsmedium ist als ISO-Image auf der realen Festplatte abgelegt und wird als virtuelles optisches Laufwerk eingebunden (wie in Abschnitt 1.1 beschrieben).

Installation in VirtualBox starten:

- VirtualBox starten
- „OpenSuse_15.6_leer“ (einmal anklicken), dann „Starten“ (alternativ: Doppelklick auf dem Maschinennamen).
- Falls nach dem Start das Fenster zu klein sein sollte (abhängig von der Bildschirmauflösung des Host-Rechners), in der Werkzeugleiste oben „Anzeige“ wählen und unter „Virtueller Monitor 1“ eine geeignete Skalierung auswählen.
- Zeile „Installation“ auswählen – Enter
- das Benachrichtigungsfenster zur Einbindung von Maus und Tastatur kann geschlossen werden.

Nach dem Start der Installation werden zunächst einige Grundeinstellungen abgefragt:

- Sprache Deutsch (wichtig wegen der Tastaturtreiber)
- Online-Repositories aktivieren (nur falls der Host-Rechner eine Internet-Verbindung hat), Voreinstellungen belassen
- Systemrolle: Desktop mit KDE-Plasma

Danach erfolgt ein entscheidender Schritt: die **Partitionierung der Festplatte**. Auf einem einfachen Desktop-PC wäre die hier vorgeschlagene Standard-Einstellung (/boot/efi, / (root) und /home) eine gute Wahl, in dieser Übung sollen Sie jedoch eine eigene Partitionierung erstellen. Die Aufteilung der Partitionen auf den beiden zur Verfügung stehenden Festplatten könnte wie folgt aussehen:

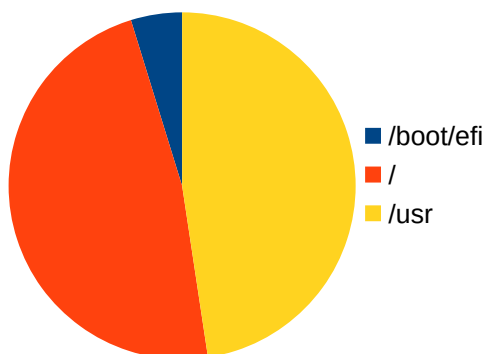


Abbildung 1: erste Festplatte /dev/sda

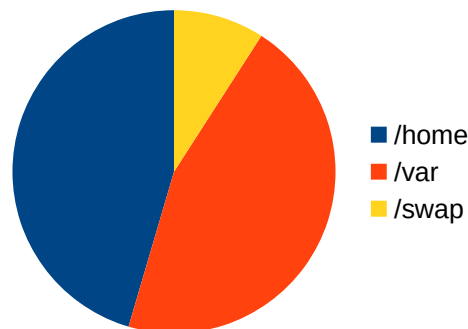


Abbildung 2: zweite Festplatte /dev/sdb

Die Wahl der Partitionen und ihrer Größen sollte überlegt erfolgen, da eine nachträgliche Änderung im laufenden System entweder gar nicht oder nur mit Aufwand realisierbar ist.

Die Partitionierung erfolgt hier mit einer GPT-Partitionstabelle (sollte „im wahren Leben“ auf einer realen Festplatte ein vorinstalliertes Windows oder anderweitig eine GPT-Partitionierung bereits vorhanden sein, so darf *keinesfalls* eine existierende **EFI-Boot-Partition** gelöscht oder formatiert werden; sie dient für alle Betriebssysteme *gemeinsam* als Speicherort für die Boot-Loader!

Vorgang:

- Im Fenster „Vorschlag für Partitionierung“ – „Experten-Partitionierer“ – Start mit vorhandenen Partitionen
- In dem sich öffnenden Fenster sollten die beiden Festplatten (Größe je 15 GiB) mit ihren Bezeichnungen /dev/sda und /dev/sdb erscheinen
- Markieren von /dev/sda dann unten auf die Schaltfläche „Partition hinzufügen“

- Zunächst die Partition für `/boot/efi` mit einer Größe von 300 MiB erstellen: „Neue Partitionsgröße“ – Benutzerdefinierte Größe – Größe: 300 MiB – Weiter
- Rolle: „EFI Boot Partition“ – Weiter
- Formatierungsoptionen (ist bereits voreingestellt, darf nicht geändert werden): Dateisystem FAT, Gerät mounten: Mount Point `/boot/efi` – Weiter
- Nun die nächste Partition auf der Festplatte `/dev/sda` auf die gleiche Weise erstellen: wieder `/dev/sda` markieren, „Partition hinzufügen“, dann „Benutzerdefinierte Größe“ **8 GiB**, Rolle: Betriebssystem, Dateisystem Btrfs, Mount Point `/usr` (muss manuell im Textfeld „Mountpoint“ eingetippt werden, *nicht* das automatisch erscheinende `/usr/local` verwenden) - Weiter
- und nochmals eine letzte Partition auf `/dev/sda` erstellen: die verbleibende maximale Größe verwenden, Rolle: Betriebssystem, Dateisystem Btrfs, Gerät mounten: Mount Point `/`
- Das gleiche Vorgehen für die Platte `/dev/sdb` (`/dev/sdb` anklicken), Vorschlag:
 - swap (Rolle Swap) 1 GiB, Dateisystem swap, Mount Point `swap`
 - `/home` (Rolle Daten...) 8 GiB, Dateisystem XFS
 - `/var` (Rolle Daten...) restlicher Platz, Dateisystem XFS
- Vorgang mit „Übernehmen“ abschließen, dann „Weiter“

Im nächsten Schritt die Uhrzeit einstellen, dann Benutzer und Passwort festlegen:

- Synchronisation mit NTP-Server (kann so übernommen werden) - Weiter
- Benutzer festlegen:
 - Der Benutzername kann beliebig (aus Buchstaben, Zahlen) gewählt werden.
 - Passwort festlegen (Achtung: unbedingt merken!!)
 - Passwort auch für den Administrator verwenden (Haken setzen)
 - „Automatische Anmeldung“ **nicht** anwählen (sonst wird immer der frei gewählte Benutzer beim Systemstart angemeldet, später muss aber mit dem Administrator (Benutzername „root“) gearbeitet werden)
- In der folgenden Zusammenfassung aller Installationseinstellungen:
 - „Firewall deaktivieren“ (Abschnitt „Sicherheit“)
 - und ganz unten (Abschnitt „Netzwerkkonfigurationen“), Networkmanager: „Wechseln zu wicked“.

Mit „Installieren“ beginnt der eigentliche Installationsprozess (ggf. die dann folgenden Abfragen bestätigen). Der nun gestartete Vorgang dauert je nach Hardware ca. 10 - 20 Minuten.

Nach der erfolgreichen Installation und einem automatischen Neustart melden Sie sich mit der Benutzerkennung „root“ und dem von Ihnen gewählten Passwort an (auf dem Login-Screen „Weitere...“ wählen, dann Benutzername „root“ eingeben. Sollten Sie das Passwort vergessen haben, so beginnen Sie einfach den gesamten Installationsprozess von vorne!

1.3 Software-Installation

Auf dem laufenden Betriebssystem können nun weitere Softwareprodukte installiert werden. OpenSuse verwendet hierzu eine grafische Oberfläche namens „yast2“; aus den verschiedenen so genannten Software-Repositories können die gewünschten Dateien über das Netzwerk einfach geladen und installiert werden. Hierzu werden i.d.R. vorgefertigte Installationsdateien (Paketmodule RPM) verwendet.

Für die verschiedenen Linux-Distributionen existieren auch verschiedene Paketverwaltungs-Tools. Die meisten Distributionen verwenden als Basis das Paketverwaltungssystem RPM (mit Ausnahme von Debian und den davon abgeleiteten Varianten wie Ubuntu). Dieses System ist jedoch sehr einfach aufgebaut und kann weder direkt auf Paketquellen (Repositories) zugreifen noch selbstständig Abhängigkeiten zwischen Paketen auflösen. Daher existieren zusätzlich darauf aufbauende Paketverwaltungssysteme wie `zypper` (OpenSuse), `yum` (Red Hat) oder `apt` (Debian, Ubuntu).

Manchmal ist es jedoch erforderlich oder (z.B. auch aus Gründen der Systemsicherheit) erwünscht, Programme aus Quelldateien selbst zu kompilieren und im System zu installieren. Hierzu ist es erforderlich, dass entsprechende Compiler installiert sind (i.d.R. `gcc` und `gcc-c++`) außerdem Werkzeuge zur Vorbereitung des Kompilierens (`make` und/oder `cmake`) sowie die Entwicklerversionen bzw. der Quellcode der einzubindenden Bibliotheksfunktionen (meistens an der Ergänzung „-devel“ im Namen erkennbar). Dieser Vorgang soll beispielhaft anhand einer Software verdeutlicht werden, die auch bei den später noch folgenden Laborübungen verwendet werden wird: „Mause Zahn“ (ein IP-Paket-Generator).

Laden Sie die Quelldateien des Software-Pakets „netsniff-ng“ (für die folgenden Vorgänge ist es erforderlich, dass der Host-Rechner eine aktive Internet-Verbindung hat):

- Start des Browsers (Task-Leiste unten: Firefox)
- Download des Software-Pakets von <https://software.opensuse.org>
- dort im Feld „Search packages“: `netsniff-ng`, dann unter Distribution `Opensuse-Leap-15.6`, „Show experimental packages“, „Expert Download“ wählen, „OpenSuse“, dann „Binärpakete direkt herunterladen (Grab binary packages directly)“, „Packages for 15.6“, „netsniff-ng-...**src**.rpm“

Nun kann das Paket installiert werden (alle Datei- und Verzeichnisnamen können mit der Tab-Taste expandiert werden):

- Konsole öffnen (Start-Button unten links, System – „Terminal SuperUser Mode“)
- Falls gewünscht, kann die Schrift mit `Strg +` vergrößert werden
- Wechsel in das Downloads-Verzeichnis `cd ~/Downloads`
- Installieren des Quellcode-Paketes: `rpm -i netsniff` (dann Tab-Taste drücken)
- Kopieren der Quelldateien ins Download-Verzeichnis:
`cp /usr/src/packages/SOURCES/netsniff-ng-0.6.9.tar.xz ~/Downloads`
- entpacken des tar-Archivs: `tar xvfJ netsniff-ng-0.6.9.tar.xz`
- Wechsel in das Unterverzeichnis, in dem sich die Quellcode-Dateien befinden:
`cd netsniff-ng-0.6.9`

Mit dem Befehl `./configure` wird das Kompilieren vorbereitet und es wird überprüft, ob die erforderlichen Voraussetzungen im Betriebssystem gegeben sind (z.B. ob überhaupt ein geeigneter Compiler zur Verfügung steht). Man wird nach Eingabe dieses Befehls feststellen: es funktioniert *nicht*, da der erforderliche C-Compiler nicht vorhanden ist. Der Compiler muss zunächst installiert werden: Administrations-Tool „**Yast**“ starten (Startbutton unten links, Applications, Settings, Yast (oder Befehl `yast2` im Terminalfenster)).

- In der linken Spalte „Software“ auswählen und rechts „Software Management“
- im Search-Feld „gcc“ eingeben, in der Liste nach `gcc-c++` suchen und dort ein Häkchen setzen (ein Plus-Zeichen erscheint),
- dann „accept“
- Es werden automatisch einige Abhängigkeiten mit anderen Softwarepaketen aufgelöst, dann „Continue“
- Dann das gleiche Vorgehen für das Softwarepaket `cmake`
- Abschließend: „Finish“.

Nun kann erneut der `configure`-Befehl gestartet werden (Hinweis: vorangegangene Befehle brauchen nicht erneut eingetippt zu werden, sie können mit der Cursor-Up-Taste aufgerufen werden). Die Fehlermeldung aufgrund des fehlenden Compilers sollte nun nicht mehr erscheinen, dafür wird eine Liste mit im System vorhandenen und nicht vorhandenen Software-Paketen ausgegeben, die zum Kompilieren erforderlich sind. Man erkennt auch, dass das Tool „mausezahn“ in diesem Zustand noch nicht installiert werden kann („tools will *not* be built“). Wie die Ausgabe zeigt („Checking libpcap ... [no]“) sind einige weitere sog. Development-Pakete nötig, z.B. „libpcap“ (Software-Bibliothek für das „Packet-Capturing“). Dieses und andere Pakete können entweder wie zuvor mithilfe der grafischen Admin-Oberfläche `yast` installiert werden oder mit dem Kommando `zypper`:

- Welchen exakten Namen das Development-Paket hat, kann zunächst mit `zypper search libpcap` festgestellt werden
- anschließend Installation mit `zypper install libpcap-devel` (Abfrage mit `y` bestätigen)

Auch dies ist noch nicht ausreichend, wie ein erneuter Start von `./configure` zeigt. Es müssen auf dieselbe Weise weitere Pakete installiert werden: `libcli-devel` und `libnet-devel`. Anschließend sollte `./configure` anzeigen („tools will be built“), dass nun „mausezahn“ kompiliert werden kann. Hierzu den Befehl `make` eingeben.

Der letzte Schritt besteht in der Einbindung der kompilierten Programme in das Betriebssystem mithilfe des Befehls `make install`

Anschließend kann „mausezahn“ endlich gestartet werden: Befehl `mausezahn -h` (es erscheint eine Hilfe bzw. eine Liste mit den möglichen Optionen).

Mit `mausezahn` lassen sich beliebig aufgebaute IP-Pakete unabhängig von Einstellungen bzw. der Konfiguration des Betriebssystems erstellen. Diese Funktion wird bei den Laborübungen zum Thema Netzwerksicherheit benötigt werden.

1.4 Apache-Installation und Grundkonfiguration

Als Beispiel für einen Server-Dienst soll nun der Apache-Webserver installiert und konfiguriert werden. Installation über Yast:

- Yast – Software-Management
- Search: `apache2` – die Checkbox vor „`apache2`“ markieren – Accept – die Abfrage nach „automatic changes“ mit Continue bestätigen, Finish

Prüfen, ob der Dienst nun im System vorhanden ist: `systemctl status apache2.service`

Die Ausgabe dieses Kommandos sollte „Loaded, inactive“ zeigen, d.h. der Dienst ist vorhanden, aber er läuft noch nicht aktiv (Ausgabe mit `q` beenden).

Der Apache-Webserver benötigt für den Start einen Eintrag in der Namensdatei `/etc/hosts`. Hierzu mit einem Text-Editor (z.B. `kate` (Startbutton – Applications – Utilities) oder `vi`) diese Datei öffnen („File – Open“, mit „Pfeil nach oben“ `^` auf die root-Ebene gehen, dann `/etc` einmal anklicken (kein Doppelklick) und die Datei `hosts` öffnen), dort eine Zeile mit der IP-Adresse und dem Rechnernamen hinzufügen:

`10.0.x.y localhost` wobei die richtige IP-Adresse (**bitte nicht x.y eintippen!**) verwendet werden muss:

Das Kommando `ip addr show` zeigt die IP-Adresse (inet) in der Form `10.0.x.y`. Das Kommando `hostname` zeigt den Rechnernamen.

Datei `hosts` speichern (in `kate` „File – Save“).

Vor dem Start des Dienstes soll zunächst eine Index-Datei erstellt werden, die der Webserver bei einer Client-Anfrage ausliefern kann. Hierzu mit einem Text-Editor (z.B. `kate` oder `vi`) folgende Datei anlegen:

`/srv/www/htdocs/index.html`

mit folgendem Inhalt: `<html> <body> <h1> Hallo... </h1> </body> </html>`

wobei die Zeichenkette „Hallo...“ beliebig ist (der Rest besteht aus HTML-Tags, die genau so geschrieben werden müssen).

Nun kann der Webserver gestartet werden: `systemctl start apache2.service`

Mit dem Kommando `wget http://10.0.x.y` (eigene IP-Adresse s.o.) oder einem Browser (mit dieser URL) kann der Webserver getestet werden; mit `wget` wird die Datei `index.html` in das aktuelle Verzeichnis geladen bzw. der Browser sollte nach Eingabe des URLs „localhost“ den Text „Hallo...“ in einer großen Schriftart darstellen.

Wenn der Webserver zufriedenstellend läuft, kann er mithilfe der System-Konfiguration (systemd) dauerhaft aktiviert werden, sodass er nach dem Systemstart nicht immer manuell gestartet werden muss:

- Yast – System – Service-Manager
- `apache2` – Enable

Yast erzeugt hiermit (als eigentliche Aktion) einen Link unter `/etc/systemd/system/apache`, der auf das entsprechende Unit-File zeigt (`/usr/lib/systemd/system/apache2.service`).

Das Kommando `ps -efa | grep http` (oder `ps tree | grep http`) zeigt mehrere `httpd`-Server-Prozesse, die vom Hauptprozess „vorausschauend“ gestartet wurden, um nicht erst in dem Moment (zeitraubend) einen Prozess zu starten, wenn eine Client-Anfrage eintrifft. Wie viele dieser Unterprozesse vorab gestartet werden, kann in der Apache-Konfiguration definiert werden.

Zum Abschluss der Übung kann die virtuelle Maschine heruntergefahren werden (Start-Button unten links, Shutdown) und falls gewünscht ein Snapshot (Sicherungspunkt) im VirtualBox-Manager erzeugt werden (Listensymbol (3 Punkte) – Sicherungspunkte (Snapshots), rechte Maustaste auf „Aktueller Zustand“ –

erzeugen). Falls die Übung von vorne begonnen werden soll, kann der Sicherungspunkt „OpenSuse_leer“ zurückgespielt werden, womit alle Veränderungen rückgängig gemacht werden.

Hinweis: Sollte die hier installierte Maschine über diese Übung hinaus weiterverwendet werden, so ist ein Update empfehlenswert (unter yast: Software – Online Update).

Zudem sollte (nachdem die Maschine heruntergefahren wurde) ein Sicherungspunkt (Snapshot) in VirtualBox angelegt werden, zu dem später wieder zurückgesprungen werden kann, falls unerwünschte Änderungen an der virtuellen Maschine durchgeführt wurden oder Fehler auftreten.

2 Linux-Einführung und Shell-Skripte

2.1 Linux

Terminalfenster (Konsole) öffnen: Symbol im Desktop-Fenster (oben links) oder mittels des Start-Buttons (unten links). Die folgenden Kommandos werden in einem Terminal-Fenster eingetippt.

Verzeichnis anlegen:

Ein Verzeichnis wird mit dem Kommando `mkdir verzeichnisname` angelegt.

In ein Verzeichnis wechselt man mit (change directory) `cd verzeichnisname`, ohne Verzeichnisnamen:

Wechsel in das eigene Home-Verzeichnis, mit `cd ..` Wechsel in das nächst höhere Verzeichnis.

Zusammenfassen mehrerer Schritte ist möglich, z.B.: `cd ../../verz1/verz2/verz3`

Anzeigen des Verzeichnisinhalts: `ls` (listing)

Einige Optionen zu `ls`:

- `ls -a` (all, anzeigen versteckter Dateien (Dateiname beginnt mit einem Punkt))
- `ls -l` (long listing, anzeigen von Dateiattributen)
- `ls -ltr` (long listing, time reverse: umgekehrte Sortierung der Ausgabe nach dem Modifikations-Datum)

Anlegen von Dateien

Legen Sie mit den folgenden Programmen einige Dateien mit beliebigen Namen an, schreiben Sie einige Daten hinein (auch mehrere Zeilen) und speichern Sie die Dateien.

- kurze Textdatei:
`cat > dateiname`, Text eingeben, Zeilen mit Return abschließen,
zum Beenden und Speichern `Strg-D`
- Text-Editor `kate` (grafische Oberfläche):
in einem Terminalfenster `kate &` eingeben (das `&` gibt das Terminal wieder frei), dann „neue Sitzung“
- für Experten: Text-Editor `vi`
`vi dateiname`
Innerhalb des `vi`-Editors wird zwischen einen Kommando- und einem Eingabe-Modus unterschieden
Wechsel in den Kommandos-Modus immer mit `ESC`
Wechsel in den Eingabe-Modus, u.a. mit den Kommandos:
 - `i` (insert)
 - `a` (append)
 - `o` (open, neue Zeile unterhalb der aktuellen Zeile)
 - `O` (neue Zeile oberhalb der aktuellen Zeile)
 Kommandos:
 - `x` (Löschen eines Zeichens hinter der aktuellen Cursorposition)
 - bzw. `X` (vor der aktuellen Position)
 - `dd` (löschen einer ganzen Zeile)
 - `dw` (löschen eines Wortes)
 - eine Zahl vor dem Kommando wiederholt den Befehl entsprechend, z.B. `3dd` löscht drei Zeilen (die Zeile an der aktuellen Cursorposition und die zwei folgenden)
 - `u` (undo) letzten Befehl rückgängig machen
 - `Y` ganze Zeile kopieren und mit `P` Kopierregister einfügen
 - Beenden und Speichern: in den Kommandomodus wechseln, dann „write/quit“ mit `:wq`
 - oder beenden ohne zu speichern (Änderungen gehen verloren) `:q!`
 - interne Hilfe-Funktion mit `F1` in einem eigenen Fensterbereich: dort mit Cursor oder Bildlauf-Tasten bewegen oder suchen mit `:help suchwort`, Verlassen der Hilfe mit `:q`, bei geöffnetem Hilfefenster wechseln in die Datei (und wieder zurück) mit `Strg-w`
 - Zeilennummern anzeigen `:set number` ausschalten mit `:set nonumber`
 schnelles Bewegen des Cursors:

- G (Dateiende)
- gg (Dateianfang), oder z.B. 5g Cursor in Zeile Nr. 5
- \$ (Zeilenende)
- 0 oder ^ (Zeilenanfang)
- falls keine Cursor-Tasten verfügbar sind: h, j, k, l

Arbeiten mit Dateien

Die zuvor angelegten Dateien können nun gelöscht, kopiert oder verschoben werden:

- Löschen `rm dateiname`
- Kopieren `cp quelldatei zieldatei` (auch mit Verzeichnissen kombinierbar z.B. `cp blah /home/tina/xyz`, ist xyz ein Verzeichnis wird die Datei blah in dieses Verzeichnis kopiert, ist xyz ein Dateiname wird blah in das Verzeichnis /home/tina kopiert und dort in xyz umbenannt, Achtung: eine vorhandene Datei mit diesem Namen wird ohne Rückfrage überschrieben!
- Verschieben (move) von Dateien: `mv quelldatei zieldatei`
- Anzeigen des Dateiinhalts (sofern als ASCII-Zeichen darstellbar):
`cat dateiname`
`more dateiname` (seitenweise Ausgabe, weiterblättern mit Leertaste, vorzeitig beenden mit Taste q)
- Suchen (Finden) von Dateien in einer Verzeichnisstruktur:
`find . -name "*bla*"` (Suchen unterhalb der aktuellen Verzeichnisposition nach Dateien, in deren Namen die Zeichenkette „bla“ vorkommt)

Wildcards

Mit Ersatzmustern (Wildcards) können bestimmte Zeichen oder Zeichenketten von der Shell durch andere Zeichen ersetzt werden. Beispielsweise muss bei der Verwendung des Kommandos `grep` als Argument eine Datei angegeben werden, in der nach einer Zeichenkette gesucht werden soll. Möchte man nicht nur in einer bestimmten Datei suchen, sondern in allen Dateien im aktuellen Verzeichnis, so kann `*` als Ersatzmuster für beliebige Dateien verwendet werden. Verwenden Sie die zuvor angelegten Dateien und suchen Sie nach den dort eingefügten Inhalten oder legen Sie weitere Dateien an, z.B. `datei1`, `datei2`, `datei3` usw.:

- Suche nach abc in der Datei `datei.txt`: `grep abc datei.txt`
- Suche nach abc in allen Dateien des aktuellen Verzeichnisses: `grep abc *`
- Suche nach abc in Dateien, deren Name mit „dat“ beginnt: `grep abc dat*`

Mit dem Fragezeichen `?` als Ersatzmuster ist ein einzelnes Zeichen gemeint: `grep abc datei?.txt`
 Hier sucht `grep` z.B. in `datei1.txt` oder `dateiX.txt` aber nicht in `datei12.txt`

Mit eckigen Klammern kann ein Zeichenbereich als Ersatzmuster angegeben werden:

Suche nach abc in den Dateien `datei1.txt` und `datei2.txt`, aber nicht in `datei3.txt` und auch nicht in `datei12.txt`:
`grep abc datei[12].txt`

Suche in Dateien, die zwischen „datei“ und „.txt“ zwar irgendein Zeichen enthalten, aber nicht die 1 oder die 2:
`grep abc datei[!12].txt`

Suche nach abc in den Dateien `datei1.txt`, `datei2.txt`, `datei3.txt` usw., auch in `datei12.txt`, aber auch in `datei12A.txt`:
`grep abc datei[0-9]*.txt`

Wichtige Kommandos

- Liste der laufenden Prozesse ausgeben: `ps`
- alle Prozesse ausgeben: `ps auxw` (hier ausnahmesweise ohne das – für Optionen)
- Prozesse beenden: `kill PID` mit PID als Prozess-Nummer (als Ausgabe von `ps`)
- laufende Anzeige von Prozessen: `top`
- Anzeige der Anzahl von Zeilen, Wörtern und Zeichen: `wc datei`
- Suchen nach Zeichenketten in Dateien: `grep zeichen datei`

Kommandos verketten, Ein-/Ausgabe-Umlenkung

Ausgaben, die ein Kommando erzeugt, können an andere Kommandos direkt weitergegeben oder in Dateien gespeichert werden oder es können Dateiinhalte in Kommandos eingelesen werden.

- Ausgabeweiterleitung (Pipe), z.B. `ps | wc` (ps: Prozessliste, wc: WordCount)
- Ausgabeumleitung, z.B. `ps > datei`
- auch Kombinationen sind möglich: `ps | wc > datei`
- Zwei Kommandos direkt nacheinander ausführen: `ps ; wc datei`
- Das zweite Kommando wird nur dann ausgeführt wenn das erste erfolgreich war (exit-Code 0):
`ls datei && echo ja`
- Das zweite Kommando wird nur dann ausgeführt wenn das erste fehlerhaft war (exit-Code größer 0):
`ls datei || echo nein`

Shell-Skripte

Legen Sie eine Textdatei mit dem Namen `Test` und dem folgenden Inhalt an (dieses Beispiel-Skript führt dann den Befehl `echo` aus):

```
#!/bin/bash
echo Hello World
```

Das Skript wird von der Kommandozeile gestartet, mit: `sh Test`

Alternativ können die Dateiattribute auf „ausführbar“ (`chmod 755 Test`) gesetzt werden, dann kann das Skript direkt mit `./Test` gestartet werden.

In Shell-Skripten werden Variablen mit dem Gleichheitszeichen (=) Werte zugewiesen; das Aufrufen der Variablen erfolgt mittels voranstellen des `$`-Zeichens vor dem Variablennamen. Legen Sie eine Textdatei mit folgendem Inhalt an und führen sie das Skript aus.

```
#!/bin/bash
VARI="Hello World"
echo $VARI
```

2.2 Shell-Skripte

Die folgenden Shell-Skripte (Lösungen in Kapitel 2.3) funktionieren (sofern nicht anders angegeben) sowohl in den virtuellen Maschinen (Labor_R1 usw.) als auch im Windows-Subsystem for Linux. Unter Windows sind jedoch die Besonderheiten bzgl. der Formatierung von Textdateien in der Editor-App zu beachten (siehe „Windows_Subsystem_Linux.pdf“ in EMIL/Moodle).

1. Das Kommando `ls -l | grep dat` listet alle diejenigen Dateien auf, die das Suchmuster „dat“ im Namen enthalten. Ein Shell-Skript namens „argumente“ soll dieses Kommando enthalten, aber das Suchmuster als Kommandozeilen-Argument übergeben bekommen. Legen Sie zunächst mehrere Dateien an, in deren Namen „dat“ vorkommt, z.B. mit `touch datei{1..5}`
2. Legen Sie ein Unterverzeichnis an und darin wieder wie zuvor mehrere Dateien. Ergänzen Sie das Skript aus Aufgabe 1 um ein zweites Kommandozeilen-Argument, mit dem der Name des Unterverzeichnisses übergeben werden kann. Beim Start des Skriptes sollen die gefundenen Dateien im Unterverzeichnis ausgegeben werden. Ein Problem: Werden nicht genau zwei Kommandozeilen-Argumente übergeben, bricht das Skript mit einer Fehlermeldung ab. Ergänzen Sie das Skript daher um eine Überprüfung, ob auch tatsächlich genau zwei Kommandozeilen-Argumente existieren, ansonsten beenden Sie das Skript mit einer „aussagekräftigen“ Fehlermeldung.
3. Das Skript aus Aufgabe 2 überprüft das Vorhandensein der Kommandozeilen-Argumente und beendet sich andernfalls. Ergänzen Sie das Skript um folgende Funktionen:
 - Überprüfung, ob das zweite Kommandozeilen-Argument (das Verzeichnis) vorhanden bzw. nicht leer ist
 - andernfalls soll die entsprechende Variable durch einen Standard- oder Vorgabewert ersetzt werden (z.B. mittels `pwd`)

- außerdem ist zu überprüfen, ob es sich wirklich um ein existierendes Verzeichnis handelt, ansonsten: Beenden des Skriptes mit einer Fehlermeldung.
4. Beim Arbeiten mit Dateien ist es sehr wichtig zu prüfen, ob diese überhaupt existieren und/oder für den Benutzer les-/schreibbar sind, um Fehlermeldungen oder das Abbrechen des Skriptes zu vermeiden. Erstellen Sie ein Skript mit folgenden Funktionen:
 - Einlesen aller Elemente (Dateien und Verzeichnisse) mit `ls` in ein Array.
 - In einer Schleife (über die Anzahl der Array-Elemente) prüfen, ob es sich erstens um eine reguläre Datei handelt und zweitens ob diese Datei für den Benutzer lesbar ist
 - Geben Sie dann diejenigen Dateien aus, auf die das zutrifft (hierzu ein weiteres Array anlegen)
 5. Abwandlung von Aufgabe 4: Prüfen Sie in der Schleife, ob es sich um Verzeichnisse handelt, die für den Benutzer ausführbar sind (also für ihn zugänglich). Geben Sie dann die Namen dieser Verzeichnisse aus, wobei berücksichtigt werden muss, dass evtl. gar kein Verzeichnis existiert oder nur ein Verzeichnis (Ausgabe im Singular „Das Verzeichnis ... ist für den Benutzer ??? zugänglich“) oder mehrere Verzeichnisse (Ausgabe im Plural „Die Verzeichnisse ... sind für ...“)
 6. Vorbereitung: Legen Sie für diese Aufgabe ein eigenes Unterverzeichnis an und innerhalb dieses Unterverzeichnisses einige weitere Unterverzeichnisse.
 Aufgabe: Innerhalb eines Shellskriptes sollen jeweils 5 Dateien innerhalb der existierenden Unterverzeichnisse angelegt werden. Der Zeitstempel (access-time) dieser Dateien soll manipuliert werden: Jede dieser Dateien soll eine um jeweils einen Tag zurückliegende access-time erhalten (datei1 von gestern, datei2 von vorgestern usw.). Hierzu wie zuvor: Prüfen ob ein Verzeichnis existiert und zugänglich ist, falls ja in einer inneren Schleife jeweils eine Datei mit dem gewünschten Zeitstempel anlegen.
 Anlegen einer Datei mit `touch` und der Option `--date=STRING`. Finden Sie mithilfe der Manual-Pages der Kommandos `touch` und `date` heraus, wie dies genau funktioniert. Hinweis: STRING kann z.B. enthalten „3 days ago“
 7. Aufgabe: Erstellen Sie ein Skript, das (wie zuvor) prüft ob und welche Unterverzeichnisse existieren und dann in diesen Verzeichnissen alle Dateien löscht, die älter als 3 Tage (oder 72 Stunden) sind. Hierzu muss in einer inneren Schleife geprüft werden ob Dateien vorhanden sind, ob sie dem Benutzer auch gehören und ob sie älter als 72 Stunden sind. Verwenden Sie hierzu sinnvollerweise das Ergebnis (angelegte Dateien in einigen Unterverzeichnissen) der vorherigen Aufgabe.

Hinweise: Mit der „Unix-Zeit“ (Sekunden seit dem 1. Jan. 1970) lässt sich mit Zeitstempeln sehr einfach rechnen (auf der Kommandozeile zeigt `date +%s` den aktuellen Sekunden-Wert an). Das Kommando `ls -l --time-style=%s ./verzeichnis/datei` gibt die Unix-Zeit im Verzeichnis-Listing aus. Lesen Sie dieses Kommando in eine Variable ein und zerlegen Sie diese Variable mithilfe der `set`-Anweisung in ihre Einzelteile. Dann können Sie prüfen, ob der so extrahierte Sekundenwert der Datei kleiner oder größer als die aktuelle Zeit minus (3*86400) Sekunden ist.

Achtung: Testen Sie das Skript zunächst ausgiebig im „Trockenlauf“ (soll heißen: statt des `rm`-Kommandos zum Löschen einfach nur ein `echo...`), da Sie ansonsten „alles mögliche“ Löschen und nicht oder nicht nur die gewünschten Dateien.

8. Interaktive Eingabe von Werten mit `read`: Erstellen Sie ein Skript, das zwei reelle Zahlen interaktiv abfragt und einliest. Diese Zahlen sollen dann (mit zwei Nachkomma-Stellen) mittels `bc` multipliziert und das Ergebnis ausgegeben werden (nicht im Windows Subsystem for Linux möglich).
9. Binäre Rechen-Operationen: Lesen Sie wie zuvor zwei Dezimalzahlen ein und berechnen Sie: Links-Shift der ersten Zahl um 3 Bits, UND-, ODER- und XOR-Verknüpfung beider Zahlen. Versuchen Sie mithilfe von `printf` eine „schöne, formatierte Ausgabe“ zu erreichen (also beide Zahlen in Binärdarstellung genau untereinander schreiben und darunter das Verknüpfungsergebnis).

2.3 Lösungen zu Kapitel 2.2

1. Lösung:

Schreiben Sie mittels eines Editors die beiden folgenden Zeilen in eine Datei „argumente“.

```
suche=$1
ls -l . | grep $suche
```

Starten Sie dann dieses Shell-Skript mit `sh argumente dat`

Hinweis: der Punkt hinter `ls -l` im Skript ist wichtig, da die Sub-Shell, in der das Skript aufgerufen wird, wissen muss, in welchem Verzeichnis gesucht werden soll (der Punkt steht für das aktuelle Verzeichnis).

Alternativ statt des Punktes: Kommando-Substitution mit `ls -l `pwd`` . Es kann in diesem einfachen Skript auf die zusätzliche Variable verzichtet werden, es geht auch `grep $1`

2. Lösung:

```
suche=$1
verzeichnis=$2
if [ $# -ne 2 ]
then
    echo "Bitte Suchwort und Verzeichnis eingeben"
    exit 1
fi
ls -l $verzeichnis | grep $suche
```

3. Lösung:

```
suche=$1
verzeichnis=$2
if [ -z $2 ]
then
    echo -n "es wurde kein Verzeichnis eingeben, es wird "
    echo `pwd` " verwendet."
    verzeichnis=${2:-`pwd`}
fi

if [ ! -d $verzeichnis ]
then
    echo "Das 2. Argument muss ein existierender Verzeichnisname sein"
    exit 1
fi

ls -l $verzeichnis | grep $suche
```

4. Lösung:

```
dateien=$(ls)
lesbare_dateien=()
j=0
while [ $j -lt ${#dateien[*]} ]
do
    if [ -f ${dateien[$j]} ] && [ -r ${dateien[$j]} ]
    then
        lesbare_dateien[$j]=${dateien[$j]}
    fi
    ((j++))
done
echo "Die folgenden Dateien im Verzeichnis " `pwd` " sind für den Benutzer '$USER' lesbar: "
echo ${lesbare_dateien[*]}
```

5. Lösung:

```

dateien=$(ls)
verzeichnisse=()
j=0
while [ $j -lt ${#dateien[*]} ]
do
    if [ -d ${dateien[$j]} ] && [ -x ${dateien[$j]} ]
    then
        verzeichnisse[$j]=${dateien[$j]}
    fi
    ((j++))
done

anzahl=${#verzeichnisse[*]}
case $anzahl in
    0) echo "Es gibt keine Unterverzeichnisse" ;;
    1) echo -n "Das Unterverzeichnis ${verzeichnisse[*]} ist" ;;
    *) echo -n "Die Unterverzeichnisse: '${verzeichnisse[*]}' sind" ;;
esac

if [ $anzahl -ne 0 ]
then echo " für den Benutzer '$USER' zugänglich."
fi

```

6. Lösung:

```

dateien=$(ls)
j=0
k=5
while [ $j -lt ${#dateien[*]} ]
do
    if [ -d ${dateien[$j]} ] && [ -x ${dateien[$j]} ]
    then
        echo "Anlegen von $k Dateien im Verzeichnis ${dateien[$j]}"
        while [ $k -ne 0 ]
        do
            echo $k
            $(touch --date="$k days ago" ./${dateien[$j]}/name$k ) ;
            ((k--))
        done
    fi
    ((j++))
    k=5
done

```

7. Lösung:

```

dateien=$(ls)
j=0
k=0
while [ $j -lt ${#dateien[*]} ]
do
    if [ -d ${dateien[$j]} ] && [ -x ${dateien[$j]} ]
    then
        datei_liste=$(ls ${dateien[$j]})
        while [ $k -lt ${#datei_liste[*]} ]
        do
            if [ -O ./${dateien[$j]}/${datei_liste[$k]} ]
            then
                set var=`ls -l --time-style=+%s ./${dateien[$j]}/${datei_liste[$k]}`
                unixzeit=$(date +%s)
                if [ $6 -lt $((($date +%s) - (3 * 86400))) ]
                then
                    echo "gelöscht wird die Datei: " $7
                    # $(rm ./${dateien[$j]}/${datei_liste[$k]})
                fi
            fi
            ((k++))
        done
    fi
    ((j++))
    k=0
done

```

8. Lösung:

```

i=0
zahl=()
while [ $i -lt 2 ]
do
    echo -n "Bitte eine Zahl eingeben: "
    read zahl[$i]
    ((i++))
done
y=`echo "scale=2; ${zahl[0]}*${zahl[1]}" | bc`
echo "Das Ergebnis der Multiplikation ist: " $y

```

9. Lösung:

... Einlesen wie oben ...

```

e=`echo "obase=2; ${zahl[0]} " | bc`
echo "Binär-Darstellung der Dezimalzahl $zahl[0]: " $e
echo "Linksshift um 3 Stellen ${zahl[0]} bzw. $e: " `echo "obase=2;${((${zahl[0]}<<3))}" | bc`

echo -e "\nUND-Verknüpfung von ${zahl[0]} und ${zahl[1]}: "
printf "%12s\n" `echo "obase=2; ${zahl[0]}" | bc`
printf "UND %8s\n" `echo "obase=2; ${zahl[1]}" | bc`
echo "      ====="
printf "%12s\n" `echo "obase=2; ${((${zahl[0]}&${zahl[1]}))}" | bc`

echo -e "\nODER-Verknüpfung von ${zahl[0]} und ${zahl[1]}: "
printf "%12s\n" `echo "obase=2; ${zahl[0]}" | bc`
printf "UND %8s\n" `echo "obase=2; ${zahl[1]}" | bc`
echo "      ====="
printf "%12s\n" `echo "obase=2; ${((${zahl[0]}|${zahl[1]}))}" | bc`

echo -e "\nXOR-Verknüpfung von ${zahl[0]} und ${zahl[1]}: "
printf "%12s\n" `echo "obase=2; ${zahl[0]}" | bc`
printf "UND %8s\n" `echo "obase=2; ${zahl[1]}" | bc`
echo "      ====="
printf "%12s\n" `echo "obase=2; ${((${zahl[0]}^${zahl[1]}))}" | bc`

```

3 Reguläre Ausdrücke

3.1 Übungsaufgaben zu *sed*

Die folgenden Aufgaben beziehen sich auf die Datei „Modulliste_MS.txt“ (Die Lösungen finden sich in Kapitel 3.4.1)

1. Lassen Sie alle Zeilen mit *sed* ausgeben, in denen Mathematik vorkommt.
2. Lassen Sie nur die Zeilen mit *sed* ausgeben, in denen Mathematik A oder B oder C vorkommt.
3. Ersetzen Sie in denjenigen Zeilen der Datei Modulliste_MS.txt, in denen Wahlpflichtmodul1 oder Wahlpflichtmodul2 vorkommt (aber nicht Wahlpflichtmodul3), das Wort „Technik“ durch „Computer“.

3.2 Übungsaufgaben zu *awk*

(Die Lösungen finden sich in Kapitel 3.4.2)

1. Im Terminal irgendein Kommando starten (z.B. *kate&*) Achtung: nicht mit „Windows Subsystem for Linux“ möglich! Nun die PID dieses Kommandos mit *ps -f* und *awk* suchen und den Prozess mit *kill* beenden (alles in einer einzigen Kommandozeile; beachten Sie die Klammerung der Kommando-Teile mit den verschiedenen Anführungszeichen ``` `'` `"`).
2. Alle Prozesse ausgeben, die innerhalb eines Terminals laufen, d.h. bei denen in der Ausgabe des *ps*-Kommandos (Spalte TTY) kein Fragezeichen steht
3. In der Datei „Modulliste_MS.txt“ nach allen Prüfungsleistungen (Abkürzung PL) suchen und nur den Namen dieser Veranstaltungen ausgeben
4. Ebenfalls in „Modulliste_MS.txt“: Ausgabe des Namens aller Laborveranstaltungen des 4. Semesters
5. Ausgabe aller Module mit der Gewichtung 4 oder 8:
6. Um wieviele Module handelt es sich bei Aufgabe 5? Hinweis: Summieren Sie die gefundenen Zeilen mittels einer Variablen *i++* und geben Sie zum Schluss in einer *END*-Umgebung den letzten Wert von *i* aus. Hinter dem Anweisungsblock zusätzlich *END{print...}*
7. In Ergänzung zu den Aufgabe 5 und 6: Produzieren Sie eine „lesbare Ausgabe“ z.B. in der Form „24 von 30 Modulen haben eine bewertete Prüfung“. Hinweis: Da Sie sowohl die Anzahl der Prüfungen als auch die Anzahl der Module (das entspricht nicht der Anzahl der Zeilen!) ausgeben sollen, müssen mehrere Suche-Anweisungs-Blöcke verwendet werden.
8. Das Kommando *ip address show* (abgekürzt *ip a s*) gibt diverse Informationen über die Netzwerkschnittstellen aus, u.a. auch die IP-Adresse(n). Filtern Sie diese Adressen mit *awk* heraus.
9. Die Ausgabe in Aufgabe 8 liefert auch die Netzwerkmaske, die mit einem Schrägstrich getrennt hinter der Adresse steht. Geben Sie nur die Adressen ohne Maske aus.
10. Die Ausgabe in Aufgabe 8 liefert auch die IPv6-Adressen (nicht bei den virtuellen Maschinen Labor_R1 usw. im Ursprungszustand). Geben Sie nur die IPv4-Adressen aus.
11. Filtern in der Datei */etc/passwd*: Ausgabe aller UIDs größer/gleich 1000
12. Nochmals Filtern in der Datei */etc/passwd*: Ausgabe aller Benutzernamen, die eine „normale Shell“ haben (hier */bin/bash*)
13. Suchen Sie im System-Log nach Login-Fehlversuchen für User root. Erzeugen Sie einen Login-Fehlversuch mit: *su student*, dann *su -* und Eingabe eines falschen Passworts, mit *exit* zurück zum Benutzer root. Das Kommando *journalctl -xr* zeigt „FAILED SU (to root)...“ und „su[1234]“. Sehen Sie sich zunächst den Aufbau der Ausgabe an (nicht mit dem Windows Subsystem for Linux).

Es lässt sich eine ganz einfache Lösung mit *grep* finden, versuchen Sie aber trotzdem *awk* und reguläre Ausdrücke zu verwenden.

14. Beschränken Sie die Suche auf Einträge des heutigen Tages: Suchen mit `grep` und als Suchmuster die Ausgabe des `date`-Kommandos. Es muss eine geeignet formatierte Ausgabe von `date` verwendet werden. Dazu mehr in den Manual-Pages `man date`

3.3 Reguläre Ausdrücke: „gieriges Verhalten von `*`“

Versuchen Sie, das Such-Verhalten bezüglich des „gierigen Verhaltens“ des Stern-Operators mit folgenden Befehlen zu interpretieren (Warum wird häufig nicht nur „Wahlpflichtmodul[1-9]“ gefunden, sondern teilweise die gesamte Zeile?):

```
grep -P 'Wahl.*[1-2]' Modulliste.txt
grep -P 'Wahl.*[1-4]' Modulliste.txt
grep -P 'Wahl.*5' Modulliste.txt
grep -P 'Wahl.*?[1-4]' Modulliste.txt    (Ausschalten des gierigen Verhaltens mit dem Operator ?)
grep -P 'Wahl.*[1-9]\b' Modulliste.txt
grep -P 'Wahl.*?5' Modulliste.txt
```

3.4 Lösungen

3.4.1 sed

1. Lassen Sie alle Zeilen mit `sed` ausgeben, in denen Mathematik vorkommt.
Lösung: `sed -n '/Mathematik/p' Modulliste_MS.txt`
2. Lassen Sie nur die Zeilen mit `sed` ausgeben, in denen Mathematik A oder B oder C vorkommt.
Lösung: `sed -n '/Mathematik [A-C]/p' Modulliste_MS.txt`
3. Ersetzen Sie in denjenigen Zeilen der Datei `Modulliste_MS.txt`, in denen Wahlpflichtmodul1 oder Wahlpflichtmodul2 vorkommt (aber nicht Wahlpflichtmodul3), das Wort „Technik“ durch „Computer“.
Lösung: `sed '/Wahl.*[1-2]/ {s/Technik/Computer/gp}' Modulliste_MS.txt`
oder `sed -n '/Wahl.*[1-2]/ {p;s/Technik/Computer/gp}' Modulliste_MS.txt`

3.4.2 awk

1. Im Terminal irgendein Kommando starten (z.B. `kate&`) Achtung: nicht mit „Windows Subsystem for Linux“ möglich! Nun die PID dieses Kommandos mit `ps -f` und `awk` suchen und den Prozess mit `kill` beenden (alles in einer einzigen Kommandozeile; beachten Sie die Klammerung der Kommando-Teile mit den verschiedenen Anführungszeichen ``` `'` `"`).
Lösung: `ps -f | kill `awk '$8 ~ "kate" {print $2}``
2. Alle Prozesse ausgeben, die innerhalb eines Terminals laufen, d.h. bei denen in der Ausgabe des `ps`-Kommandos (Spalte TTY) kein Fragezeichen steht
Lösung: `ps -ef | awk '$6 != "?" {print $0}'`
3. In der Datei „Modulliste_MS.txt“ nach allen Prüfungsleistungen (Abkürzung PL) suchen und nur den Namen dieser Veranstaltungen ausgeben
Lösung: `awk -F"|" ' $9 ~ "PL" {print $2}' Modulliste_MS.txt`
4. Ebenfalls in „Modulliste_MS.txt“: Ausgabe des Namens aller Laborveranstaltungen des 4. Semesters
Lösung: `awk -F"|" ' $3 == 4 && $5 ~ "Lab" {print $4}'`
5. Ausgabe aller Module mit der Gewichtung 4 oder 8:
Lösung: `awk -F"|" ' $11 ~ /[48]/ {print $2}'`

6. Um wieviele Module handelt es sich bei Aufgabe 5? Hinweis: Summieren Sie die gefundenen Zeilen mittels einer Variablen `i++` und geben Sie zum Schluss in einer `END`-Umgebung den letzten Wert von `i` aus. Hinter dem Anweisungsblock zusätzlich `END{print...}`
 Lösung: `awk -F"| " '$11 ~ /[48]/ {i++} END{print i}'`
7. In Ergänzung zu den Aufgabe 5 und 6: Produzieren Sie eine „lesbare Ausgabe“ z.B. in der Form „24 von 30 Modulen haben eine bewertete Prüfung“. Hinweis: Da Sie sowohl die Anzahl der Prüfungen als auch die Anzahl der Module (das entspricht nicht der Anzahl der Zeilen!) ausgeben sollen, müssen mehrere Suche-Anweisungs-Blöcke verwendet werden.
 Lösung: `awk -F"| " '$1 ~ ".M" {j++} $11 ~ /[48]/ {i++} END{print i " von" j " Modulen haben eine Prüfung"}'`
8. Das Kommando `ip address show` (abgekürzt `ip a s`) gibt diverse Informationen über die Netzwerkschnittstellen aus, u.a. auch die IP-Adresse(n). Filtern Sie diese Adressen mit `awk` heraus.
`ip a s | awk '$1 ~ "inet" {print $2}'`
9. Die Ausgabe in Aufgabe 8 liefert auch die Netzwerkmaske, die mit einem Schrägstrich getrennt hinter der Adresse steht. Geben Sie nur die Adressen ohne Maske aus.
 Lösung: `ip a s | awk '$1 ~ "inet" {print $2}' | awk -F"/" '{print $1}'`
 alternativ mit der `awk`-Funktion „split“:
`ip a s | awk '$1 ~ "inet" {print $2}' | awk '{split($1,a,"/"); print a[1]}'`
10. Die Ausgabe in Aufgabe 8 liefert auch die IPv6-Adressen (nicht bei den virtuellen Maschinen Labor_R1 usw. im Ursprungszustand). Geben Sie nur die IPv4-Adressen aus.
 Eine mögliche, aber im Sinne von regulären Ausdrücken wenig elegante Lösung:
`ip a s | awk '$1 ~ "inet" {print $2}' | awk -F"." '$1<=255 {print $1"."$2"."$3"."$4}'`
 Etwas bessere Lösung mittels regulärer Ausdrücke (nicht ganz vollständig!):
`ip a s | awk '$1 ~ "inet" {print $2}' | grep -P '(([0-9]\.){3}|([1-9][0-9]\.){3}|([1][0-9][0-9]\.){3}|([2][0-4][0-9]\.){3}|([2][5][0-5]\.){3})'`
11. Filtern in der Datei `/etc/passwd`: Ausgabe aller UIDs größer/gleich 1000
 Lösung: `awk -F: '$3 >= 1000 {print $0}' /etc/passwd`
12. Nochmals Filtern in der Datei `/etc/passwd`: Ausgabe aller Benutzernamen, die eine „normale Shell“ haben (hier `/bin/bash`)
 Lösung: `awk -F: '$7 ~ "/bin/bash" {print $1}' /etc/passwd`
13. Suchen Sie im System-Log nach Login-Fehlversuchen für User `root`. Erzeugen Sie einen Login-Fehlversuch mit: `su student`, dann `su -` und Eingabe eines falschen Passworts, mit `exit` zurück zum Benutzer `root`. Das Kommando `journalctl -xr` zeigt „FAILED SU (to root)...“ und „su[1234]“. Sehen Sie sich zunächst den Aufbau der Ausgabe an (nicht mit dem Windows Subsystem for Linux).
 Es lässt sich eine ganz einfache Lösung mit `grep` finden, versuchen Sie aber trotzdem `awk` und reguläre Ausdrücke zu verwenden.
`journalctl | awk -F: '$3 ~ /su\[[0-9]\]/ && $4 ~ "FAIL.*to root" {print $0}'`
14. Beschränken Sie die Suche auf Einträge des heutigen Tages: Suchen mit `grep` und als Suchmuster die Ausgabe des `date`-Kommandos. Es muss eine geeignet formatierte Ausgabe von `date` verwendet werden. Dazu mehr in den Manual-Pages `man date`
 Lösung wie bei Aufgabe 13, zusätzlich: `| grep "`date '+%h %e'`"`

4 Dateisysteme

4.1 MBR-Partitionierung

4.1.1 Partitionen anlegen

Auf den virtuellen Rechnern sind jeweils drei zusätzliche virtuelle Festplatten mit einer Größe von jeweils 5 GB vorhanden; mit dem Kommando `parted -l` lassen sich alle am Rechnersystem verfügbaren Festplatten auflisten. Diese Platten erhalten eine vom Betriebssystem dynamisch vergebene Geräte-Bezeichnung (Device-File), die sich im Verzeichnis `/dev` befindet. Diese Bezeichnungen haben die Form `/dev/sdX#`. Hierbei steht „sd“ für SCSI-Disk (bei SATA, SAS oder SCSI-Hardware, beim alten IDE-Bus auch „hd“), „X“ ist ein Buchstabe, der dynamisch vom System vergeben wird und „#“ steht für die Nummer einer Partition. Die Device-Bezeichnung `/dev/sdb3` steht also für die Partition 3 auf der zweiten Platte, wobei sich die Reihenfolge in Bezug auf Hardware- oder Anschluss-Kennungen ergibt oder aufgrund der zeitlichen Abfolge des Anschließens). Entfernt man eine Platte oder vertauscht man (bei realer Hardware) die Anschlüsse auf dem Mainboard, so ändern sich auch diese Bezeichnungen. Daher sind diese Device-Bezeichner nur bedingt für feste Konfigurationsdateien oder das manuelle Bearbeiten von Partitions-Einstellungen (wie jetzt in dieser Übung) geeignet.

Daher sollen im Folgenden eindeutige Bezeichner für eine Festplatte verwendet werden. Mit der GPT-Partitionstabelle (s.u.) ist dies insofern einfacher, da es mit der UUID tatsächlich solche unverwechselbaren Kennungen sowohl für das Gerät als auch für die Partitionen als auch für das Filesystem gibt. Bei der MBR-Partitionstabelle existieren die UUIDs erst nach einer Formatierung einer Partition mit einem Filesystem. Da die Festplatten-Hardware i.d.R. jedoch über eine Seriennummer verfügt, wird zumeist daraus eine Kennung im Verzeichnis `/dev/disk/by-id` erzeugt.

1. Mit dem Kommando `parted -l` lassen sich die Partitionstabellen aller angeschlossener Platten auslesen, auch die der Systemplatte oder virtuellen Platten. Die Device-Bezeichnungen sind `/dev/sda` usw. wobei die virtuellen Platten an der Modell-Bezeichnung „ATA VBOX HARDDISK“ zu erkennen sind. Die Ausgabe „Error...“ ist kein Fehler sondern weist lediglich auf die noch nicht existierende Partitionstabelle hin.
2. Finden Sie zunächst die Seriennummer der virtuellen Platte heraus:
`hdparm -I /dev/sdX | grep Serial`
(mit „X“ als Buchstabe des Device-Files der Platte, ohne die Nummer dahinter)
3. Überprüfen Sie nun, welche Device-Bezeichnungen die Platte im Verzeichnis `/dev/disk/by-id` erhalten hat. Format z.B.: `ata-VBOX..._Serien-Nr`
4. Legen Sie nun Partitionen an (**Achtung: keinesfalls die System-Platte bearbeiten: `/dev/sda`**). Hinweis zum Umgang mit solchen sehr langen Bezeichnungen in der Konsole: mit der Tabulator-Taste können die Eingaben automatisch erweitert werden.
`parted /dev/disk/by-id/ata-VBOX...1234_ABC123FFF`
5. Nun die Partitionierung innerhalb der parted-Umgebung beginnen

```
print
```

falls Partitionen vorhanden sein sollten, diese zuerst löschen, z.B. löschen von Partition Nr. 1:

```
rm 1
```

nun eine „Partition Table“ vom Typ „msdos“ (MBR) neu anlegen mit:

```
mklabel msdos
```

dann Partitionen anlegen, Kommandosyntax: `mkpart <PART-TYPE> <START> <END>`
mögliche Werte für PART-TYPE: primary, extended oder logical

START und END: entweder absolute Größe z.B. 3 GB in 10er-Potenzen oder 3 GiB in 2er-Potenzen oder relativ (20%)

Legen Sie zuerst eine **primäre** Partition an (nicht zu groß), z.B.

```
mkpart primary 0GB 2GB
```

Legen Sie dann eine **erweiterte** Partition an (diese soll später weitere logische Partition aufnehmen, daher etwas größer setzen):

```
mkpart extended 2GB 5GB
```

Legen Sie nun *innerhalb* der erweiterten Partition zwei logische Partitionen an (nicht zu groß):

```
mkpart logical 2GB 4GB
```

```
mkpart logical 4GB 5GB
```

prüfen Sie das Ergebnis mit einem erneuten `print` und beenden Sie mit `quit`

(Hinweis zur Größenbezeichnung: 1 GB entspricht 10^9 Byte, 1 GiB entspricht 2^{30} Byte)

Die Festplatte enthält nun einen Master-Boot-Record (MBR), der im Folgenden ausgewertet werden soll.

4.1.2 MBR-Block auslesen

Der MBR hat folgendes Format:

Boot-Loader (440 Byte)	4	2	Partition 1	Partition 2	Partition 3	Partition 4	55AA
------------------------	---	---	-------------	-------------	-------------	-------------	------

Partitionstabelle: 4 Einträge à 16 Byte

Die einzelnen **16-Byte-Partitionstabellen** enthalten folgende Informationen, die Sie für Ihre primäre und die erweiterte Partition auslesen können (Hinweis: die Partition 1 beginnt bei Byte-Position 446 (dez.) bzw. 1BE (hex.)): (Die Werte sind im sog. Little-Endian-Format dargestellt, d.h. zuerst wird das niedrigwertigste Byte geschrieben, zuletzt das höchstwertigste Byte; die jeweiligen Werte müssen daher *byteweise* von rechts nach links gelesen werden).

Das Kommando `dd` eignet sich zum byte-weisen Auslesen von Dateien oder auch block-orientierten Geräten, sodass hiermit nun die ersten 512 Byte der Festplatte in eine Datei kopiert werden können:

```
dd if=/dev/disk/by-id/ata-VBOX... of=datei-mbr bs=512 count=1
```

(„if“ Input-File, „of“ Output-File, „bs“ Blocksize, „count“ Anzahl der zu lesenden Blöcke)

Die Ausgabedatei (hier im Beispiel: `datei-mbr`) kann aufgrund ihres Binärformats natürlich nicht einfach in einem Texteditor gelesen werden, sondern muss mit einem Hex-Editor geöffnet werden:

```
hexdump -C datei-mbr
```

Die Ausgabe zeigt hexadezimale Werte, pro Zeile 16 Byte (am linken Rand eine hexadezimale Zeilen-Nummerierung, am rechten Rand ein *Versuch*, die Werte im ASCII-Format lesbar darzustellen). Fortgesetzte Zeilen, die nur Nullen enthalten, werden nicht dargestellt und nur mit einem * gekennzeichnet (ein Beispiel mit Erläuterungen findet sich in Kapitel 4.9.1).

```
R3:~ # hexdump -C datei_mbr
00000000 fa b8 00 10 8e d0 bc 00 b0 b8 00 00 8e d8 8e c0 |.....|
00000010 fb be 00 7c bf 00 06 b9 00 02 f3 a4 ea 21 06 00 |...|.....!..|
00000020 00 be be 07 38 04 75 0b 83 c6 10 81 fe fe 07 75 |...8.u.....u|
00000030 f3 eb 16 b4 02 b0 01 bb 00 7c b2 80 8a 74 01 8b |.....|...t..|
00000040 4c 02 cd 13 ea 00 7c 00 00 eb fe 00 00 00 00 00 |L....|.....|
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
000001b0 00 00 00 00 00 00 00 00 20 50 90 56 00 00 00 20 |..... P.V...|
000001c0 21 00 83 1b 28 f3 00 08 00 00 00 90 3b 00 00 1b |!... (.....;...|
000001d0 29 f3 0f b4 a8 8c 00 98 3b 00 00 68 64 00 00 00 |).....;..hd...|
000001e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
000001f0 00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa |.....U..|
00000200
```

Suchen Sie die vier Partitionseinträge im MBR heraus (es sollte zwei Einträge mit Nicht-Null-Werten für die eine primäre und die erweiterte Partition geben, die beiden weiteren Partitionseinträge sollten Nullen enthalten, da sich die Partitionseinträge für die logischen Partitionen nicht im MBR sondern in den Startblöcken der jeweiligen Partition befinden). Die Partitionstabelle beginnt an der Byte-Position 446, d.h. in der Zeilennummer 432 (hex.: 1b0) und in dieser Zeile bei Byte 14.

Partition 1 (primäre Partition):

Byte-position (hex)	Größe des Eintrags (Byte)	Inhalt	Hexadezimaler Wert (in den jeweiligen Einträgen werden die einzelnen Byte-Werte von rechts nach links gelesen!!)	Dezimaler Wert
0x00	1	Wert 80 (hex) oder 00		
0x01	3	CHS des ersten Sektors		
0x04	1	Partitionstyp (0x83)		
0x05	3	CHS des letzten Sektors		
0x08	4	Startsektor		
0x0C	4	Anzahl der Sektoren		

Partition 2 (erweiterte Partition):

Byte-position (hex)	Größe des Eintrags (Byte)	Inhalt	Hexadezimaler Wert (in den jeweiligen Einträgen werden die einzelnen Byte-Werte von rechts nach links gelesen!!)	Dezimaler Wert
0x00	1	Wert 80 (hex) oder 00		
0x01	3	CHS des ersten Sektors		
0x04	1	Partitionstyp (0x0F)		
0x05	3	CHS des letzten Sektors		
0x08	4	Startsektor		
0x0C	4	Anzahl der Sektoren		

Vergleichen Sie die von Ihnen berechneten Werte mit der Ausgabe von `parted`. Hierzu `parted` starten, dann das Ausgabeformat auf „Sektor“ umstellen (`unit s`) und Partitionstabelle ausgeben (`print`):

```
parted /dev/disk/by-id/ata-VBOX..._1234_ABC123FFF
unit s
print
```

4.1.3 Partitionstabelle der logischen Partition auslesen

Die Partitionstabelle der ersten **logischen Partition** befindet sich im ersten Block dieser Partition. An welcher Stelle diese Partition beginnt, zeigt der Eintrag „Startsektor“ der erweiterten Partition. Lesen Sie diesen Sektor wieder mit `dd` aus:

```
dd if=/dev/disk/by-id/ata-VBOX... of=datei-erw bs=512 skip=12345 count=1
```

wobei `skip=12345` den **tatsächlichen** Positions-Wert des Startsektors haben muss.

Die Partitionstabelle dieses Blocks enthält zwei Einträge: der erste gibt Startposition und Größe der ersten logischen Partition an, der zweite Eintrag verweist auf den Startblock der nächsten logischen Partition. Tragen Sie die entsprechenden Werte nochmals in folgende Tabelle ein:

Partition 3 (erste logische Partition):

Byte-position (hex)	Größe des Eintrags (Byte)	Inhalt	Hexadezimaler Wert (in den jeweiligen Einträgen werden die einzelnen Byte-Werte von rechts nach links gelesen!!)	Dezimaler Wert
0x00	1	Wert 80 (hex) oder 00		
0x01	3	CHS des ersten Sektors		
0x04	1	Partitionstyp		
0x05	3	CHS des letzten Sektors		
0x08	4	Startsektor		
0x0C	4	Anzahl der Sektoren		

4.2 GPT-Partitionierung

Statt der zuvor verwendeten MBR-Partitionierung soll nun zur GUID Partition Table (GPT) gewechselt werden.

Wieder `parted` starten (mit der richtigen Kennung!)

```
parted /dev/disk/by-id/ata-VBOX...
```

Partitionierung innerhalb der `parted`-Umgebung

```
print
```

falls Partitionen vorhanden sein sollten, diese zuerst löschen, z.B. löschen von Partition Nr. 1:

```
rm 1
```

die „Partition Table“ neu anlegen mit (ggf. mit „Yes“ bestätigen):

```
mklabel gpt
```

Partition anlegen, prinzipielle Kommandosyntax `mkpart <NAME> <START> <END>`

NAME ist eine beliebige Bezeichnung mit max. 36 Zeichen

START und END: entweder absolute Größe (z.B. 10GB oder 10GiB) oder relativ (10%)

Legen Sie mehrere Partitionen an (nicht zu groß), z.B.

```
mkpart beliebiger_Name 0% 30%
```

```
mkpart 'anderer Name mit Leerzeichen' 30% 70%
```

```
mkpart irgendwas 70% 100%
```

prüfen Sie das Ergebnis mit einem erneuten `print` und beenden Sie mit `quit`

Im Verzeichnis `/dev/disk/by-partuuid` sollte nun für jede Partition automatisch eine Gerätedatei angelegt worden sein. Prüfen mit `ls -l /dev/disk/by-partuuid`

Lesen Sie (wie im vorherigen Kapitel mit MBR) nun den GPT-Header und die Partitionseinträge aus; da GPT 34 Sektoren belegt sieht das entsprechende `dd`-Kommando wie folgt aus:

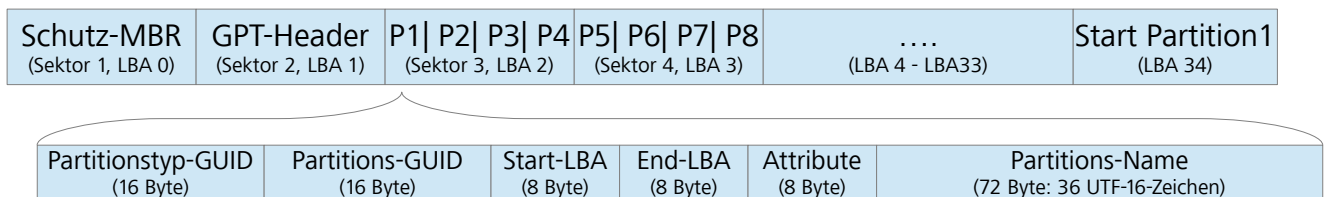
```
dd if=/dev/disk/by-id/ata-VBOX... of=gpt-head bs=512 count=34
```

Mit `hexdump` diese Binärdatei lesbar darstellen:

```
hexdump -C gpt-head
```

(Ein Beispiel mit Erläuterungen findet sich in Kapitel 4.9.2).

Die 34 Sektoren des GPT-Headers bzw. die einzelnen Partitionseinträge sind wie folgt aufgebaut:



Finden Sie die Partitionseinträge für die von Ihnen erstellten Partitionen (sollten einfach an dem gewählten Namen erkennbar sein (ASCII-Darstellung in der rechten Spalte)) und notieren Sie die Partitions-UUID im hexadezimalen *Ziffern*-Format 8-4-4-4-12 (achten Sie darauf, dass die ersten 8 Byte (jeweils in einem Feld) von rechts nach links geschrieben werden, vergleichen Sie die hexdump-Ausgabe mit den Gerätedateinamen in `by-id`)

Partitions-UUID Partition 1 (hex)	
Partitions-UUID Partition 2 (hex)	
Partitions-UUID Partition 3 (hex)	
Partitions-UUID Partition 4 (hex)	

Notieren Sie die Start- und End-Positionen der Partitionen in Sektoren und in GB (beachten Sie, dass wieder alle Byte-Werte von rechts nach links zu lesen sind)

	Start-LBA (dez.)	Start in GB	End-LBA (dez.)	Ende in GB
Partition 1				
Partition 2				
Partition 3				
Partition 4				

4.3 Formatierung mit einem Filesystem

Nach dem erfolgreichen Partitionieren der Festplatte kann nun die Formatierung vorgenommen werden.

1. Filesystem anlegen und formatieren:
Der Befehl `mke2fs` legt Dateisysteme vom Format `ext2`, `ext3` oder `ext4` mit der gewünschten Blockgröße 1024, 2048 oder 4096 Byte an; die Device-Bezeichnung ist die Partitions-UUID, so wie sie im vorherigen Kapitel erzeugt wurde. (nochmals der Hinweis: Formatieren Sie keinesfalls die Systempartitionen unter `/dev/sda` teilweise auch `/dev/sdb`)
`mke2fs -t ext2 -b 1024 /dev/disk/by-id/ata-VBOX...-part1`
2. Mount-Point anlegen:
Um die Partition im Betriebssystem nutzen zu können, muss sie eingehängt werden (mount). Hierzu muss zunächst ein „Mount-Point“ (ein Verzeichnisname) existieren: Name beliebig, sinnvoll ist die Einrichtung dieses Verzeichnisses als Unterverzeichnis von `/mnt`
`mkdir /mnt/platte_ext2`
3. Partition einhängen: Im Folgenden können zur Vereinfachung auch die kürzeren dynamisch vergebenen Device-Bezeichnung `/dev/sd $x$ #` verwendet werden; stellen Sie sicher, dass diese Bezeichnungen wie z.B. `/dev/sdc3` auch tatsächlich den zuvor angelegten Partitionen entsprechen
`mount /dev/sd $x$ # /mnt/platte_ext2`

Das Kommando `df` sollte nun das neue Verzeichnis auf der Festplatte anzeigen.

Im Verzeichnis `/dev/disk/by-uuid` befindet sich nach der erfolgreichen Formatierung ein weiteres Device-File mit der bei der Partitionierung erzeugten UUID des Filesystems. Diese UUID findet sich in der Ausgabe des `mkfs`-Kommandos oder lässt sich nachträglich mit `dumpe2fs -h /dev/sdx#` auslesen. Versuchen Sie, diese UUID auf der Festplatte zu finden: Sie befindet sich i.d.R. in den ersten Sektoren der formatierten Partition. Lesen Sie mit `dd` diese Bereiche aus und analysieren Sie dies mit `hexdump`. Der nötige Wert für „skip“ findet sich in Ihrer obigen Tabelle als Start-Sektor der Partition; es genügt dann „count=3“.

4.4 Verschwendung von Plattenplatz bei kleinen Dateien

1. Wechseln Sie in das zuvor angelegte Verzeichnis
`cd /mnt/platte_ext2a`
 und lassen den Verzeichnisinhalt anzeigen: `ls -l`
 Sollte *kein* Unterverzeichnis „lost+found“ existieren, so ist das Verzeichnis nicht richtig gemountet oder die Partition nicht richtig angelegt worden; in diesem Fall keinesfalls weitermachen sondern zuerst das Problem lösen!
2. Legen Sie in diesem Verzeichnis eine kleine Datei an. Entweder einen Texteditor öffnen, einige wenige Buchstaben eintippen und die Datei in obigem Verzeichnis abspeichern, oder das Kommando `dd` verwenden. Im folgenden Beispiel wird die Datei (Output-File of) „datei1“ erzeugt und mit drei Byte (Blocksize `bs=3` und Anzahl `count=1`) Nullen gefüllt (Input-File `if=/dev/zero`):
`dd if=/dev/zero of=datei bs=3 count=1`
 Inhalt ansehen, z.B. mit (falls die Datei mit `dd` angelegt wurde, enthält sich lediglich drei Nullen):
`hexdump -c datei`
3. Das Kommando
`ls -l`
 zeigt die Größe der Datei an (falls die Datei mit obigem `dd`-Kommando angelegt wurde, so beträgt die angezeigte Dateigröße 3 Byte), das Kommando `df` zeigt die Anzahl der benutzten 1kB-Blöcke, z.B.:
`df /dev/sdc5`
4. Legen Sie eine zweite kleine Datei an und wiederholen Sie danach das `ls`- und `df`-Kommando. Die neue Datei wird mit `ls -l` mit der tatsächlichen Größe angezeigt. Der Vergleich der aktuellen Ausgabe des `df`-Kommandos mit der vorherigen zeigt jedoch, dass für die neue Datei ein ganzer 1kB-Block verbraucht wurde (Spalte „Used“ um 1 erhöht, Spalte „Available“ um 1 verringert).
5. Noch deutlicher wird dies mit dem Kommando `du` (Disk Usage)
 Anzeige des tatsächlich belegten Plattenplatzes durch die einzelnen Dateien
`du -a --block-size=1`
 Anzeige der eigentlichen Größe der Dateiinhalte
`du -a --apparent-size --block-size=1`

Die Problematik von großer Blocksize (bis zu 4096 Byte) und Speicherung von sehr vielen kleinen Dateien wird später im Zusammenhang mit der Thematik „Inodes“ noch weiter vertieft und verdeutlicht.

4.5 Superblock und Inodes

Im Aufgabenteil von Kapitel 4.1 wurden bereits mehrere Partitionen angelegt. Formatieren Sie nun eine dieser Partitionen wieder mit dem Filesystem `ext2`, diesmal jedoch mit der Blockgröße 2048 Byte.

1. Lesen Sie den Superblock aus (Device-Bezeichnung hier wieder nur **beispielhaft** `sdc6`, bitte die tatsächlichen Bezeichnungen verwenden!)
`dumpe2fs -h /dev/sdc6`
 und notieren Sie sich die Werte für `INODES_PER_GROUP`, `INODE_COUNT`, `BLOCK_COUNT`, `BLOCKS_PER_GROUP`, `INODEBLOCKS_PER_GROUP` und `BLOCKSIZE`
2. Mit dem Kommando (ohne Option `-h`)
`dumpe2fs /dev/sdc6`
 werden Informationen zu allen Blockgroups ausgegeben (u.a. freie Inodes und Blöcke). Speichern Sie diese Ausgabe in einer Datei für spätere Vergleiche, z.B.
`dumpe2fs /dev/sdc6 > ~/ausgabe_dump1`

Ausgabe von dumper2fs (Ausschnitt BlockGroup0)

```
Group 0: (Blocks 0-16383)
  Primary superblock at 0, Group descriptors
  at 1-3
  Reserved GDT blocks at 4-515
  Block bitmap at 516 (+516), Inode bitmap at
  517 (+517)
  Inode table at 518-773 (+518)
  15596 free blocks, 2033 free inodes, 2
  directories
  Free blocks: 784-1039, 1042, 1045-16383
  Free inodes: 12, 17-2048
```

Die erste BlockGroup enthält den Superblock (1024 Byte) und die Group Descriptor Table GDT hier in den Blöcken bis 515, danach Block- und Inode-Bitmap (belegte Blöcke bzw. Inodes), danach die Inode-Tabelle (ab 518 mit einer Größe von 256 Blöcken).

3. Legen Sie einige kleine Dateien mit `dd` an (jeweils `bs=10` und `count=1`). Vergleichen Sie die Ausgabe von `dumper2fs` mit der vorherigen; in Group 0 sollten so viele Inodes verbraucht sein wie Dateien angelegt wurden, und entsprechend viele Blöcke.
4. Legen Sie eine große Datei an (`bs=BLOCKSIZE` und `count` etwas größer als `BLOCKS_PER_GROUP`); nun sollte nur ein Inode verbraucht sein, aber fast alle Free Blocks in Group 0. In Group 1 sollte sich die Anzahl der Inodes nicht verändert haben, jedoch einige Blöcke belegt sein.
5. Erstellen Sie ein Shell-Skript (s.u.), das in einer Schleife sehr viele kleine Dateien anlegt (Anzahl der Dateien bzw. Schleifendurchläufe größer als `INODES_PER_GROUP`). Nun sind in einer BlockGroup die Inodes verbraucht, aber noch viele Blöcke frei (ggf. `dumper2fs`-Ausgaben speichern und per `diff` vergleichen)
6. Legen Sie eine weitere Partition (Formatieren mit `ext2`) an, die jedoch sehr klein sein sollte (z.B. 50MB). Ändern Sie das Skript so, dass die maximale Zahl der Inodes verbraucht wird (entspricht maximal `INODES_PER_GROUP * (BLOCK_COUNT / BLOCKS_PER_GROUP)` oder der aktuelle Wert `INODE_COUNT`). Für die Laufzeit des Skriptes muss mit ca. 20.000 Dateien pro Minute gerechnet werden (aus diesem Grund sollte eine wirklich kleine Partition verwendet werden). Eine Fehlermeldung „No Space left on Device“ o.ä. zeigt an, dass die Festplatte voll ist. Das Kommando `df /dev/sdc6` zeigt jedoch einen Nutzungsgrad von nur wenigen Prozent an (viele 1k-Blöcke sind noch frei), ebenso wie (vorher in das Mount-Verzeichnis der Partition wechseln) `du -s` Das „wahre Ausmaß“ des Problems ist mit `du -s --apparent-size` zu erkennen, denn hiermit wird die tatsächlich genutzte Datenmenge angezeigt, die nochmals erheblich kleiner ist (je nach gewähltem `bs`-Wert im `dd`-Kommando). `dumper2fs /dev/sdc6` macht deutlich, warum nun keine Dateien mehr angelegt werden können.

Beispiel eines Shell-Skripts zum Anlegen vieler Dateien

```
#!/bin/bash
echo Anzahl anzulegender Dateien?
read a
while [ $a != 0 ]; do
  dd if=/dev/zero of=/mnt/ext_zwei_2048/dateix$a bs=8 count=1
  let a-=1
done
```

4.6 Synchrones oder asynchrones Schreiben

Unabhängig von der Formatierung eines Dateisystems können erst beim Einhängen (mounten) verschiedene Optionen angegeben werden.

Mit der Option „sync“ kann das Dateisystem synchron beschrieben werden, d.h. eine Schreiboperation gilt erst als abgeschlossen, wenn die Daten physikalisch auf der Festplatte liegen, und nicht wie bei „async“ in einem Zwischenspeicher. Dieser Cache wird vom Betriebssystem verwaltet und kann daher prinzipiell so groß wie der freie RAM des Rechners sein.

Unabhängig davon verfügen auch die Festplatten selbst über einen weiteren Hardware-Puffer in der Größenordnung einiger MB.

Mount-Option async

Die Standard-Mount-Option ist „async“, d.h. der System-Cache wird genutzt. Testen Sie die Schreibraten mittels des Kommandos `dd` mit unterschiedlichen Blocksizes. Folgendes Kommando schreibt 8B * 10.000.000 = 80MB

```
dd if=/dev/zero of=/mnt/platte_ext2a/eine_datei bs=8 count=10000000
```

Wiederholen Sie dieses Kommando mit stufenweise größeren Blocksizes (z.B. `bs=32`, `bs=1k`, `bs=8M`, `bs=32M`) und entsprechend angepasstem „count“ (die gesamte Datenmenge sollte bei größeren Blocksizes bei ca. 1GB liegen, um die Effekte überhaupt feststellen zu können). Die Datenraten können bei größeren Blocksizes scheinbar weit oberhalb der theoretischen Übertragungsrate (60 MB/s) liegen und können (selbst bei gleichen Parametern) eher zufällig variieren.

Mount-Option sync

Hängen Sie die Partition aus (zuvor alle Prozesse, die auf die Partition zugreifen, beenden und ggf. im Terminal-Fenster mit `cd` das Mount-Verzeichnis verlassen)

```
umount /dev/sdc6
```

Nun die Partition mit der sync-Option wieder einhängen (Partitionsbezeichnung und Mount-Point hier wieder nur *beispielhaft* angegeben, verwenden Sie die tatsächlichen Bezeichnungen):

```
mount -o sync /dev/sdc6 /mnt/platte_ext2a
```

Testen Sie auch hiermit wieder die Schreibraten mittels des Kommandos `dd` mit unterschiedlichen Blocksizes

(hiermit werden 10.000 * 8 Byte = 80kB geschrieben):

```
dd if=/dev/zero of=/mnt/platte_ext2a/eine_datei bs=8 count=10000
```

Wiederholen Sie dieses Kommando mit stufenweise größeren Blocksizes (z.B. `bs=32`, `bs=1k`, `bs=8M`, `bs=32M`) und entsprechend angepasstem „count“ (zu beachten: die Schreibraten im sync-Modus liegen erheblich unter denen des async-Modus).

Filesystem-Check

Das Kommando

```
dumpe2fs -h /dev/sdc6
```

zeigt den Inhalt des Superblocks an, u.a. auch den Zustand des Dateisystems (Filesystem state), i.d.R. „not clean“ sofern seit der Formatierung noch kein Filesystem-Check stattgefunden hat. Ein Filesystem-Check prüft die Konsistenz des Systems, vor allem ob Inodes auch wirklich auf die in der Inode-Tabelle angegebenen Blöcke verweisen. Ist dies nicht der Fall (z.B. nach einem Systemabsturz), kann das Kommando `e2fsck` ggf. Reparaturen durchführen. Zunächst die Platte aushängen (`umount`), dann

```
e2fsck /dev/sdc6
```

Ein erneutes `dumpe2fs` sollte nun einen sauberen Zustand anzeigen.

4.7 Journaling-Filesysteme ext3 und ext4

Dateisysteme ohne Journal-Funktion wie ext2 können nach einem System-Absturz (Stromausfall, Fehlersituation usw.) in einen inkonsistenten Zustand geraten, wenn während des Crashes ein Schreibvorgang aktiv war. Dann sind ggf. Metadaten (z.B. Inodes) und Datenblöcke nicht mehr „im Einklang“. Ein Filesystem-Check kann dies u.U. reparieren, dauert aber bei großen, gut gefüllten Dateisystemen mehrere Stunden; in dieser Zeit ist das System nicht verwendbar. Zudem ist der Erfolg der Reparatur nicht gewährleistet, Verluste einzelner Dateien sind wahrscheinlich, selbst ein Totalverlust einer Partition ist nicht auszuschließen.

Journaling-Filesysteme arbeiten daher mit einer Art Transaktions-Log (im Prinzip eine Datei auf der Festplatte), in das die Daten zunächst geschrieben werden. Erst wenn dieser Vorgang abgeschlossen ist, werden aus dem Journal heraus die physikalischen Blöcke und Inodes geschrieben. Nach einem Crash können entweder die Daten aus einem (vollständigen) Journal heraus erneut geschrieben werden oder es wurden (sofern das Journal nicht vollständig ist) die Daten auf der Festplatte noch gar nicht „angefasst“ und sind somit zwar in ihrem alten aber dafür konsistenten Zustand. Ein Filesystem-Check braucht zudem nur die betroffenen Inodes zu prüfen (also diejenigen die nach dem Neustart des Systems im Journal auftauchen), was erheblich schneller geht, als ein Check des gesamten Filesystems.

Nachteil: Die Journal-Option erfordert das doppelte Schreiben sämtlicher Daten. Daher gibt es die Journal-Modi „writeback“ und „ordered“, die nur die Metadaten in das Journal schreiben. Damit kommt es zwar genauso wie bei Non-Journal-Systemen ggf. zu Verlusten einzelner Dateien im Crash-Fall, aber ein Filesystem-Check benötigt nur wenige Minuten.

Vorbereitung

Legen Sie wie in Kapitel 4.1 eine neue Partition an (Größe 5GB) und formatieren Sie diese mit dem Journal-Filesystem ext3 (Partitionsbezeichnung hier wieder nur *beispielhaft*)

```
mkfs.ext3 /dev/sdc7
```

Auch ein neuer Mount-Point sollte angelegt werden, z.B.

```
mkdir /mnt/dateisystem_ext3
```

Journal-Mode „ordered“

Hängen Sie diese Partition mit der Option „ordered“ ein:

```
mount -o data=ordered /dev/sdc7 /mnt/dateisystem_ext3
```

und schreiben Sie wie in Kapitel 4.6 einige Dateien mit dd in das Verzeichnis und notieren Sie sich die Schreibrate.

Journal-Mode „journal“

Hängen Sie die Partition wieder aus und im Journal-Modus wieder ein:

```
mount -o data=journal /dev/sdc7 /mnt/dateisystem_ext3
```

und schreiben Sie wieder einige Dateien. Abgesehen von dem „Zufälligkeitsfaktor“ des async-Modus sollten die Schreibraten erheblich unter denen im Ordered-Modus liegen.

Auch hier muss in der Praxis – genauso wie bei sync und async – zwischen Datensicherheit und Performance entschieden werden.

4.8 Filesystem Btrfs

Einen anderen Weg als die Journaling-Filesysteme ext3 und ext4 geht das Filesystem Btrfs (B-Tree-Filesystem): Um ein stabiles Filesystem herzustellen, wird kein Journal angelegt sondern das Prinzip „Copy-on-Write“ (CoW) angewendet. Hierbei werden Datenblöcke nach einer Änderung nicht an dieselbe Blockposition zurückgeschrieben, sondern es wird ein neuer Block dafür angelegt. Auch bei diesem Prinzip kann also in einer Fehlersituation auf die alten Daten zurückgegriffen werden, da diese bei einem Schreibvorgang (zunächst) gar nicht überschrieben werden. Damit lassen sich prinzipbedingt auch sehr einfach Snapshots (virtuelle Kopien) anlegen, da ältere Dateien auch nach Änderungen weiterhin auf der Platte vorhanden sind. Diese Dateien im Snapshot werden also nicht physikalisch kopiert sondern lediglich in einem neuen Filesystem-Baum referenziert, dessen Metadaten auf die realen Datenblöcke zeigen.

Vorbereitungen:

Wie zuvor muss zunächst die Festplatte partitioniert werden (ob MBR oder GPT spielt dabei keine Rolle), anschließend kann die neue Partition mit Btrfs formatiert werden (`sdX1` nur beispielhaft für `/dev/sdb1` oder `/dev/sdc2` oder `/dev/sdd1` usw. abhängig davon, auf welcher Festplatte die Partition zuvor angelegt wurde)

```
mkfs.btrfs -f /dev/sdX1
```

Dann ein Mount-Verzeichnis anlegen (der Name BTRFS ist beliebig)

```
mkdir /mnt/BTRFS
```

und einhängen

```
mount /dev/sdX1 /mnt/BTRFS
```

Dann einige Unterverzeichnisse und Dateien im gemounteten Verzeichnis anlegen, z.B.:

```
cd /mnt/BTRFS
mkdir verz1 verz2 verz3
touch verz{1,2,3}/datei{1,2}
```

Das Kommando `ls -lR` zeigt alle Unterverzeichnisse und die dort vorhandenen Dateien an.

Read-Only-Snapshot:

Für die hiermit angelegte und befüllte Partition soll nun ein Snapshot angelegt werden. Eine klassische Anwendung für (read-only) Snapshots ist das Anlegen einer feststehenden Kopie eines Verzeichnisses oder Filesystems, von der ein (konsistentes) Backup gemacht werden kann, d.h. die Dateien im Snapshot bleiben während der Laufzeit des Backups unverändert. Eine andere Anwendung sind Test-Verzeichnisse oder -Filesysteme: Sollten sich Änderungen oder Updates im Verzeichnis als fehlerhaft oder unbrauchbar erweisen, so kann auf den Zustand eines früheren Zeitpunkts zurückgesprungen werden.

Anlegen eines read-only Snapshots (der Name `snap1` ist beliebig):

```
btrfs subvolume snapshot -r /mnt/BTRFS /mnt/BTRFS/snap1
```

Es sollte nun ein Unterverzeichnis `snap1` geben, in dem alle zuvor angelegten Verzeichnisse und Dateien vorhanden sind; prüfen mit:

```
cd /mnt/BTRFS/snap1
ls -lR
```

Die Liste aller aktiver Snapshots lässt sich ansehen mit:

```
btrfs subvolume list -pt /mnt/BTRFS
```

In diesem read-only-Snapshot lassen sich natürlich keine Dateien ändern und man kann auch nichts löschen, auch nicht das ganze Unterverzeichnis `snap1`; dies ist nur durch das Löschen des gesamten Subvolumes möglich:

```
btrfs subvolume delete /mnt/BTRFS/snap1
```

Schreibbares Snapshot:

Jetzt soll ein neues, schreibbares Subvolume angelegt werden (ohne Option `-r`):

```
btrfs subvolume snapshot /mnt/BTRFS /mnt/BTRFS/snap2
```

Ändern oder löschen Sie nun sowohl im Verzeichnis `snap2` als auch in den ursprünglichen Verzeichnissen einige Dateien oder erstellen Sie neue Verzeichnisse. Bei solchen Änderungen wird im prinzip-bedingt per CoW ein neuer physikalischer Datenblock erzeugt. Der alte Datenblock wird jedoch nicht für das Überschreiben freigegeben sondern bleibt im Snapshot-Baum bzw. im ursprünglichen Filesystem-Baum erhalten.

Ein Snapshot wird grundsätzlich in Form eines sog. Sub-Volume angelegt. Dieser „Subvolume-Filesystem-Baum“ kann die Daten sowohl als einfaches Unterverzeichnis darstellen als auch als eigenständiges Volume, das separat gemountet werden kann. Legen Sie einen neuen Mount-Point an, z.B.

```
mkdir /mnt/BTRFS2
```

und hängen Sie das Snapshot-Subvolume dort ein (dies ist übrigens auch dann möglich, wenn das ursprüngliche Filesystem gar nicht mehr gemountet ist):

```
mount -o subvol=snap2 /dev/sdX1 /mnt/BTRFS2
```

Mit `df` prüfen, ob alles funktioniert hat. Wechseln Sie in das neue Verzeichnis `/mnt/BTRFS2`, es sollten alle Änderungen, die zuvor im Verzeichnis `snap2` gemacht wurden, auch hier sichtbar sein.

Snapshot einspielen:

Möchte man mit dem jetzigen Snapshot als Standard-Filesystem weiterarbeiten (da z.B. alle Änderungen durchgeführt und erfolgreich getestet sind), muss der Snapshot als Default-Volume gekennzeichnet werden. Das ursprüngliche Filesystem (`/mnt/BTRFS`) muss hierzu unbedingt gemountet sein! Für das entsprechende Kommando wird die Subvolume-ID benötigt, zu finden mit:

```
btrfs subvolume list -pt /mnt/BTRFS
```

dann

```
btrfs subvolume set-default ### /mnt/BTRFS
```

mit `###` als ID-Nummer des Subvolumes.

Nun die beiden mnt-Filesysteme aushängen (zuvor alle Verzeichnisse verlassen):

```
umount /mnt/BTRFS und umount /mnt/BTRFS2
```

Nach einem erneuten mount (ohne die subvol-Option)

```
mount /dev/sdX1 /mnt/BTRFS
```

wird der Snapshot als Standard eingehängt.

Falls man es sich später „doch noch mal anders überlegen“ sollte, kann wieder auf das ursprüngliche Subvolume zugegriffen werden. Dieses Standard-Subvolume hat immer die ID-Nummer 5 (wird jedoch nicht im `list`-Kommando angezeigt):

```
mount -o subvolid=5 /dev/sdX1 /mnt/BTRFS2
```

4.9 Hinweise zur Interpretation der Kommando-Ausgaben

4.9.1 Aufgabenteil 2.1 (MBR-Partition):

Es wurden entsprechend der Aufgabenstellung eine primäre, eine erweiterte und (*innerhalb* der erweiterten) zwei logische Partitionen erstellt. Ausgabe des Kommandos `parted /dev/sdc`

```
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdc: 5369MB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number   Start    End      Size    Type     File system  Flags
  1       1049kB   2000MB   1999MB   primary                type=83
  2       2000MB   5369MB   3369MB   extended              lba, type=0f
  5       2001MB   4000MB   2000MB   logical                type=83
  6       4001MB   5369MB   1367MB   logical                type=83
```

Es soll der Master-Boot-Record (der allererste 512-Byte-Block auf der Festplatte) gelesen werden:

`dd if=/dev/sdc of=datei_mbr bs=512 count=1`

und anschließend die Ausgabedatei `datei_mbr` mit dem Kommando `hexdump -C datei_mbr` dargestellt werden:

```
R3:~ # hexdump -C datei_mbr
00000000 fa b8 00 10 8e d0 bc 00 b0 b8 00 00 8e d8 8e c0 |.....|
00000010 fb be 00 7c bf 00 06 b9 00 02 f3 a4 ea 21 06 00 |...|.....!..|
00000020 00 be be 07 38 04 75 0b 83 c6 10 81 fe fe 07 75 |...8.u.....u|
00000030 f3 eb 16 b4 02 b0 01 bb 00 7c b2 80 8a 74 01 8b |.....|...t..|
00000040 4c 02 cd 13 ea 00 7c 00 00 eb fe 00 00 00 00 00 |L....|.....|
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
000001b0 00 00 00 00 00 00 00 00 20 50 90 56 00 00 00 20 |..... P.V...|
000001c0 21 00 83 1b 28 f3 00 08 00 00 90 3b 00 00 1b |!...(. ....;...|
000001d0 29 f3 0f b4 a8 8c 00 98 3b 00 00 68 64 00 00 00 |).....;..hd...|
000001e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
000001f0 00 00 00 00 00 00 00 00 00 00 00 00 55 aa |.....U..|
00000200
```

Die linke Spalte enthält die Zeilennummerierung in hexadezimalen Zahlen. Die beiden mittleren Blöcke enthalten die hexadezimalen Werte der eingelesenen Bytes (16 Byte pro Zeile) und die rechte Spalte ist ein Versuch, die Werte als ASCII-Zeichen darzustellen (das ergibt in diesem Fall jedoch keine sinnvollen Werte). Für die Auswertung der 16 Byte der ersten Partitionstabelle muss zunächst deren Startposition gefunden werden. Die Partitionstabelle beginnt bei Byte-Position 446 (hexadezimal 1BE), die mithilfe der Zeilennummerierung gefunden werden kann: die Zeile „000001b0“ (dezimal 432) beginnt mit dem Byte 432. Man zähle von dort aus bis zum Byte 446, das das erste Byte der Partitionstabelle von Partition 1 ist. Der markierte Bereich enthält also die 16 Byte der Partitionstabelle 1. Die einzelnen Elemente der Partitionstabelle sind:

Byte-position (hex)	Größe des Eintrags (Byte)	Inhalt	Hexadezimaler Wert (in den jeweiligen Einträgen werden die einzelnen Byte-Werte von rechts nach links gelesen!!)	Dezimaler Wert
0x00	1	Wert 80 (hex) oder 00	00	
0x01	3	CHS des ersten Sektors	00 21 20	
0x04	1	Partitionstyp	83	
0x05	3	CHS des letzten Sektors	f3 28 1b	
0x08	4	Startsektor	00 00 08 00	2048
0x0C	4	Anzahl der Sektoren	00 3b 90 00	3.903.488

Die Werte sind im sog. Little-Endian-Format dargestellt, d.h. zuerst wird das niedrigwertigste Byte geschrieben, zuletzt das höchstwertigste Byte; die jeweiligen Werte müssen daher *byteweise* von rechts nach links gelesen werden.

Das erste Byte zeigt an, dass es sich um eine nicht-bootfähige Partition handelt. Die CHS-Werte (Cylinder-Head-Sector) beziehen sich auf eine ältere, heute nicht mehr verwendete Festplatten-Geometrie. Der Partitionstyp ist „83“ (steht für „Linux-Partition“). Die beiden wichtigsten Werte sind der Startsektor, er beginnt bei

$$2048 * 512 \text{ Byte} = 1.048.576 \text{ Byte} = 1024 \text{ kiB} = 1 \text{ MiB}$$

$$\text{oder } 2048 * 512 \text{ Byte} = 1.048.576 \text{ Byte} = 1048,6 \text{ kB} = 1,05 \text{ MB}$$

Die (aufgerundete) Angabe 1049 kB findet sich auch in der Ausgabe des obigen `parted`-Kommandos.

Entsprechend ist die Größe der Partition:

$$3.903.488 \text{ Sektoren} * 512 \text{ Byte} = 1.951.744 \text{ kiB} = 1906 \text{ MiB} = 1,86 \text{ GiB}$$

$$\text{oder } 3.903.488 \text{ Sektoren} * 512 \text{ Byte} = 1.998.586 \text{ kB} = 1998,6 \text{ MB} = 2 \text{ GB}$$

Die (aufgerundete) Angabe 1999 MB entspricht der Angabe in der Spalte „Size“ in der Ausgabe des obigen `parted`-Kommandos.

4.9.2 Aufgabenteil 2.2 (GPT-Partition)

Der GPT-Header auf Platte `/dev/sdd` (umfasst 34 Blöcke zu je 512 Byte) wird mit `dd` ausgelesen und in die Datei `datei_gpt` geschrieben. Die `hexdump`-Ausgabe ist:

```
00000000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... |
*
000001c0  02 00 ee fe ff ff 01 00 00 00 ff ff 9f 00 00 00 00 | ..... |
000001d0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... |
*
000001f0  00 00 00 00 00 00 00 00 00 00 00 00 55 aa | .....U. |
00000200  45 46 49 20 50 41 52 54 00 00 01 00 5c 00 00 00 | EFI PART...\ |
00000210  4f cb a8 cb 00 00 00 00 01 00 00 00 00 00 00 00 | 0..... |
00000220  ff ff 9f 00 00 00 00 00 22 00 00 00 00 00 00 00 | ..... |
00000230  de ff 9f 00 00 00 00 00 f3 69 ac d6 c6 5e a2 4b | .....i..^..K |
00000240  88 c6 d9 68 85 94 9c f6 02 00 00 00 00 00 00 00 | ...h..... |
00000250  80 00 00 00 80 00 00 00 07 28 98 8a 00 00 00 00 | .....(..... |
00000260  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... |
*
00000400  af 3d c6 0f 83 84 72 47 8e 79 3d 69 d8 47 7d e4 | .=. . . .rG.y=i.G}. |
00000410  ba d0 f8 d4 7e 25 f2 44 92 a5 69 ee 07 7f 75 29 | .....~%.D..i...u |
00000420  00 08 00 00 00 00 00 00 ff ff 2f 00 00 00 00 00 | ...../..... |
00000430  00 00 00 00 00 00 00 00 53 00 54 00 4f 00 52 00 | .....S.T.O.R. |
00000440  5f 00 50 00 61 00 72 00 74 00 69 00 74 00 69 00 | _ .P.a.r.t.i.t.i. |
00000450  6f 00 6e 00 20 00 45 00 69 00 6e 00 73 00 00 00 | o.n. .E.i.n.s... |
00000460  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... |
*
```

Der erste 512-Byte-Block entspricht dem Schutz-MBR (bis einschließlich Zeile `1f0`, dezimal 496), er endet mit der Magic-No. `55 aa` an Byte-Position 512. Danach folgt der GPT-Header (ab Zeile `200`, dezimal 512), anschließend die erste Partitionstabelle (ab Zeile `400`, dezimal 1024). Eine Partitionstabelle umfasst ein Viertel eines Sektors, also 128 Byte (von Zeile `400` bis einschließlich Byte-Position 479). Die erste Zeile (`400`) der Partitionstabelle P1 enthält die 16 Byte große Partitionstyp-GUID, danach (Zeile `410`) die Partitions-GUID, gefolgt von den je 8 Byte großen Werten für Start-LBA und End-LBA (Zeile `420`). Der Name (in UTF-16-Codierung) beginnt in Zeile `430` an Byte-Position 438 und ist in lesbarer Form als ASCII-Zeichen in der rechten Spalte ausgegeben.

Die Werte für Start- und End-LBA (Logical Block Address) jeweils von *rechts nach links* gelesen (hier ohne führende Nullen angegeben):

	Start-LBA (hex./dez.)	Start in MB	End-LBA (hex./dez.)	Ende in MB
Partition 1	08 00 / 2048	$2048 * 512 / 10^9 = 1,05 \text{ MB}$	2f ff ff / 3.145.727	$3.145.727 * 512 / 10^9 = 1611 \text{ MB}$

5 IP-Adressierung

5.1 IPv4 und Netzwerk-Masken

Der theoretische Hintergrund zu dieser Laborübung (IP-Adressen, Subnetzmasken) ist im Vorlesungsskript Kapitel 7.2.2 bzw. 7.2.3 ausführlich erklärt; Kapitel 7.2.4 zeigt zusätzlich ein Beispiel entsprechend zu dieser Laborübung. Die Einrichtung bzw. der Umgang mit virtuellen Maschinen ist im PDF-Dokument „Virtualbox_Installation“ (EMIL/Moodle) erläutert.

Ergänzend wird in einer Video-Aufzeichnung die Konfiguration einer Netzwerkkarte mit dem Admin-Tool „Yast“ unter Linux anschaulich vorgeführt.

[Video in der Mediathek \(Channel „Informatik B“\): IP-Adressen \(Konfiguration unter Linux\)](#)

Lösungen siehe Abschnitt 5.4.

5.1.1 Zwei LANs mit je einem Rechner

Aufgabe/Lernziel: Verständnis von IPv4-Adressen, Netzwerkmasken, Gateway-Adresse

Für diesen ersten Teil der Laborübung wird das in Abbildung Fehler: Verweis nicht gefunden gezeigte Netzwerk aus virtuellen Maschinen verwendet. Als **Router** wird die VM „R1“ eingesetzt, da sie bereits über alle erforderlichen Grundeinstellungen für die Netzwerkkonfiguration verfügt.

Es sollen auf Basis der IP-Netzadresse 192.168.1.0 die **kleinst-möglichen und aufeinander folgenden** Netzadressen verwendet werden. Vergeben Sie geeignete IP-Adressen und Netzwerkmasken für die Interfaces am Router sowie für die Rechner.

Virtuelle Verkabelung der virtuellen Maschinen

Die virtuellen Rechner („R1“ bis „R4“) verfügen jeweils über mehrere virtuelle Netzwerkkarten. Die virtuelle Verkabelung (intnetA, intnetB) zwischen den Rechnern ist bereits vorhanden, es müssen daher nur die Netzwerk-Interfaces konfiguriert werden. Bei der folgenden Konfiguration der Rechner muss genau darauf geachtet werden, welches Interface (Netzwerkkarte) verwendet wird (siehe Abbildungen).

Anlegen der IP-Adressen im Linux-Rechner

Die Einstellungen für die IP-Adressierung werden mit dem Konfigurations-Tool „Yast2“ vorgenommen (Start-Button unten links, dann „System“ - „Yast“):

System - Network Settings - Overview

Auswahl der Netzwerkkarte nach den Bezeichnungen in Abbildung Fehler: Verweis nicht gefunden. Wichtiger Hinweis: Bei der Konfiguration der Interfaces muss der in der Abbildung genannte „Device Name“ (**ethX**, Spalte „Device“) verwendet werden, der nicht mit dem in Yast angezeigten Namen „Ethernet Card Y“ (Spalte „Name“) zu verwechseln ist!

Dann „Edit“ und Eintragen von **IP-Adresse** und **Maske** mit „Next“ abschließen, der Hostname kann leer bleiben. Dann unter Reiter „Routing“ die **Gateway-Adresse** eintragen (das ist die IP-Adresse des Routers in demjenigen Subnetz, an dem der Rechner hängt). Wichtig: Mit „OK“ abschließen, sonst wird die Netzwerk-Konfiguration nicht aktiviert. Der Router R1 benötigt keine Gateway-Adresse.

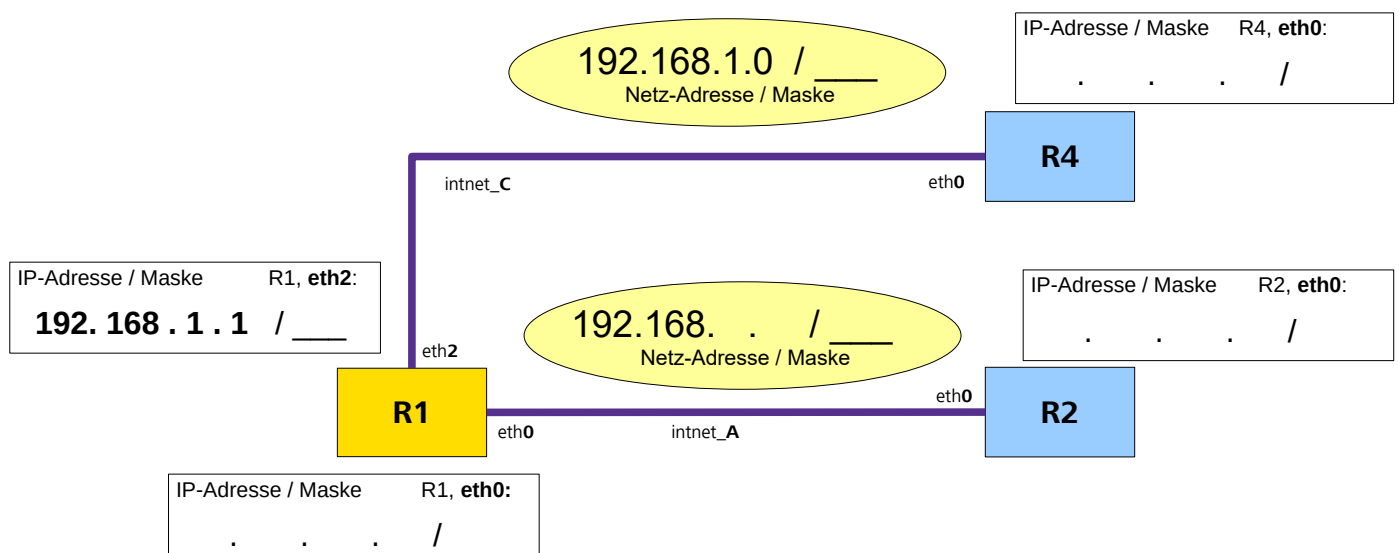


Abbildung 3: Aufgabe 5.1.1

Prüfen Sie mit den Kommandos

`ping <IP-Adresse>` also z.B. `ping 192.168.1.2` oder
`traceroute -n <IP-Adresse>` z.B. `traceroute -n 192.168.1.2`

ob der Rechner in jeweils anderen Netzwerk erreichbar ist (das ping-Kommando mit `Strg-C` beenden).

Eine schnelle Überprüfung der eingestellten Werte für IP-Adressen, Masken oder MAC-Adressen ist mit dem Kommando `ip address show` (oder abgekürzt: `ip a s`) möglich. Es werden u.a. die MAC-Adresse, die IP-Adresse und Maske sowie (s.u.) die IPv6-Adressen angezeigt.

Die Gateway-Einstellung kann mit `ip route show` (abgekürzt: `ip r s`) überprüft werden.

5.1.2 LAN um einen Rechner erweitern

Das zweite LAN (am Router-Interface `eth0`) soll um einen zweiten Rechner (R3) erweitert werden (Abbildung 4). Sofern unter Abschnitt 5.1.1 die kleinst-mögliche Maske verwendet wurde, ist dies **nicht** auf Basis dieser Netzadresse realisierbar! Beachten Sie hierzu auch das Beispiel im Vorlesungsskript, Kapitel 7.2.4.

Aufgabe: Warum kann die vorhandene LAN-Konfiguration nicht einfach mit einer weiteren Adresse ergänzt werden?

Passen Sie die Adressen und die Maske von Rechner R2 und dem Router-Interface `eth0` (R1) an.

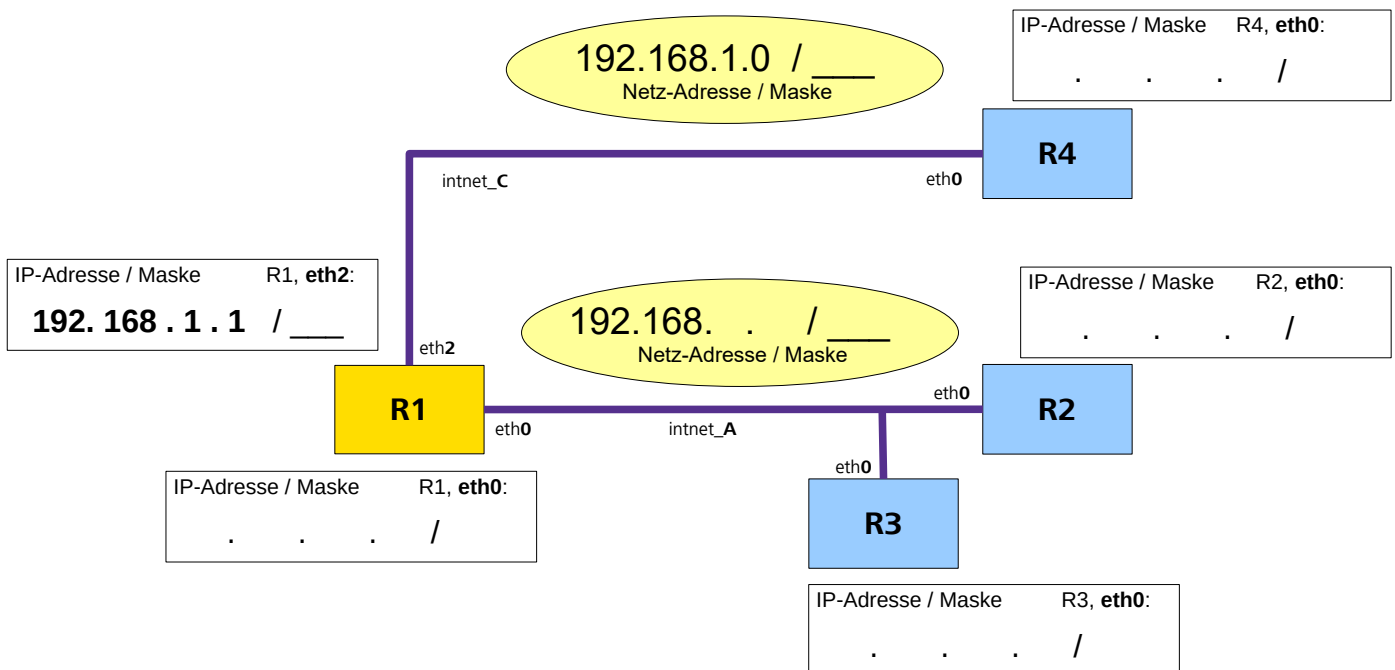


Abbildung 4: Aufgabe 5.1.2

Testen Sie erneut mit `ping` und `traceroute`, ob alle Rechner erreichbar sind.

5.1.3 Ein weiteres Subnetz hinzufügen

Im nächsten Schritt soll ein drittes Subnetz hinzugefügt werden (Abbildung 40). Verwenden Sie für dieses Subnetz die **geeignetste** Netzadresse und die **kleinst-mögliche** Netzmaske. Verwenden Sie Rechner R3 für das neue, dritte Netzwerk (beachten Sie die veränderte Interface-Bezeichnung).

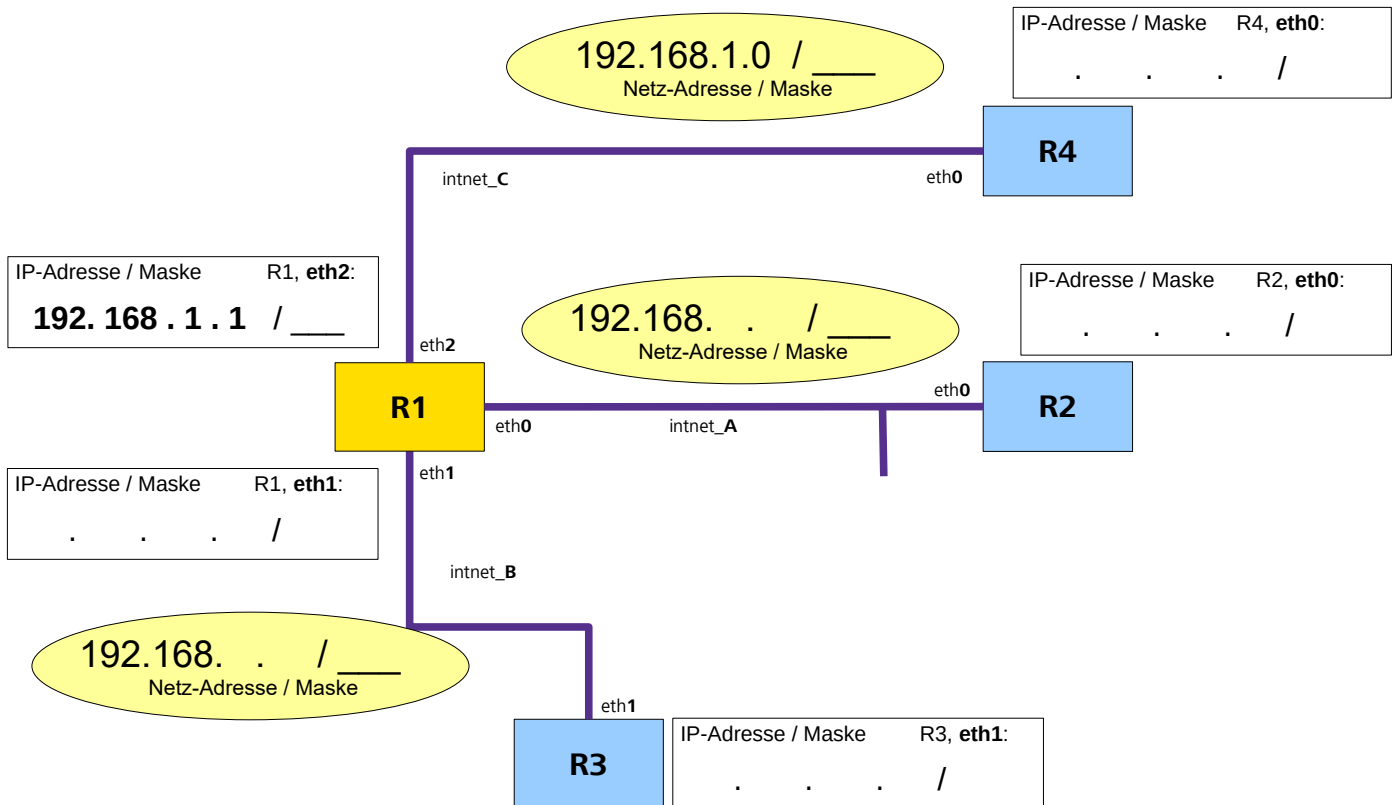


Abbildung 5: Aufgabe 5.1.3

5.2 Address Resolution Protocol ARP

Die Aufgabe des ARP-Protokolls (Vorlesungsskript Kapitel 7.4.1) ist die Zuordnung einer MAC-Adresse (Hardware-Adresse der Netzwerkkarte) zur IP-Adresse. Mit der IP-Adresse (Vermittlungsschicht) wird das endgültige Ziel eines Paketes benannt. Die MAC-Adresse (Sicherungsschicht) ist notwendig, um in einem lokalen Subnetz ein Paket zu einem lokalen Ziel (insbesondere dem Router) zu transportieren. Adress-Informationen, die über das ARP-Protokoll gewonnen werden, sind im ARP-Cache gespeichert. Der Inhalt kann mit dem Kommando `arp -an` oder `ip neighbor show` (abgekürzt `ip n s`) ausgelesen werden. In dieser Tabelle sind Rechnername (nur wenn ein Nameserver erreichbar ist, sonst ein Fragezeichen), IP-Adresse, MAC-Adresse und Interface-Bezeichnung temporär gespeichert. Liegen in dieser Tabelle Informationen über eine lokale Ziel-IP-Adresse bzw. über die Router-IP-Adresse (bei entferntem IP-Ziel) *nicht* vor, so muss vor der Versendung eines Paketes zunächst mittels ARP die MAC-Adresse des lokalen Ziels herausgefunden werden.

Mit dem Programm `wireshark` können Pakete detailliert analysiert werden, die von einem Rechner empfangen oder gesendet werden. Starten Sie `wireshark` (im Desktop-Folder oben links), wählen Sie das richtige (also aktive) Interface und starten Sie das Capturing mit einem Doppelklick auf dem Interface-Namen (alternativ: Capture – Interfaces – Interface markieren – Start).

Sollten im ARP-Cache alle notwendigen Informationen bereits vorliegen, so ist es für einen Rechner nicht notwendig ARP-Requests zu senden. Der Cache muss daher ggf. mit dem Kommando `arp -d <IP-Adresse>` (z.B. `arp -d 192.168.33.17`) oder `ip neigh del <IP-Adresse> dev <Interface>` (z.B. `ip neigh del 192.168.12.7 dev eth0`) geleert werden. Wird danach ein Paket generiert (z.B. mit `ping`) dann muss zuerst die MAC-Adresse des (lokalen) Ziels herausgefunden werden, weshalb ein ARP-Request gesendet wird; dieses wird mit einem ARP-Reply von demjenigen Rechner beantwortet, zu dem die angefragte IP-Adresse gehört.

5.3 IPv6-Adressen

5.3.1 Grundkonfiguration

Das in Kapitel 5.1.3 aufgebaute Netzwerk soll nun mit IPv6-Adressen (Vorlesungsskript Kapitel 7.3.1) konfiguriert werden. Die vorhandene IPv4-Adressierung ist völlig unabhängig von IPv6 und kann daher bestehen bleiben.

Prinzipiell werden IPv6-Adressen automatisch vergeben, nur in Ausnahmefällen (z.B. Server-Rechner) ist eine manuelle Konfiguration mit einer festen IPv6-Adresse erforderlich. Für diese automatische Adressvergabe wird im Gegensatz zu IPv4 aber *kein* spezieller Dienst (DHCP) benötigt, da jeder IPv6-Router diese Aufgabe übernehmen kann. Ein Router muss auf Anfragen von Rechnern (sog. „Router-Solicitations“) mit „Router-Advertisements“ antworten und den IPv6-Netzwerk-**Präfix** liefern können. Daraus bildet sich ein Rechner mithilfe seiner eigenen MAC-Adresse eine individuelle IPv6-Adresse (siehe Skript Kapitel 7.3.2).

Technischer Hinweis: Bei Linux können die Funktionen „IPv6-Router“ und „Router-Advertise-Daemon“ getrennt auf verschiedenen Rechnern bereitgestellt werden. Für diese Laborübung ist der Router „R1“ mit beiden Funktionen bereits vorbereitet und einsatzbereit.

Konfiguration für IPv6:

- Yast – System – Network Settings – Reiter „Global Options“, dort „Enable IPv6“ (ist in den Basis-einstellungen der Rechner bereits vorhanden)
- Auf den Rechnern **R2**, **R3** und **R4** prüfen, ob unter dem Reiter „Routing“ das IPv6-Forwarding **deaktiviert** ist (sonst hält sich ein Rechner für einen IPv6-Router und akzeptiert keine Zuweisung von IPv6-Adressen von außen).
- Auf dem **Router „R1“** muss diese Funktion jedoch aktiviert sein.
- Hinweis: Im vorgegebenen Ablauf dieser Übung sind die Netzwerkschnittstellen bereits zuvor verwendet worden und sind daher aktiviert. *Falls* die Rechner vor diesem Übungsteil neu initialisiert wurden, sind die Schnittstellen jedoch im Zustand „Not configured“ (Network Setting, Reiter Overview). **Nur in diesem Fall:** die Schnittstellen aktivieren (Network Settings, Edit, „Dynamic Address“ anwählen).

Mit dem Kommando `cat /proc/sys/net/ipv6/conf/all/forwarding` kann zusätzlich überprüft werden, ob das IPv6-Forwarding aktiv ist (Wert „0“: deaktiviert, Wert „1“: aktiviert)

Der Rechner sollte nun automatisch eine sog. Link Local Adresse erzeugt haben, beginnend mit `fe80::/10`.

Berechnen Sie die Link Local Adresse eines der Rechner (siehe Skript Kapitel 7.3.1), mit `ip address show` prüfen

5.3.2 Neighbor-Solicitation

Stoppen Sie zunächst das Netzwerk am Rechner mit dem Kommando `rcnetwork stop`
Die Link Local Adresse sollte nun verschwunden sein, prüfen mit: `ip address show`

Nun `wireshark` starten (Interface „any“), als Filter `icmpv6` setzen und (erst dann) das Netzwerk wieder starten: `rcnetwork start`. Es beginnt jetzt der Prozess der automatischen Adresszuweisungen. Da das Router-Interface zum jetzigen Zeitpunkt noch nicht für IPv6 konfiguriert ist, erzeugt der Rechner lediglich seine Link Local Adresse neu. Es werden zwar Neighbor- und Router-Solicitations verschickt, es kommt jedoch keine Antwort.

5.3.3 Router aktivieren

Alle drei Router-Interfaces (auf „R1“) sollen nun für IPv6 konfiguriert werden. Ein Router benötigt dafür feste eindeutige Adressen für jedes Interface, sog. Unique Global Address (in der Praxis beginnend mit `2000::/3`).

Es existiert darüber hinaus ein frei verwendbarer Adressbereich: Unique Local Address `FC00::/7`, vergleichbar mit `192.168.0.0` bei IPv4. Dieser Adressbereich soll hier verwendet werden. Nehmen Sie im Folgenden beliebige, **eindeutige** Adressen aus dem Bereich `FC00` mit der Maske `/64`. Konfigurieren Sie alle drei Interfaces des Routers und beachten Sie dabei, dass es nicht zu Überlappungen bei den Netzwerkadressen kommt.

Auf Router „R1“ Yast starten und (wie oben) die Netzwerkkonfiguration aufrufen:

- Das gewünschte Interface (`eth0`, `eth1` oder `eth2`) markieren und mit „Edit“ in das „Network Card Setup“ wechseln.
- Im unteren Bereich („Additional Addresses“) den Button „Add“ anklicken,
- im Feld „IP Address“ die IPv6-Adresse eintragen (z.B. `FC00:A:B:C::1`) und im Feld „Netmask“ die Maske `/64`. In diesem Beispiel ist `FC00:A:B:C:` die Netzadresse und die `1` hinter dem doppelten Doppelpunkt der Rechneranteil der Adresse. Das Feld „Label“ kann leer bleiben.
- Mit den anderen beiden Interfaces genauso verfahren, wobei unbedingt darauf zu achten ist, dass jeweils verschiedene Netzwerk-Adressen verwendet werden.
- Unter dem Reiter „Routing“: Enable IPv6 Forwarding aktivieren

Da unter Linux die Funktionen „IPv6-Routing“ und „Router-Advertisement“ getrennt sind, muss zusätzlich der „Router-Advertise-Daemon“ (`radvd`) konfiguriert werden. Hierzu die Datei `/etc/radvd.conf` mit einem Texteditor öffnen (`vi` oder `kate`) und für alle drei Interfaces den jeweiligen IPv6-**Präfix** (nicht die vollständige IPv6-Adresse!) eintragen:

```
interface eth0
{
    AdvSendAdvert on;
    AdvDefaultLifetime 1000;
    prefix fc00:a:b:c::/64 { };    Achtung: in der Standard-Datei
                                das #-Zeichen entfernen!
};
interface eth1
{
    ... Einträge für die weiteren Interfaces eth1 und eth2
```

Nach dem Speichern dieser Datei muss der Dienst gestartet werden: `systemctl start radvd.service`

Sollten später Änderungen an dieser Datei erforderlich sein, so muss der Dienst anschließend neu gestartet werden mit: `systemctl restart radvd.service`

Die anderen Rechner erhalten nun aufgrund ihrer regelmäßig an die Multicast-Adresse `ff02::2` gesendeten Router-Solicitations vom `radvd`-Dienst eine Antwort in Form des Router-Advertisements mit dem IPv6-Präfix für das jeweilige Netzwerk und bilden sich daraus selbstständig ihre eigenen IPv6-Adressen. Nach kurzer Zeit (kann bis zu zwei Minuten dauern) zeigt `ip address show` zwei weitere IPv6-Adressen für den Rechner, die mit `FC00` beginnen.

Mit diesen Adressen kann der Rechner nun im „IPv6-Internet“ kommunizieren. Testen Sie die Verbindungen zu anderen IPv6-Adressen im Netzwerk mit

```
ping6 -n fc00:....
traceroute6 -n fc00:.....
```

Die Adressen der jeweiligen Rechner können mit `ip address show` angezeigt werden.

Um im Folgenden den gesamten Prozess der Neighbor- und Router-Solicitation/Advertisement nachvollziehen zu können, stoppen Sie den `radvd`-Dienst auf dem Router „R1“ mit `systemctl stop`

`radvd.service` und starten Sie das Netzwerk der *anderen* Rechner neu mit `rcnetwork restart`, um die vorhandenen IPv6-Adressen zu löschen. Starten Sie das Capturing in `Wireshark` (auf richtiges Interface achten; mit Filter: `icmpv6`) auf diesen Rechnern und starten Sie den `radvd`-Dienst auf R1. Hinweis: ein Neighbor-Advertisement gibt es nur dann, wenn zwei Rechner im Netzwerk angeschlossen sind, zwischen Router und Rechner ist dies wegen des Router-Advertisements nicht erforderlich. Ähnlich wie beim ARP-Cache lassen sich auch hier die Rechner im lokalen Subnetz anzeigen:

```
ip -6 neigh show
```

Um bestimmte Neighbor-Advertisement zu erzwingen, sollten die entsprechenden Einträge zunächst aus der Neighbor-Tabelle gelöscht werden (hier beispielhaft für das Interface `eth0`):

```
ip -6 neigh del fc00:... dev eth0
```

Danach kann per `ping6` ein Neighbor-Advertisement ausgelöst werden (um es z.B. in `Wireshark` aufzeichnen zu können).

5.3.4 Unique Local/Global Address im Linux-Rechner

Die automatische Konfiguration mittels Router-Advertisement und dynamischer Berechnung einer Unique Global bzw. Unique Local Address ermöglicht dem Rechner eine Kommunikation mit anderen Geräten im IPv6-Internet. Da solche Adressen jedoch von der MAC-Adresse der Netzwerkkarte abhängig sind (und sich daher ändern können), sind sie für Anwendungen wie Web- oder Mail-Server, die feste Adressen voraussetzen, nicht geeignet.

Wie bereits für den Router R1 geschehen, kann jedem der Rechner jedoch auch eine fest definierte IPv6-Adresse zugewiesen werden.

Hierzu in `Yast` (wie oben bereits durchgeführt) in die Adresskonfiguration gehen und im Fenster „Network Card Setup, Address“ `Add` anklicken. Im sich öffnenden Fenster eine feste (eindeutige) IPv6-Adresse eintragen, die aus demjenigen Subnetzbereich stammen muss, der durch das entsprechende Router-Interface definiert ist. Die Maske ist hier immer `/64`. Vorgang abschließen.

Der Rechner sollte jetzt über vier IPv6-Adressen verfügen: die Link Local Address (beginnend mit `FE80`), die beiden dynamisch bestimmten Unique Local Address (eine mittels MAC-Adresse, die andere zufällig) und die feste Unique Local Address. Ggf. muss mit einem `ping6` an das Router-Interface ein erneutes Router-Advertisement erzwungen werden.

Testen Sie die festen Adressen im Netzwerk wieder mit `ping6` bzw. `traceroute6`

5.4 Lösungen

Lösung zu Aufgabe 5.1.1:

- **intnetC**: Netzwerkmaske 255.255.255.252 (Dezimal-Schreibweise) oder /30 (Binär-Schreibweise), IP-Adressen 192.168.1.1 (R1, eth2), 192.168.1.2 (R4), Netzwerkadresse 192.168.1.0, automatisch festgelegte Broadcast-Adresse 192.168.1.3
- **intnetA**: Netzwerkmaske 255.255.255.252 oder /30, IP-Adressen 192.168.1.5 (R1, eth0), 192.168.1.6 (R2), Netzwerkadresse 192.168.1.4, automatisch festgelegte Broadcast-Adresse 192.168.1.7

Lösung zu Aufgabe 5.1.2:

- **intnetC** ändert sich nicht.
- **intnetA**: Achtung: Die Netzadresse aus Aufgabe 5.1.1 kann nicht wiederverwendet werden! Mit der Netzwerkmaske /30 aus Aufgabe 5.1.1 stehen nur vier Adressen zur Verfügung, wovon die erste/kleinste für die Netzadresse und die letzte/größte für die Broadcast-Adresse verwendet wird, daher muss die Netzadresse bzw. Maske angepasst werden. Es muss als Maske **/29** (255.255.255.248) gewählt werden (insgesamt acht Adressen). Netzadresse 192.168.1.8, Adressen R1 (eth0), R2 (eth0) und R3 (eth0) beliebig aus dem Bereich 192.168.1.9 bis 192.168.1.14, die Broadcast-Adresse ist automatisch 192.168.1.15

Lösung zu Aufgabe 5.1.3:

- **intnetC** ändert sich nicht.
- **intnetA** ändert sich nicht (lediglich Rechner R3 wird herausgenommen und in das neue Netzwerk eingebunden).
- **intnetB**: idealerweise Netzadresse 192.168.1.4, Netzwerkmaske 255.255.255.252 oder /30, Adressen R1 (eth1) und Adresse R3 (eth1) beliebig aus dem Bereich 192.168.1.5 bis 192.168.1.6, die Broadcast-Adresse ist automatisch 192.168.1.7 (alternativ Netzwerkadresse 192.168.1.16 und entsprechende Rechneradressen)

Lösung zu Aufgabe 5.3.1:

Die IPv6 Link Local Adresse beginnt mit **FE80::** und die rechten 64 Bit bestehen aus der erweiterten MAC-Adresse: linke drei Byte der MAC-Adresse, dann einfügen von **FFFE**, gefolgt von den rechten drei Byte der MAC-Adresse, zudem wird im linken Byte das 2. Bit von rechts invertiert. Beispielsweise wird mit der MAC-Adresse **08:CE:11:BB:22:FD** die IPv6-LL-Adresse **FE80::0ACE:11FF:FE BB:22FD/64**

Lösung zu Aufgabe 5.3.3:

Interfaceadressen von Router R1 (soll mit **FC00** beginnen und die Maske /64 besitzen, Rest beliebig):

- eth2: **FC00:A:B:C::1 /64**
- eth0: **FC00:A:B:D::1 /64**
- eth1: **FC00:A:B:E::1 /64**

Bei den Rechnern werden zwei Unique-Local-Adressen mithilfe des Präfix erzeugt: eine wird aus der MAC-Adresse gebildet und die andere zufällig (Privacy Extension).

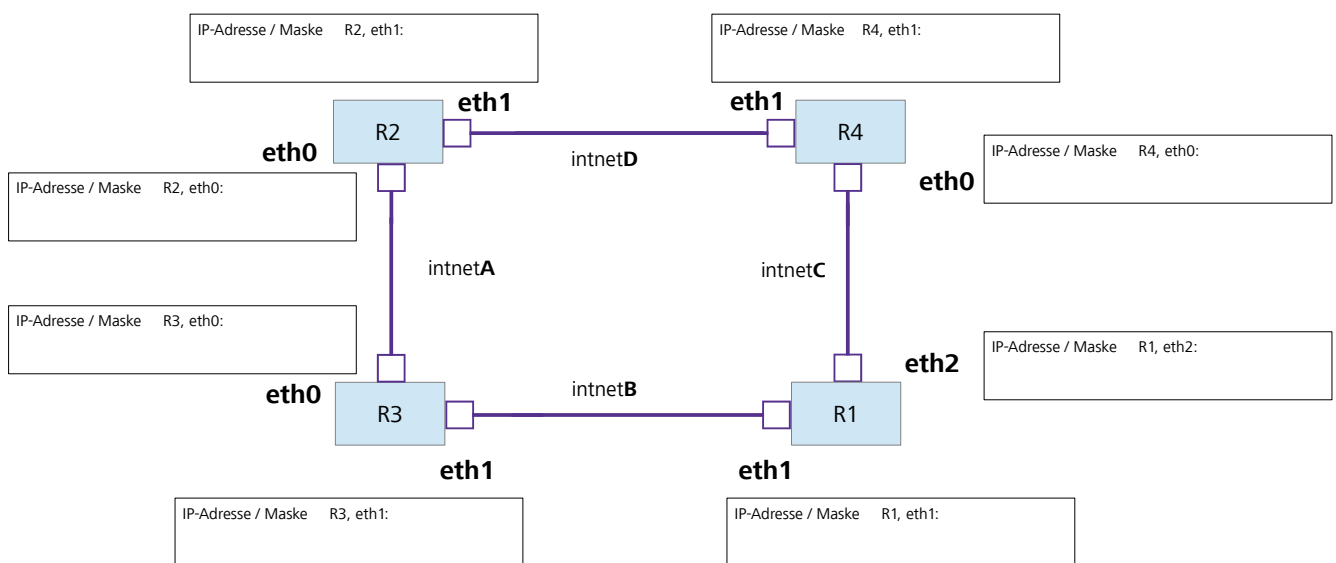
6 Routing-Verfahren

6.1 Grundaufbau

Im ersten Schritt wird ein Netzwerk aus vier virtuellen Linux-Routern (VMs „R1“ bis „R4“) aufgebaut. Die Verkabelung und die Konfiguration der Interfaces erfolgt nach unten stehender Abbildung: die Rechner R1, R2, R3 und R4 sind in der Standard-Einstellung mit den virtuellen Kabeln „intnetA“ bis „intnetD“ bereits „fest verkabelt“.

Für jedes Netzwerk kann eine beliebige IP-Adresse gewählt werden, wobei darauf zu achten ist, dass **jede** Verbindung zwischen zwei Rechnern eine **eigene**, eindeutige **Netzadresse** erhalten muss, und daher die Rechner-Interfaces mit einer IP-Adresse aus diesem Adressbereich konfiguriert werden müssen. Soll das Netzwerk zwischen R2 und R4 (intnetD) beispielsweise die Netzadresse 192.168.7.0/24 bekommen, so müssen R2 (am Interface eth1) und R4 (am Interface eth1) jeweils Adressen aus diesem Bereich erhalten (also z.B. R2: 192.168.7.1 und R4: 192.168.7.2). Bei allen Rechnern darf *kein* Default-Gateway eingetragen werden, da diese Rechner als Router eingesetzt werden sollen. Es ist jedoch darauf zu achten, dass bei den Rechnern das „IP-Forwarding“ aktiviert ist. Wichtiger Hinweis: Bei der Konfiguration der Interfaces muss der in der Abbildung genannte „Device Name“ (**ethX**) verwendet werden, der nicht mit dem in Yast verwendeten Namen „Ethernet Card Y“ zu verwechseln ist!

Video in der Mediathek (Channel „Informatik 2“): [Routing \(RIP\)](#)



Nachdem alle Interfaces konfiguriert wurden, können die einzelnen Verbindungen getestet werden; hierzu jeweils von einem Rechner/Router ein ICMP-Echo-Request (**ping**-Kommando) an den **Nachbarn** schicken. Beispielsweise mit den zuvor genannten IP-Adressen von Rechner R2 den Nachbarn R4 „anpingen“:

```
ping 192.168.7.2
```

und umgekehrt. Es müssen **alle** Einzel-Verbindungen aktiv sein (jeweils mit **ping** testen!), da andernfalls das im Folgenden einzurichtende Routing nicht funktionieren kann.

6.2 RIP-Routing

Der Grundaufbau ermöglicht noch keine Verbindungen über einen Router hinweg, d.h. R3 kann beispielsweise R4 nicht erreichen, da keine direkte Verbindung besteht und das Routing noch nicht aktiviert ist. Im nächsten Schritt wird daher das dynamische Routing mithilfe des Routing Information Protocol (RIP) konfiguriert.

Auf den Linux-Routern ist die Open-Source-Software „quagga“ bereits installiert, mit deren Hilfe ein Linux-Rechner als Router eingerichtet werden kann (sofern er über mehrere Netzwerk-Interfaces verfügt). Die Software besteht aus zwei Elementen: Die Grundkonfiguration der Netzwerk-Interfaces und die Konfiguration des verwendeten Routing-Protokolls.

Die Grundkonfiguration der einzelnen Router erfolgt jeweils in der Datei `/etc/quagga/zebra.conf`. Es ist darauf zu achten, dass die hier eingetragenen IP-Adressen und Masken denjenigen der tatsächlichen IP-Konfiguration des Rechners genau entsprechen.

```
hostname R2_quagga

password quagga

interface eth0
    ip address 192.168.1.1/24

interface eth1
    ip address 192.168.2.1/24
```

Die Konfiguration des Routing-Protokolls RIP erfolgt in der Datei `/etc/quagga/ripd.conf` wobei darauf zu achten ist, dass hier **Netzwerk**-Adressen angegeben werden müssen (im Beispiel 192.168.1.0/24) und *nicht* die Interface-Adressen! Es dürfen *nur* die IP-Adressen derjenigen Netzwerke eingetragen werden, die direkt an diesem Router angeschlossen sind (d.h. diejenigen Netzwerke, für die dieser Router „zuständig ist“).

```
hostname R2_ripd

password quagga

router rip
    timers basic 30 180 90
    network 192.168.1.0/24
    network 192.168.2.0/24
```

Anschließend werden auf den Linux-Routern die beiden erforderlichen Dienste gestartet:

```
rczebra start
rcripd start
```

Falls später Änderungen an diesen Dateien erforderlich sein sollten, müssen die Dienste immer neu gestartet werden, um die Änderungen zu übernehmen.

```
rczebra restart
rcripd restart
```

Sind alle Einstellungen korrekt vorgenommen worden, so können sich alle beteiligten Rechner/Router gegenseitig erreichen, auch über mehrere Router hinweg.

Testen der Verbindungen mit `traceroute -n 192.168.x.y` (es sollten die Wege zum jeweiligen Zielrechner angezeigt werden).

6.3 Distance-Vector-Routing

6.3.1 Routing Information Protocol RIP

In diesem Übungsteil soll die Funktionsweise eines Distance-Vector-Verfahrens am Beispiel des zuvor aktivierten RIP-Routings nachvollzogen werden (siehe Vorlesungsskript Kapitel 7.5.1 und 7.5.2).

Routing-Tabellen

Notieren Sie sich in den vier vorbereiteten Tabellen für die Analyse des RIP-Protokolls zunächst alle Routen, die den Routern derzeit bekannt sind, einschließlich Zielnetzadresse, Nachbar-Adresse und Metrik. Loggen Sie sich hierfür auf den Linux-Routern in die Quagga-Umgebung ein mit

```
telnet 192.168.x.y 2601 (Passwort „quagga“)
```

(oder alternativ lokal auf den einzelnen Rechnern mit: `telnet localhost 2601`)

Die Routing-Tabelle des Rechners lässt sich anzeigen jeweils mit dem Kommando

```
show ip route (verlassen der telnet-Umgebung mit exit)
```

Jeweils zwei Zielnetzwerke sind „directly connected“ (Zeile beginnt mit einem „C“, in diesem Fall gibt es keinen „Nachbarn“), die beiden anderen sind über einen der Nachbar-Router erreichbar, mit dem per RIP-Protokoll kommuniziert wird (Zeile beginnt mit einem „R“). Diese Routing-Tabelle wird aufgrund derjenigen Informationen gebildet, die über das RIP-Protokoll von den Nachbar-Routern empfangen werden. Sie enthält gelernten Routen mit der dazugehörigen Gesamt-Metrik (die Zahl hinter 120), die in diesem einfachen Netzwerk immer „2“ ist.

Routing-Tabellen

Linux-Router R1		
Zielnetzwerk	via Nachbar	Metrik

Linux-Router R2		
Zielnetzwerk	via Nachbar	Metrik

Linux-Router R3		
Zielnetzwerk	via Nachbar	Metrik

Linux-Router R4		
Zielnetzwerk	via Nachbar	Metrik

Paket-Aufzeichnung Linux--Router

Die Informationen, aus denen die Routing-Tabellen gebildet werden, stammen aus denjenigen RIP-Paketen, die von den Nachbar-Router empfangen wurden. Diese Pakete sollen auf den Linux-Routern mit `wireshark` aufgezeichnet und analysiert werden. Hierzu `wireshark` starten und das „Capturing“ für alle Interfaces (any) aktivieren (Zeile „any“ doppelt anklicken). Eine Aufzeichnungsdauer von ca. einer Minute sollte genügen, um RIP-Pakete (Protocol: RIPv2) von beiden Nachbarn zu empfangen. Die Source-Address ist entweder eine der beiden eigenen Adressen oder eine derjenigen der beiden Nachbar-Rechner, die Destination-Address ist die immer RIP-Multicast-Adresse 224.0.0.9 unter der sich alle RIP-Router „angesprochen fühlen“.

Analyse

Klicken Sie im oberen Feld des Wireshark-Fensters eine Zeile (einfach) an, mit einer Source-Adresse eines der Nachbarn; im mittleren Feld werden die einzelnen Protokoll-Header angezeigt, die in diesem Paket verwendet werden. Klicken Sie auf den kleinen Pfeil links neben „Routing Information Protocol“, um die Inhalte des RIP-Headers zu öffnen. Notieren Sie die in den RIP-Paketen übertragenen Zielnetzadressen und Metriken jeweils getrennt nach den Nachbarn, *von denen* diese Pakete gesendet wurden.

Wireshark-Beispiel RIP-Protokoll:

No.	Time	Source	Destination	Protocol	Length	Info
18	51.821879657	172.16.3.2	224.0.0.9	RIPv2	88	Response

Frame 18: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface any, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 172.16.3.2, Dst: 224.0.0.9
User Datagram Protocol, Src Port: 520, Dst Port: 520
Routing Information Protocol
Command: Response (2)
Version: RIPv2 (2)
IP Address: 172.16.1.0, Metric: 2
Address Family: IP (2)
Route Tag: 0
IP Address: 172.16.1.0
Netmask: 255.255.255.0
Next Hop: 0.0.0.0
Metric: 2
IP Address: 172.16.2.0, Metric: 1
Address Family: IP (2)
Route Tag: 0
IP Address: 172.16.2.0
Netmask: 255.255.255.0
Next Hop: 0.0.0.0
Metric: 1

In diesem Beispiel wird ein IP-Paket (in diesem Fall) vom Nachbar-Rechner 172.16.3.2 an die Multicast-Zieladresse 224.0.0.9 gesendet (unter dieser Adresse werden alle RIP-Router angesprochen). Innerhalb dieses IP-Paketes befindet sich ein UDP-Paket (mit dem Ziel-Port 520 für den RIP-Dienst). In diesem UDP-Paket befindet sich schließlich das RIP-Paket mit der Information über zwei Netzwerk-Adressen (einschließlich Maske und Metrik), die dem Nachbarn bekannt sind. Die Next-Hop-Adresse ist Null, daher werden alle Pakete, die später an diese Netzwerkadresse geschickt werden sollen, an denjenigen Nachbarn weitergegeben, von dem die Information kam.

Eigentlich sollte jeder Nachbar jeweils *vier* Zielnetze im RIP-Paket übermitteln (alle vorhandenen Zielnetze). Es werden offenbar aber nicht alle Netzwerkadressen über alle Nachbarn bekannt gegeben: Verantwortlich hierfür ist die Funktion „Split-Horizon“ (Vorlesungsskript Kapitel 7.5.1). Welche Netzadressen fehlen jeweils? Starten Sie Wireshark auf zwei der Router und tragen Sie jeweils diejenigen Routing-Informationen in die Tabellen ein, die von den beiden jeweiligen Nachbarn übermittelt wurden.

einer der Router:

RIP-Information von Nachbar 1 _____	
Zielnetzwerk	Metrik

RIP-Information von Nachbar 2 _____	
Zielnetzwerk	Metrik

ein zweiter Router:

RIP-Information von Nachbar 1 _____	
Zielnetzwerk	Metrik

RIP-Information von Nachbar 2 _____	
Zielnetzwerk	Metrik

6.3.2 Split Horizon

In der Standard-Konfiguration wird der RIP-Dienst mit der Funktion „Split Horizon“ aktiviert (Vermeidung des Count-to-Infinity-Problems dadurch, dass gelernte Routen nicht an denjenigen Nachbarn zurückgesendet werden, über den diese Information ursprünglich kam, siehe Vorlesungsskript Kapitel 7.5.1). Diese Funktion soll nun abgeschaltet werden. Vergleichen Sie dann die Paketinhalte bei einer erneuten Paket-Analyse.

Achtung: die Logik dieser Funktion erfordert es, dass erst auf **allen benachbarten Routern** Split-Horizon deaktiviert sein muss, bevor die folgende Paketanalyse gestartet werden kann!! Wird diese Funktion lediglich auf dem eigenen Router deaktiviert, ändert sich gar nichts.

Deaktivieren Sie die Split-Horizon-Funktion auf den beiden Nachbar-Routern und analysieren Sie anschließend (genauso wie zuvor) die RIP-Pakete, die von diesen beiden *Nachbar*-Routern *empfangen* werden.

Hierzu die Administrator-Umgebung von „ripd“ starten:

```
telnet 192.168.x.y 2602)      (nicht wie oben Port 2601 !)  Passwort „quagga“

> enable
# conf term
(config)# interface ethX      (statt ethX die richtige Bezeichnung verwenden, z.B. eth0)
(config-if)# no ip rip split-horizon
(config-if)# exit
(config)# interface ethY      (statt ethY die richtige Bezeichnung verwenden, z.B. eth1)
(config-if)# no ip rip split-horizon
(config-if)# end
```

Tip: Befehle können mit der Cursor-Taste „Up“ erneut aufgerufen werden.

6.4 Statisches Routing

Das dynamische Routing soll nun abgeschaltet und stattdessen ein statisches Routing aktiviert werden. In realen Netzwerken müssen hierzu die kürzesten Wege zunächst mit einem Routing-Algorithmus berechnet werden, in diesem einfachen Fall des kleinen virtuellen Netzwerkes können die Informationen aus den zuvor notierten Routing-Tabellen verwendet werden.

Abschalten des dynamischen Routings

```
rcripd stop
rczebra stop
```

Aktivieren des statischen Routings

Die Routing-Tabellen befinden sich in der Datei `routes` im Verzeichnis `/etc/sysconfig/network`

Öffnen Sie diese Datei mit einem Text-Editor (z.B. `vi` oder `kate`).

Die zu erstellende Routingtabelle besteht aus mehreren Zeilen, jeweils eine pro Ziernetzwerk. Jede Zeile beginnt mit der IP-Adresse des Zielnetzwerks, gefolgt von der unmittelbaren Nachbar-IP-Adresse desjenigen Routers, über den der Weg zu diesem Ziernetzwerk führt. Dann folgt die Netzwerkmaske und der Interface-Name am eigenen Router, über das der Nachbar angeschlossen ist. Bei den Linux-Routern müssen (im Gegensatz zu den hier nicht verwendeten Cisco-Routern) auch die direkt angeschlossenen Netzwerke eingetragen werden (mit „Nachbar-Adresse“ 0.0.0.0). Verwenden Sie keine Tabulatoren sondern Leerzeichen.

Datei `routes`:

```
192.168.1.0    192.168.4.1    255.255.255.0 eth1
192.168.7.0    192.168.10.1   255.255.255.0 eth2
192.168.4.0    0.0.0.0        255.255.255.0 eth1
```

Anschließend das Netzwerk neu starten (auch immer nach Änderungen an dieser Datei):

```
rcnetwork restart
```

Das Netzwerk sollte nun wieder vollständig funktionsfähig sein, testen mit `traceroute`

6.5 IPv6-Routing

Bei Interesse: **Zusatz-Aufgabe.**

Abschließend kann eine Konfiguration mit IPv6 vorgenommen werden, einschließlich statischem Routing. In Yast das IPv6-Forwarding unter Linux aktivieren und feste, eindeutige IPv6-Adressen für jedes Interface vergeben (siehe Laborübung 5).

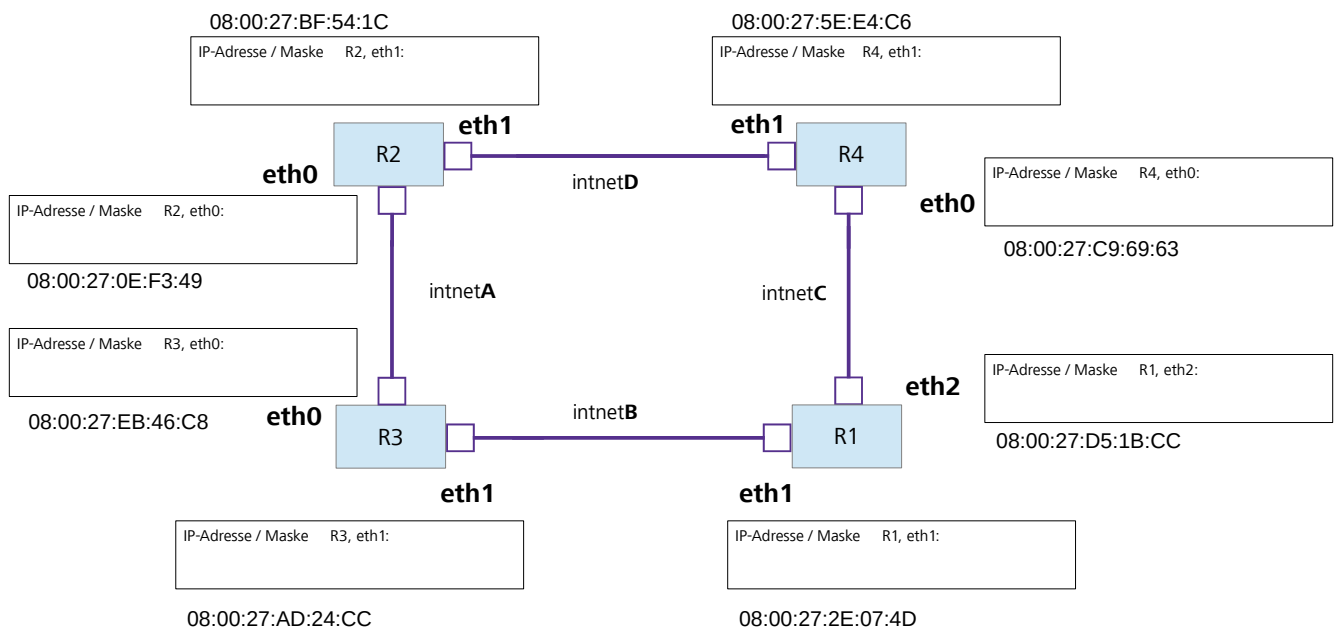
Die statischen IPv6-Routen über Yast eingeben: Network Settings – Reiter Routing – „Routing Table“ Add: Destination Network z.B. `fc00:2::` Mask /64 Gateway (*Nachbar-IPv6*) z.B. `fc00:1::2` und Device (der *eigene* Interface-Name in Richtung des Nachbarn z.B. `eth1`)

Das Kommando `ip -6 route show` sollte nun alle gewünschten Routen anzeigen und `traceroute6` die richtigen Wege ausgeben.

7 OSPF-Routing

7.1 Grundaufbau

In dieser Übung wird das Link-State-Routingverfahren Open Shortest Path First (OSPF) eingesetzt. Hierzu wird der gleiche Grundaufbau des Netzwerkes wie in Aufgabe 6.1 verwendet. Für die Verbindungsnetze zwischen den Routern müssen jeweils *verschiedene* Netzwerkadressen gewählt werden. Für den weiteren Ablauf dieser Übung ist es sinnvoll, wenn die Adressen so gewählt werden, dass sich später Adressen zu Blöcken zusammenfassen lassen, z.B. lassen sich aus den Adressen 172.16.1.0/24 bis 172.16.4.0/24 zwei Blöcke bilden, mit 172.16.0.0/22 (also von .0.0 bis .3.0) und 172.16.4.0/24.



Die (frei zu wählenden) IP-Adressen müssen – wie zuvor – mit einem Texteditor (z.B. `kate`) in die Konfigurationsdatei `/etc/quagga/zebra.conf` eingetragen und genauso im Betriebssystem (Yast) konfiguriert werden. Dort muss auch das IP-Forwarding auf allen Rechnern aktiviert sein.

Der Dienst `zebra` kann anschließend auf allen Rechnern gestartet werden:

```
systemctl start zebra.service
```

Dieser Dienst kann während der gesamten Übung aktiv laufen; falls später jedoch Änderungen bzw. Korrekturen an der IP-Konfiguration vorgenommen werden sollten, muss der Dienst neu gestartet werden:

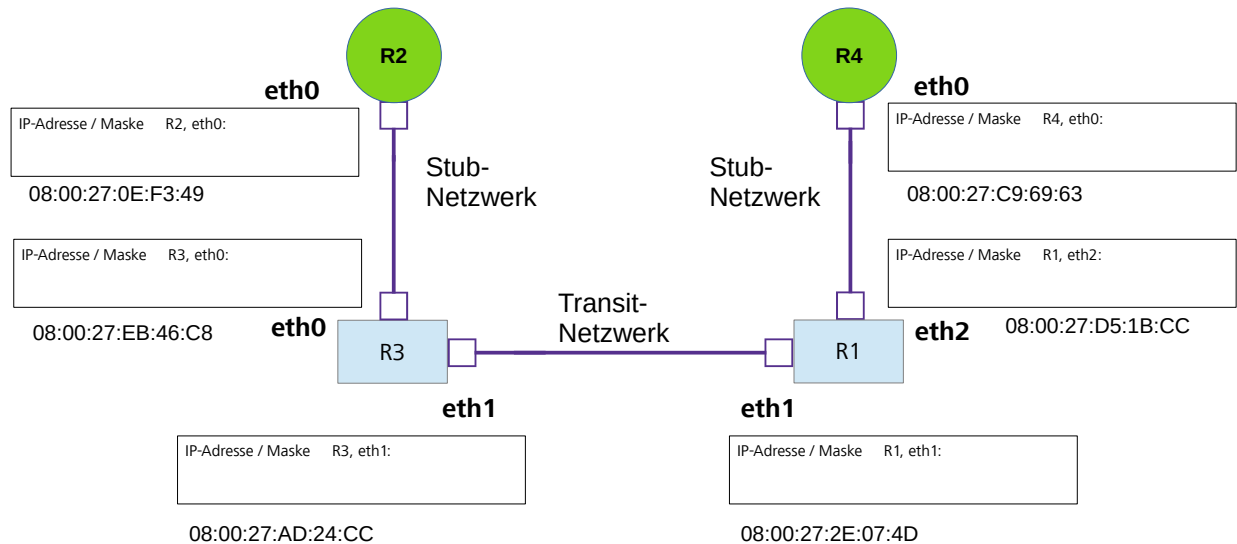
```
systemctl restart zebra.service
```

Um unnötige Probleme mit fehlerhaft konfigurierten IP-Adressen im weiteren Verlauf der Übung zu vermeiden, sollten nach der Konfiguration der Adressen alle einzelnen Verbindungen zwischen benachbarten Rechnern mit `ping` geprüft werden.

7.2 Stub-Netzwerke

7.2.1 Konfiguration des OSPF-Routing

In dieser Aufgabe soll zunächst nur ein Teil des Netzwerkes (mit nur zwei Routern, in der Abbildung beispielhaft die Rechner R1 und R3) in einer einzigen OSPF-Area konfiguriert werden.



Die entsprechenden Einträge werden in der Konfigurationsdatei `/etc/quagga/ospfd.conf` von R1 und R3 vorgenommen.

Es wird eine Authentifizierung zur Absicherung der Kommunikation verwendet; in der untenstehenden Beispielformatung das frei wählbare Passwort `Abc123`, das für jeweils miteinander verbundene Interfaces auf den beiden Routern dasselbe sein muss (zur Vereinfachung sollte für alle Interfaces dasselbe Passwort verwendet werden). Hinweis: Diese Art der Authentifizierung (mit MD5-Hash) wird nur bei der hier eingesetzten Version OSPFv2 verwendet. Die IPv6-Version OSPFv3 setzt stattdessen den „IPv6 Authentication Header“ ein.

Im Abschnitt `router ospf` werden (nur) die **direkt** am jeweiligen Router angeschlossenen Netzwerke mit Netzwerkadresse, Maske und Nummer der Area angegeben. Die Area soll zunächst für alle Router einheitlich sein (also nur *eine* Area; später in Kapitel 7.4 werden zwei Areas konfiguriert). Die Router-ID ist eine beliebige 32-Bit-Nummer im Dezimalformat (wie eine IP-Adresse), die für jeden Router *eindeutig* sein muss (im Beispiel 3.3.3.3 für Router 3). Die tatsächlichen Interface-Bezeichnungen „ethX“ sind zu beachten!

```
hostname R2_ospfd
!
password quagga
!
interface eth0
    ip ospf authentication message-digest
    ip ospf message-digest-key 1 md5 Abc123
!
interface eth1
    ip ospf authentication message-digest
    ip ospf message-digest-key 1 md5 Abc123
!
router ospf
    ospf router-id 3.3.3.3
    network 172.16.1.0/24 area 0.0.0.1
    network 172.16.4.0/24 area 0.0.0.1
```

Anschließend Start des OSPF-Dienstes auf R1 und R3:

```
systemctl start ospfd.service
```

Falls später Änderungen oder Korrekturen an dieser Datei vorgenommen werden, so muss der Dienst mit `systemctl restart ospfd.service` erneut gestartet werden.

Die beiden anderen Rechner (im Beispiel R2 und R4) sollen noch keine OSPF-Konfiguration erhalten; sie haben nur die Funktion eines Rechners innerhalb eines LANs.

Die Rechner R2 und R4 benötigen daher jeweils eine Gateway-Adresse. Eintrag in Yast oder per Kommando:

```
ip route add default via 172.16.x.y
```

(Die Gateway-Adresse für R2 ist diejenige IP-Adresse von R3 an eth0, für R4 diejenige von R1 an eth2)

Eintrag wieder löschen mit: `ip route del default`

Hinweis: Die Festlegung der Gateway-Adresse per `ip`-Kommando muss nach einem Neustart des Netzwerks oder Reboot des Rechners wiederholt werden.

In diesem Übungsteil ist damit ein Netzwerk mit zwei „**Stub-Netzwerken**“ und einem „**Transit-Netzwerk**“ aufgebaut.

7.2.2 Routing-Tabellen

Nach dem Start der beiden OSPF-Router sollte dort nach höchstens zwei Minuten jeweils eine vollständige Routing-Tabelle vorliegen.

Die Standard-Metrik (bzw. die „Interface-Kosten“) pro Netzwerk ist bei Quagga **10**.

Ausgabe der Routing-Tabelle mit dem Kommando `ip route show` (abgekürzt `ip r s`)

Ausgabeformat:

```
172.16.2.0/24 via 172.16.1.1 dev eth0 proto zebra metric 20
```

Alternativ `telnet localhost 2604` (Passwort siehe obige Konfigurationsdatei)

Nach dem Login, Kommando: `show ip ospf route`

Ausgabeformat:

```
172.16.3.0/24 [20] area 0.0.0.1 via 172.16.4.1, eth1
```

Die Gesamt-Metrik ist die Zahl in der eckigen Klammer.

Routing-Tabelle

Netzadresse	Nachbar	Metrik

Test:

Ist das Netzwerk richtig eingerichtet, so sollte das Kommando `traceroute` die vollständigen Wege anzeigen (wird `traceroute` auf R2 mit der Zieladresse von R4 gestartet, so sollte der Weg über die beiden Router R3 und R1 angezeigt werden)

7.2.3 Paket-Analyse

Mit `wireshark` soll nun analysiert werden, wie die Routing-Tabellen entstehen.

Hierzu muss zunächst der OSPF-Dienst auf allen Rechnern gestoppt werden (`zebra` kann weiterlaufen).

Dann wird auf beiden Routern `wireshark` gestartet (Capture „any“, also alle Interfaces) und erst dann werden wieder alle OSPF-Dienste auf allen Rechnern gestartet. Die Aufzeichnung in `wireshark` sollte nach dem Start des letzten OSPF-Dienstes noch mindestens zwei Minuten laufen, bis ein eingeschwungener Zustand im Netzwerk erreicht ist.

OSPF Hello

In der Aufzeichnung sollten zunächst diverse OSPF-Hello-Pakete erscheinen, jeweils *gesendet* vom Router (auf dem `wireshark` läuft) an alle Ausgangsleitungen; empfangen werden Hello-Pakete aber nur von der benachbarten Quelladresse. Die Zieladresse ist immer die OSPF-Multicast-Adresse 224.0.0.5. Mit diesen Paketen wird die Nachbarschaftsbeziehung festgestellt. Die Pakete enthalten (im OSPF-Header und im OSPF-Hello) u.a. die Quell-Router-ID, Authentifizierungs-Informationen, die Area, IP-Adresse mit Maske. Die Rollen „Designated Router“ (DR) und „Backup Designated Router“ (BDR) sind in den ersten Hello-Paketen noch nicht vergeben (zunächst Adresse 0.0.0.0), da sie erst im weiteren Verlauf ausgehandelt werden (es gewinnt derjenige Router mit der höchsten OSPF-Priority, bei gleicher Priority (default 1) gewinnt der Router mit der größten Router-ID; der Router mit der zweithöchsten ID wird BDR).

Informationen aus den OSPF-Hello-Paketen (einschl. OSPF-Header):

Quell-IP	Source OSPF-Router	Area	Active Neighbor	IP Designated Router (DR)	Maske

No.	Time	Source	Destination	Protocol	Length	Info
4009	9299.425609717	172.16.1.1	224.0.0.5	OSPF	100	Hello Packet

Frame 4009: 100 bytes on wire (800 bits), 100 bytes captured (800 bits) on interface any, id 0
Linux cooked capture v1

Internet Protocol Version 4, Src: 172.16.1.1, Dst: 224.0.0.5

Open Shortest Path First

OSPF Header

Version: 2
Message Type: Hello Packet (1)
Packet Length: 48
Source OSPF Router: 1.1.1.1
Area ID: 0.0.0.1
Checksum: 0x0000 (None)
Auth Type: Cryptographic (2)
Auth Crypt Key id: 1
Auth Crypt Data Length: 16
Auth Crypt Sequence Number: 1750693263
Auth Crypt Data: 3f4de4e5c85385b6b7fd04f2157abb1c

OSPF Hello Packet

Network Mask: 255.255.255.0
Hello Interval [sec]: 10
Options: 0x02, (E) External Routing
Router Priority: 1
Router Dead Interval [sec]: 40
Designated Router: 172.16.1.2
Backup Designated Router: 172.16.1.1
Active Neighbor: 4.4.4.4

OSPF Router LSA

Die Nachbarschafts-Informationen aus den Hello-Paketen werden anschließend in „Link State Updates“ an alle anderen Router im Netzwerk (innerhalb einer Area) verteilt. Diese Verteilung kann aufgrund des zu diesem Zeitpunkt noch nicht aktiv laufenden Routings nur per Flooding geschehen, indem die Updates an die Nachbarn geschickt werden und diese die Informationen wiederum an ihre Nachbarn weitergeben. Daher ist die Quelle solcher Pakete immer ein direkter Nachbar, die eigentlichen Informationen können aber von irgendeinem entfernten Router stammen.

Diese Informationen werden in „Link State Advertisements“ (LSA) verpackt, von denen die Typen 1 (Router LSA) und 2 (Network LSA) in der *wireshark*-Analyse in **LS-Update**-Paketen sichtbar sein sollten.

Aus diesen Informationen können die anderen Router in der Area jeweils für sich die Topologie des Gesamtnetzwerkes zusammensetzen und dann selbstständig mittels eines Routing-Algorithmus (Quagga verwendet Dijkstra) die kürzesten Wege berechnen.

In einem **Router LSA** sendet ein Router alle relevanten Informationen zu den an ihm angeschlossenen Netzwerken sowie deren Status („Link Type“). Router LSAs werden nicht über Area-Grenzen hinaus verteilt, siehe Kapitel 7.4.

Während Network LSAs (s.u.) immer nur vom DR eines Netzwerks verschickt werden, werden Router LSAs von allen Routern erzeugt und per Flooding verteilt. Andere Router erhalten damit die Information über Netzwerkadressen sowie den Status eines Netzwerks (z.B. Transit- oder Stub-Netzwerk).

Die ersten Router LSAs (nach dem Starten der OSPF-Dienste) enthalten noch Informationen zu Stub-Netzwerken (Link Type 3), die es gar nicht gibt (im Beispiel die Verbindung zwischen R1 und R3). Grund dafür ist, dass die OSPF-Dienste auf den einzelnen Routern nacheinander gestartet werden und die LSAs anschließend per Flooding verteilt werden. Ein Router hat daher zunächst noch keine Informationen von seinem Nachbarn bekommen und definiert daher das bei ihm angeschlossene Netzwerk als Stub-Netzwerk (das ist ein Netzwerk, über das keine weiteren Verbindungen laufen). Erst später kommen Updates von anderen Routern und es wird damit erkannt, dass ein zunächst als Stub-Netzwerk definiertes Netz doch ein Transit-Netz (Link Type 2) ist.

Es sollten daher für die Analyse nicht die frühen LS-Update-Pakete in der *wireshark*-Aufzeichnung verwendet werden, sondern die letzten.

Analyse der Router LSA (Typ 1): (zu finden in LS-Update-Paketen)

Adv. Router	Link Type	Link-ID (Netz-IP bei Stub , DR-IP bei Transit)	Maske (bei Stub) Interface-IP (bei Transit)	Metrik

wireshark-Ausgabe Router LSA (Beispiel)

```

No.      Time                Source            Destination      Protocol Length Info
   42  29.643717311    172.16.1.2        224.0.0.5        OSPF      288    LS Update

Frame 42: 288 bytes on wire (2304 bits), 288 bytes captured (2304 bits) on interface any, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 172.16.1.2, Dst: 224.0.0.5
Open Shortest Path First
  OSPF Header
  LS Update Packet
    Number of LSAs: 5
    LSA-type 1 (Router-LSA), len 48
      .000 0000 0000 1001 = LS Age (seconds): 9
      0... .. = Do Not Age Flag: 0
      Options: 0x02, (E) External Routing
      LS Type: Router-LSA (1)
      Link State ID: 2.2.2.2
      Advertising Router: 2.2.2.2
      Sequence Number: 0x80000005
      Checksum: 0x9c3c
      Length: 48
      Flags: 0x00
      Number of Links: 2
      Type: Transit ID: 172.16.3.2 Data: 172.16.3.1 Metric: 10
      Type: Stub ID: 172.16.4.0 Data: 255.255.255.0 Metric: 10

```

OSPF Network LSA

In einem **Network LSA** werden alle Router-IDs zusammengefasst, die zu einem Multiple-Access-Netzwerk gehören (also solche Netzwerke, an die mehrere Router angeschlossen sind). In Falle das VM-Netzwerks in dieser Übung sind es immer zwei Router in einem Netzwerk. Ein Network LSA wird immer nur vom DR verschickt.

Ein Network LSA enthält die IDs der an einem Netzwerk angeschlossenen Router (Attached Router), die Link State ID (IP-Adresse des DR) sowie die Maske und die ID des sendenden Routers (Advertising Router). Diese Informationen sollen in der folgenden Tabelle zusammengefasst werden. Es ist zu beachten, dass aufgrund des Flooding-Verfahrens dieselben Informationen mehrfach in verschiedenen Paketen vorkommen können, denn ein LSA wird von einem Nachbarn empfangen (Quell-IP des Nachbarn) und kann dann an den anderen Nachbarn weitergegeben werden (eigene Quell-IP des anderen Netzwerks).

Network LSAs werden nicht über Area-Grenzen hinaus verteilt. Für die Übermittlung von Informationen in andere Areas werden Summary LSAs (Type 3) verwendet, siehe Kapitel 7.4.

Analyse der Network LSA (Type 2): (zu finden in LS-Update-Paketen)

Quell-IP	Adv. Router	Link State ID	Maske	Attached Router	Attached Router

wireshark-Ausgabe Network LSA (Beispiel)

No.	Time	Source	Destination	Protocol	Length	Info
42	29.643717311	172.16.1.2	224.0.0.5	OSPF	288	LS Update

Frame 42: 288 bytes on wire (2304 bits), 288 bytes captured (2304 bits) on interface any, id 0
 Linux cooked capture v1
 Internet Protocol Version 4, Src: 172.16.1.2, Dst: 224.0.0.5
Open Shortest Path First
 OSPF Header
 LS Update Packet
 Number of LSAs: 5
 (...)
LSA-type 2 (Network-LSA), len 32
 .000 0000 0000 1010 = LS Age (seconds): 10
 0... .. = Do Not Age Flag: 0
 Options: 0x02, (E) External Routing
 LS Type: Network-LSA (2)
 Link State ID: 172.16.3.2
 Advertising Router: 3.3.3.3
 Sequence Number: 0x80000001
 Checksum: 0x4731
 Length: 32
 Netmask: 255.255.255.0
 Attached Router: 2.2.2.2
 Attached Router: 3.3.3.3

Routing-Informationen

Alle zuvor analysierten Informationen aus den Link-State-Paketen können im Quagga-Dienst abgerufen werden. Hierzu ist eine telnet-Verbindung zum lokalen OSPF-Dienst auf TCP-Port 2604 erforderlich:

telnet localhost 2604 (Passwort quagga bzw. entsprechend der Einstellung in ospfd.conf)

Kommandos:

```
show ip ospf route
show ip ospf neighbor
show ip ospf database network
show ip ospf database router
```

7.3 Transit-Netzwerke

Das OSPF-Routing unterscheidet Transit-Netzwerke und Stub-Netzwerke. Über Transit-Netzwerke können andere Bereiche des Netzwerkes erreicht werden, während Stub-Netzwerke die Endpunkte eines Netzwerkes darstellen. Diese Situation wurde im vorangegangenen Aufgabenteil hergestellt.

Nun sollen alle vier Rechner als Router konfiguriert werden, sodass ein Netzwerk aus mehreren Transit-Netzwerken entsteht. Entsprechend Kapitel 7.2.1 muss die OSPF-Konfigurationsdatei `/etc/quagga/ospfd.conf` auf den weiteren Rechnern (im Beispiel R2 und R4) erstellt und der OSPF-Dienst dort gestartet werden.

Die Gateway-Adressen auf diesen beiden Rechnern müssen entfernt werden; bei festem Eintrag: `Yast`, bei manuellem Eintrag (mit dem `ip`-Kommando): `ip route del default`

Nach dem Start aller OSPF-Dienste sollte das Routing im gesamten Netzwerk funktionieren, d.h. das `traceroute`-Kommando sollte alle kürzesten Wege von einem Router zu einem anderen korrekt anzeigen.

Wie zuvor können (nun auf allen) Routern die Routing-Informationen ausgelesen werden.

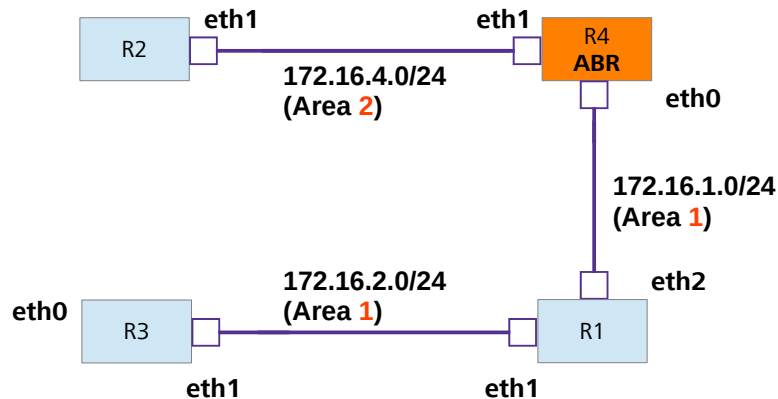
Routing-Tabelle (eines der Router)

Netzadresse	Nachbar	Metrik

In der *wireshark*-Analyse sollten jetzt nur noch Router-LSAs mit dem Link-Typ 2 (Transit-Netzwerke) sichtbar sein sowie Network-LSAs von dem jeweiligen Designated Router der Mehrfachzugriffsnetze.

7.4 Area Border Router ABR

In den bisherigen Aufgabenteilen wurde nur eine einzige OSPF-Area verwendet. Beim Routing-Verfahren OSPF können jedoch sehr große Netzwerke in kleinere Teile – die „Areas“ – zerlegt werden. Das Link-State-Routing wird hierbei nur innerhalb der einzelnen Areas eingesetzt. Die Areas werden mittels Area Border Router ABR miteinander verbunden. Die ABR kennen jeweils beide angeschlossene Areas und können die Informationen über die jeweils im anderen Netzwerk verfügbaren IP-Netzadressen weiterleiten. Die Router innerhalb der Areas sind somit nur über die Netzwerkadressen (ggf. nur zusammengefasste Adressbereiche) und den ABR informiert, sie kennen jedoch nicht die innere Struktur der anderen Areas.



In diesem Übungsteil sollen alle Rechner als Router eingesetzt werden, wobei es nun **zwei Areas** geben soll. Auch in obiger Abbildung ist die Verwendung der Router und Netzadressen wieder nur *beispielhaft* dargestellt.

Änderungen an den `ospfd.conf`:

- Die Netzwerkadresse für die Verbindung zwischen R2 und R3 muss aus der OSPF-Konfiguration dieser *beiden* Router entfernt werden (entweder die entsprechende Zeile jeweils löschen oder mit einem Ausrufe-Zeichen am Zeilenanfang auskommentieren, dann den Dienst neu starten).
- Der Router R2 liegt in der Abbildung in Area 2, daher muss die `ospfd.conf` entsprechend angepasst werden.
- Der Router R4 übernimmt die Rolle des Area Border Routers (ABR); er benötigt zusätzliche Konfigurationseinstellungen:

```

hostname R4_ospfd
!
password quagga
!
interface eth0
    ip ospf authentication message-digest
    ip ospf message-digest-key 1 md5 Abc123
!
interface eth1
    ip ospf authentication message-digest
    ip ospf message-digest-key 1 md5 Abc123
!
router ospf
    network 172.16.1.0/24 area 0.0.0.1
    network 172.16.4.0/24 area 0.0.0.2
    ospf abr-type standard
! area 0.0.0.1 range 172.16.0.0/22

```

Die Adressen sind **nur beispielhaft** angegeben: Das Netzwerk zwischen R2 und R4 gehört zur Area 2, das andere Netzwerk zur Area 1. Die auskommentierte Zeile (`! area 0.0.0.1 ...`) *kann* in der Konfiguration optional hinzugefügt werden, *wenn* sich die Netzadressen mit einer geeigneten Maske zusammenfassen lassen (im Beispiel werden die Adressen 172.16.0.0 bis 172.16.3.0 mit der Maske /22 zusammengefasst).

Wie zuvor soll zunächst `wireshark` gestartet werden (nicht auf dem ABR), anschließend die OSPF-Dienste auf allen Routern.

In der `wireshark`-Analyse sollten nun (in einigen LS-Updates) zusätzlich zu den bekannten Router- und Network-LSAs auch **Summary-LSAs (LSA Type 3)** erscheinen, die nur vom ABR gesendet werden. Die Router LSAs enthalten nur noch Informationen über die Adressen der jeweiligen Area. Alle Router innerhalb einer Area kennen daher nur noch die anderen Router und Adressen der eigenen Area.

Mithilfe der Summary-LSAs des ABR werden die zusammengefassten Adressen der anderen Area übermittelt, sodass in der Routing-Tabelle wieder alle Adressen dargestellt werden können.

Routing-Tabelle eines *nicht*-ABR-Routers R2:

`telnet localhost 2604` dann Kommando `sh ip ospf route`

```

R2_ospfd> sh ip ospf route
===== OSPF network routing table =====
N IA 172.16.1.0/24      [20] area: 0.0.0.2
                        via 172.16.4.2, eth1
N IA 172.16.2.0/24      [30] area: 0.0.0.2
                        via 172.16.4.2, eth1
N   172.16.4.0/24      [10] area: 0.0.0.2
                        directly attached to eth1

===== OSPF router routing table =====
R   4.4.4.4            [10] area: 0.0.0.2, ABR
                        via 172.16.4.2, eth1

```

Der Netzwerke 172.16.1.0 und 172.16.2.0 sind bekannt und als Inter-Area-Routen (IA) gekennzeichnet. Der Router 4.4.4.4 ist als ABR bekannt. Die Topologie der Area 1 ist jedoch nicht bekannt, d.h. bei Router R2 sind die Router R1 und R3 nicht bekannt (siehe auch Kommando `sh ip ospf database`).

Alle Wege führen über den ABR (hier R4), der beide Areas kennt:

```
R4_ospf> sh ip ospf route
===== OSPF network routing table =====
N    172.16.1.0/24      [10] area: 0.0.0.1
                                directly attached to eth0
N    172.16.2.0/24      [20] area: 0.0.0.1
                                via 172.16.1.1, eth0
N    172.16.4.0/24      [10] area: 0.0.0.2
                                directly attached to eth1
```

wireshark-Ausgabe Summary LSA (Beispiel)

No.	Time	Source	Destination	Protocol	Length	Info
702	1687.715259289	172.16.4.2	224.0.0.5	OSPF	106	LS Update

Frame 702: 106 bytes on wire (848 bits), 106 bytes captured (848 bits) on interface eth1, id 0
 Ethernet II, Src: PCSSystemtec_5e:e4:c6 (08:00:27:5e:e4:c6), Dst: IPv4mcast_05 (01:00:5e:00:00:05)
 Internet Protocol Version 4, Src: 172.16.4.2, Dst: 224.0.0.5
 Open Shortest Path First

OSPF Header
 LS Update Packet
 Number of LSAs: 1
 LSA-type 3 (Summary-LSA (IP network)), len 28
 .000 0000 0000 0001 = LS Age (seconds): 1
 0... = Do Not Age Flag: 0
 Options: 0x02, (E) External Routing
 LS Type: Summary-LSA (IP network) (3)
 Link State ID: 172.16.2.0
 Advertising Router: 4.4.4.4
 Sequence Number: 0x80000002
 Checksum: 0x1e5b
 Length: 28
 Netmask: 255.255.255.0
 TOS: 0
 Metric: 20

Die vom ABR übermittelte Metrik (hier 20) zählt ab dem ABR.